

This is an excerpt from Frank Elavsky's dissertation on *Tool-making as an Intervention on the Accessibility of Interactive Data Experiences*, which can be accessed in full at this archival link (once published):

<http://reports-archive.adm.cs.cmu.edu/anon/hcii/CMU-HCII-26-103.pdf>

This document contains the following sections:

- Table of contents
- Chapter 1: What do we mean by “accessibility” and “tools?”
- Chapter 4: *Chartability*: Heuristics as a Tool and Resource
- Chapter 6: *Skeleton*: Visual Authoring of Non-visual Data Experiences
- Chapter 9: Findings (Sections 9.1, 9.2, and 9.6 only)
- Chapter 10: Discussion & Future Work (Sections 10.2 and 10.3 only)
- References

This document does not contain the following chapters:

- Chapter 2: Background & Related Work
- Chapter 3: Overview of Contributions
- Chapter 5: *Data Navigator*: Low-level Tooling for Creating Navigable Data Structures
- Chapter 7: *Cross-perception*: Rethinking Input Design Towards Blind Analytical Interaction
- Chapter 8: *Softerware*: Enabling Personalization of Interactive Data Representations for Users with Disabilities
- Chapter 12: The *Softerware* Option Taxonomy
- Chapter 11: Biographical Sketch

Contents

I	Introduction	1
1	What do we mean by “accessibility” and “tools?”	3
1.1	Is “accessible visualization” really an oxymoron?	3
1.2	On <i>tools</i> , <i>tool-making</i> , and <i>human-tool</i> interaction	5
II	Evaluation: Helping Practitioners Identify Accessibility Barriers	7
4	Chartability: Heuristics as a Tool and Resource	9
4.1	Preface	9
4.1.1	Context	9
4.2	Overview	12
4.3	Existing Work in Data Visualization and Accessibility	13
4.3.1	Research Advancements in Data Visualization and Accessibility	13
4.3.2	Accessibility Practices in Data Visualization Tools and Libraries	14
4.3.3	Accessibility in Practice, Broadly	15
4.3.4	Using Heuristics to Break Into Under-addressed Areas	15
4.4	Making Chartability	16
4.4.1	Compromising	17
4.4.2	Assistive	18
4.4.3	Flexible	18
4.5	Using Chartability	18
4.5.1	Visual Testing (Perceivable)	19
4.5.2	Keyboard Probing (Operable, Assistive)	19
4.5.3	Screen Reader Inspecting (Perceivable, Operable, Robust, Assistive)	20
4.5.4	Checking Cognitive Barriers (Understandable, Compromising)	20
4.5.5	Evaluating Context (Robust, Assistive, Flexible)	23
4.6	Validating Chartability	23
4.6.1	Pre-Validations and Flipped Roles: Participants Question <i>Me</i>	24
4.6.2	Discovering “Critical” Heuristics	25
4.6.3	Selecting Participants and Projects	25
4.7	Study Results	26
4.7.1	Real Access has more Considerations than Colorblindness	27
4.7.2	Audits are Slow, but Help me Focus	27
4.7.3	Chartability Helps me Remember and Stay Consistent	28

4.7.4	Access is an Experience, not just Compliance	28
4.7.5	Everyone wants More Evaluation Resources and Tools	28
4.7.6	Experts: “Novices will Struggle.” Novices: “This was so helpful”	28
4.7.7	What about Auditors with Disabilities?	29
4.8	Extended Results	29
4.9	Discussion	30
4.10	Conclusion	31
4.11	Postface	32

III Navigation: Making Data Structures Traversable 33

6	<i>Skeleton: Visual Authoring of Non-visual Data Experiences</i>	35
6.1	Preface	35
6.1.1	Context	35
6.2	Overview	37
6.3	Related Work	39
6.3.1	Visualizing Non-visual Structure	39
6.3.2	Non-visual Data Experiences	39
6.3.3	Authoring Non-visual Data Experiences	40
6.4	Co-design Foundations	41
6.4.1	Geologic Map	42
6.4.2	Design System Library	43
6.4.3	Open Source Visualization Library	43
6.4.4	Infrastructure from Practice	44
6.5	<i>Skeleton: System Design</i>	50
6.5.1	Staging: Input and Preparation for Authoring	50
6.5.2	Edit: Interacting with Topology, Layout, and Semantics	51
6.5.3	Test: Debugging Interaction Interactively	53
6.5.4	System Implementation: Technical Details	54
6.6	User Study	56
6.6.1	Participants	58
6.6.2	Procedure	58
6.6.3	Analysis	59
6.7	Results	59
6.7.1	Seeing Navigation Made Structural Problems Legible as Design Problems	59
6.7.2	Practitioners Developed a Designerly Interest in What Constitutes Good Navigation	60
6.7.3	Iteration Was Substantive, Self-directed, and Concentrated on Semantics	61
6.7.4	Seeing Navigation Prompted Practitioners to Reconsider the Architecture of Their Own Charts	62
6.7.5	Experiencing Keyboard Navigation Surfaced a Broader Range of Users and Input Technologies	62
6.8	Discussion	64

6.8.1	Visibility as a Precondition for Iteration	64
6.8.2	From Compliance to Design	64
6.8.3	Bespoke Visualizations as an Unaddressed Accessibility Research Problem	65
6.8.4	What Visualization Owes Accessibility	65
6.9	Limitations and Future Work	66
6.10	Conclusion	67
6.11	Postface	68
VI	Conclusion	69
9	Findings	71
9.1	Access Barriers Are Produced for Practitioners, Both With and Without Disabilities	71
9.2	Tools Can Reduce the Work Required to Make Things Accessible	72
9.6	The Hardest Barriers Required Intervention at the Substrate	72
10	Discussion & Future Work	75
10.2	“Applied” accessibility work and <i>low research</i>	75
10.3	Who is responsible for repair?	76
	References	79

List of Figures

4.1	A low contrast chart (left) compared to a higher contrast version (right). A drop-per tool is extracting the fill color of the bar and then a contrast ratio has been calculated. Note that the fill color is the same on both bars, but darker borders have been added to ensure the visualization passes contrast tests.	19
4.2	Keyboard navigation paths on a stacked bar chart. The left shows a serial navigation example, typically just a default of rendering order. The right shows both groups (the stack of bars) and categories (the color/texture shared among bars across stacks) as dimensions to explore laterally or vertically.	20
4.3	A mouse cursor is selecting a bar (left, shown with a thick indication border) in a stacked bar chart to filter a dataset (on the right). A system alert (red box) notifies the user of their interaction result. This selection capability must also be provided for the keyboard interface and the alert must be announced to screen readers.	21
4.4	Charts must have a visually available textual explanation provided that summarizes the outcome. “Client Registration Chart” for “Product X” (left) is inaccessible while “New Product Launch a Success” (right) gives a clear takeaway. . . .	21

4.5	An interactive chart displaying only “Image” as semantic information with no feedback provided on selection. The robust semantics given to a screen reader, “toggle button” (middle) as well instant feedback, “selected” (right) are considered proper semantics for an interactive experience.	22
4.6	A line chart (left) with a single line and an accompanying data table (right). This line chart would not provide enough low-level information about each datapoint without the table provided. A table alone however would also be inaccessible. Providing both can satisfy conflicting accessibility needs for different audiences.	22
4.7	A scatterplot with many points, where a single point within the chart can be accessed by a screen reader (left). Navigating this data piece by piece is unnecessarily tedious, so an annotation callout is provided to help the reader focus on an outlier cluster (right). The callout is being accessed by a screen reader, which is displaying the annotation’s summary as well.	23
4.8	A bar chart with categories (left) shown not conforming to Windows High Contrast White Mode. High contrast mode on Windows requires limiting color palettes, using only black or white for most elements (shown on the right).	24
6.1	<i>Skeleton</i> being used to build a navigation structure over a line chart. A : a dual tree visualization of nodes and edges built for a line chart with 3 lines, spanning over 12 months. B : a direct-manipulation canvas that resides over a static png image of a line chart. C : a skeleton-like structure of nodes, edges, and group outlines that correspond to the schema shown in (A), overlaid on the visual space of the chart in (B). D : controls for rapidly scaffolding the nodes, edges, and their shapes shown in (C), using Vega’s visualization engine for automatically generating spatial properties.	37
6.2	Our visual design work in Figma over a static geologic infographic map of Wisconsin. We use visual forms and illustrations over and beside the map to communicate flows, structure, navigation styles, and interaction patterns.	42
6.3	Input data transformed into a navigable structure using the <i>Dimensions API</i> and visualized with our <i>Inspector</i> gadget (left). The input chart (middle). The navigable structure is transformed and drawn over the chart using the <i>Scaffolding Engine</i> (right).	51
6.4	Group label pattern builder, including an array of aggregate summary options, template formatter field, and preview.	53
6.5	Computed layout for marks via Vega’s visualization engine plus group outlines (left) and with padding estimation using our non-ML computer vision approach (right) for various types of common visualizations.	57
6.6	Re-creation of P8’s moment of realization, placing nodes manually: not every element in their voronoi pie chart <i>should</i> be navigable.	63

Part I

Introduction

Chapter 1

What do we mean by “accessibility” and “tools?”

This thesis is a body of research situated within the existing research area focused on making interactive data representations more accessible for people with disabilities. Much of the work in this existing area is situated within the context of making interactive data *visualizations* accessible, particularly (but not exclusively) for people who are blind. Our work, contributed here in this thesis, is focused on using *tools* as a specific intervention and sub-area of study for making interactive data *representations* accessible for people with disabilities, broadly speaking. (“Representations” here is an intentionally broader term than “visualizations,” which are exclusively visual representations of data.)

Before we begin, two things must be understood up front, or else the rest of this thesis could be interpreted with disruptive assumptions: we must interrogate the phrase “making visualizations accessible” and unpack why *tools* are a meaningful area of study.

1.1 Is “accessible visualization” really an oxymoron?

The first assumption that must be disrupted is perhaps the motivating cornerstone of this research, which is that the phrase “making visualizations accessible,” while a noble goal, is not the semantically correct phrasing nor precisely what describes our work. This can be misleading. We do use the phrase “accessible visualization” but will admit that this seems to confuse certain people with very particular opinions about things. We will clear this up.

Villains of our field’s past have written incendiary and ableist perspectives on why “no forms of data visualization, not just dashboards jam-packed with graphics, can be made fully accessible to someone who is blind,” and that “[a blind man] will never be able to analyze data as I do visually, because many aspects of vision cannot be duplicated by his other senses” [48]. However, this position misunderstands what the goal of accessibility is, and arguably even what the goal of visualization itself is.

Making visualizations accessible *isn’t* about the visualization, it’s about making the outcomes of the visualization accessible.

Visualizations are ubiquitous and paramount for decision-making. However, the *artifact* that is a visualization is not even the goal of the act of visualizing: developing understanding, insight, confidence, and communication among and between human beings are the goals of visualization. Visualization is about making data easier to use for all kinds of things. Yes, our visual system enables us more than any other form of sensory cognition that we have [19, 60, 189]. But we aren’t trying to make sight itself accessible. We are trying to make it possible for people to make meaningful decisions, gain valuable information, build conjectures, and effectively communicate with others.

Many, many people who I, Frank, have spoken to over the course of my career, even before embarking on this thesis journey, misunderstand this simple fact: making a “visualization” accessible *isn’t* about the visualization itself but rather making what the visualization is meant to

accomplish accessible. It's about equal outcomes, not equal interactions with an artifact.

People with disabilities are no small portion of the world's population. In the United States, 26% of people self-report living with at least one disability that affects their daily lives [140] and all of us will eventually age into disability (if we are lucky to live a long life).

People with disabilities deserve to participate fully in life. They deserve financial independence. They also deserve loving care and interdependence [7, 83, 128]. People with disabilities have a right to make informed decisions, to know about the status of a global pandemic, and to have an understanding of local and national politics [46]. While we use visualizations to navigate all of these domains, the goal is not to make the charts and graphs themselves somehow equally useful to all people. That would be a false measurement of success.

Our goal then, is measured by the success of lives led by people with disabilities [196]. Many other measurements are just metrics along the journey towards that goal. We then ask: Can people with disabilities also use data to live full lives? Can they make *fast* decisions based on data? Meaningful, careful, *slow* decisions? Communicate complex ideas? Crunch and clean data, develop models, find errors, and build hypotheses? Can they have memorable, immersive, beautiful, aesthetic experiences with data too [83]? Ultimately, our aim is not accessible "visualizations" but equitable and accessible *outcomes*, everything an interactive data experience *accomplishes* for the people who use it.

Again, if the goal of accessible visualization were about visualizations themselves, then the correct course of action would be one framed by the medical model of disability [119]: that there is a normative state of behavior and capability (in this case, it would be "normal" to be able to read a visualization and make a decision) and any deviation from that norm must be corrected. This framing first assumes that the visualization should not be altered or improved. And then this framing puts the burden on the bodies of people with disabilities: that they must be "fixed" and given sight or brought to some equivalent state as someone who is "healthy," normal, and sighted. Plenty of scholars have already discussed why this framing is a problem, not only because it places undue burden on people with disabilities and produces pathologies and hierarchies of disability, but also because it is fundamentally not economically or ethically feasible.

So we then turn to other models of disability, such as the social model. The social model is heavily discussed by disability scholars and is not the end-game or last and total way of thinking about disability [119, 152, 154, 187, 222]. But the core motivation is that society, not medicine, is also a path towards solving problems that people with disabilities face. A few important concepts and concretely actionable things come from the social model that can help motivate the work of this thesis.

First, we look to the historical birth of the social model of disability: in the 504 sit-ins that took place in the United States in 1977. Cities had curbs and curbs are a barrier for people who use wheelchairs. So protests happened because decisions were being made without people with disabilities at the table. In this instance, people acknowledged that political power was an exclusive club and fought to ensure their cry "nothing about us, without us!" materialized.

And this leads us to the first and most-foundational philosophical framing for this thesis: that our *artifacts*, these things we've created from curbs to data visualizations, can become *barriers* for people with disabilities. And it is then the artifact, not the body of the person with a disability, where disability is produced in this model. Rather than a comparison to a normative state as a

way to frame disability (the medical model), we instead must observe and evaluate material outcomes based on human-made problems.

So, the social model is framed around society “solving” inequities: we get involved and make political and legal change tangible. But a second model also emerges from within the social model: one where we can now frame *who is first responsible* for repair: the curb designers and implementers.

And knowing who is first responsible for access leads us into the moral and ethical imperative that motivates this thesis: the builders and makers of visualizations are ultimately the ones who provide exclusive value for only a subset of people: those *without* disabilities. **We must first change how builders and makers do their work.**

So the phrase “accessible visualization” is really about recognizing that visualizations produce barriers for people. That means that it is our ethical imperative, as builders and makers, to fix them. And that act of fixing barriers leads us away from mere visual representations of data into a wide variety of other senses and interaction modalities. There are many paths forward towards fuller and more-equitable lives led by people with disabilities.

1.2 On *tools*, *tool-making*, and *human-tool* interaction

Then the act of making becomes immensely important: we, the builders and makers of our world, need to get things right; there is a risk involved when making things that we will exclude people with disabilities. We need to make sure that we build a better world than the one we have now. We must care for new things we create and tend to the repair and maintenance of what we’ve already made. And this ethical imperative leads us to the topic of *tools* and *tool-making*.

So the second thing that must be understood before we embark on this thesis is that *tools* are not the same as *solutions* or *applications*. Sometimes tools can be used to *solve* things and are certainly, in ideal circumstances, *applied* in various contexts. But understanding the role of the “tool” in human-tool interaction is paramount for engaging in the work of making anything accessible for people with disabilities.

We use tools to shape our world, break old things, and make new things. But a tool, like the hammer (as an example), does not inherently *solve* something like homelessness. But a hammer can be used to build homes if there are social policies in place and proper resources allocated. This means that for the success of tooling, there is often a larger material, social, legal, and policy reality that supports and necessitates those tools. This thesis will not be focusing on changing the upstream dependencies, but optimistically operating as if they were true (or will be true in time).

However, in some cases, tools can *destroy*. The hammer has a claw and can easily pry apart boards and tear down homes. So tools carry potential to do all kinds of things, both good and bad, and how a tool is used is often open-ended, variable, and heavily dependent on socio-technical realities. Tools participate in personal and political agendas [217] and are sometimes, for this reason, regulated or made proprietary and controlled by powerful entities [65, 211].

So tools are not without any sort of ethics. We cannot just blame tool-users for outcomes when much of a tool depends on these larger systems and structures. Technologies (tools included) encode the assumptions and biases of their *creators* as much as, if not more than, their

users. Tools that build things for others to use can be loaded with assumptions about what people are *able* to do [218] and also rules and guardrails about what anyone downstream from that tool’s design *should* do [65, 210]. These assumptions, biases, and rules *limit*, *enforce*, *magnify*, *exclude*, and *enable* what a tool-user is capable of.

Tools for visualizing data are a perfect case study in this problem: virtually every major data visualization library, application, or software ever made was made entirely with the assumption that data should be transformed into visual representations. This is a reasonable assumption, since virtually all of the tool-makers are sighted and visualization is incredibly helpful to our cognition of and communication with data [55].

So data visualization, as a field, has focused its tool-making efforts on reducing the difficulty involved in visualizing data. Some visualization tools are concise [162], others are lower level but much more expressive [14]. Tool-making in visualization has focused on making it easier to scaffold a wide variety of interactions both with the visualizations as well as with their underlying data models [74].

However, as time has moved on, people began to speak out about color-vision deficiency in data visualization. Some people, primarily those with X/Y chromosomes (largely men) who are of European ancestry, have a deficiency in their ability to perceive certain colors. Then a plethora of research arose that began to look into the barriers that folks who are colorblind face in data visualization. As a result, our practices and tools improved. We began to educate practitioners, develop new color palettes, researched new methods for testing our designs, and built new systems for handling automatic color encoding. Our tools evolved.

But now data visualizations have arguably become ubiquitous in daily life. By comparison, we have far more tools now for making visualizations quickly and easily than we do for representing data in non-visual ways. We also have far more research, relatively speaking, into how sighted end users interact with visualizations.

So this thesis engages gaps that arise in this space: Practitioners face immense challenges when crafting accessible data experiences. We first need to educate practitioners on what accessibility barriers actually are in interactive visualizations. Then, we must help them engage the hardest barriers in this work and create building blocks that help them to construct navigable data experiences, build design frameworks that can inform entirely new kinds of data interaction, and develop software systems for end-user personalization and agency. Our research seeks to advance the state of the art in tools that assist in accessible data interaction while also using tool-making as an intervention that helps us to better understand and characterize *why* and *how* data practitioners face barriers themselves in this work.

Part II

Evaluation: Helping Practitioners Identify Accessibility Barriers

Chapter 4

Chartability: Heuristics as a Tool and Resource

4.1 Preface

Novices and experts have struggled to evaluate the accessibility of data visualizations because there are no common shared guidelines across environments, platforms, and contexts in which data visualizations are authored. Between non-specific standards bodies like WCAG, emerging research, and guidelines from specific communities of practice, it is hard to organize knowledge on how to evaluate accessible data visualizations.

In this chapter, we present *Chartability*, a set of heuristics synthesized from these various sources which enables designers, developers, researchers, and auditors to evaluate data-driven visualizations and interfaces for visual, motor, vestibular, neurological, and cognitive accessibility. In this paper, we outline our process of making a set of heuristics and accessibility principles for *Chartability* and highlight key features in the auditing process.

Working with participants on real projects, we found that data practitioners with a novice level of accessibility skills were more confident and found auditing to be easier after using *Chartability*. Expert accessibility practitioners were eager to integrate *Chartability* into their own work. Reflecting on *Chartability*'s development and the preliminary user evaluation, we discuss tradeoffs of open projects, working with high-risk evaluations like auditing projects in the wild, and challenge future research projects at the intersection of visualization and accessibility to consider the broad intersections of disabilities.

4.1.1 Context

Most of the prefaces for each chapter tie the overall narrative together, and help kick start each chapter with an explanation and perspective that help motivate why the problem mattered to address, what interesting elements ultimately shaped the start of the work, and even some caveats or scoping concerns to the work worth mentioning up front.

But *Chartability* started the thesis and all of the work branched from here. So what started *Chartability*? A long time ago, I (Frank) first did a “Research Experiences for Undergraduates” (REU) hosted by the National Science Foundation at Clemson University. This was long ago, before my industry career in visualization. The REU Site focused exclusively on data visualization. It had various projects a student could choose from, one of which was focused on helping a humanities scholar, Gabriel Hankins, visualize a network of letters sent between historical figures. I had just gotten a copy of Tamara Munzner’s *Visualization Analysis and Design* [133] and Johanna Drucker’s *Graphesis* [38], hot off the press. I felt ready for that project. But because it was an REU Site, it had many different areas of expertise besides humanities and visualization in the program. And I was assigned a secondary advisor for the summer, Jill Gemmill, who wanted me to learn more about *graph visualization* and *color vision deficiency*. So my first foray into visualization was a mix of humanities, graphs, and accessibility.

I carried that work with me into a career in visualization. I was interested in accessibility, but only slowly gained little bits of expertise here and there as I matured as an engineer and designer: add alternative text (and keep it short), don't make your text too small or light, use clear, summary titles, and so on. I collected this set of tacit knowledge on accessibility over the years. But at Visa, later in my career, I had a mandate to make a design system library of chart components as accessible as possible by default. And I had just had success integrating motion design into the library, so I was looking for my next project to tackle at a deep level. It was a perfect topic.

But digging in to what it meant to make the most-accessible visualization library possible led to many dead ends and impotent guidelines. In the Web Content Accessibility Guidelines, contrast ratios frustrated me to no end. I knew we had good research in visualization on how to discriminate between elements based on size [188], but accessibility standards didn't align with the models in the latest research. And on broader topics, there is only so much applicable knowledge you can gain from reading 30, 40 articles like "top 5 tips for making your charts more accessible" or from accessibility research papers where they built some kind of obscure dogle and tested it and wrote guidelines for other researchers along the lines of, "ensure your dogle's focus levels are maximum dogle" or some even less-useful collection of knowledge.

I needed to know answers to specific, grounded, concrete questions. And I was shocked: is there really no clear set of low-level, detailed guidelines for how someone should actually do this work? We have so much accessibility knowledge spread out in different places, but none of it was really designed to be used in the context of visualization work.

So, I gathered a group of professionals (now friends) interested in the topic and we met almost monthly for nearly two years on the topic of visualization and accessibility. It was a combination of folks from that group, as well as others who were members of groups that I was a part of but didn't organize (on the topic of accessibility more broadly) that ultimately contributed to the work that became the first version of *Chartability*. The research paper we published attributes authorship to a small subset of the full scope of people who contributed to the project itself. When I talk about *Chartability*, I tend to differentiate between the *project* and the *paper* where we talk about how we made the project and tested it.

And testing it was interesting. We don't go into deep detail in the paper on this, but I wasn't affiliated with CMU at the time and didn't even anticipate ever writing a research paper on *Chartability*. But I wanted to see how people worked with it and designed what I figured was a minimal sort of study to see how people used it and how it influenced their work. We had *many* participants in that and I did quite a lot of work over those two years with random folks, many who will remain anonymous.

But when we got around to writing the paper, we had to go back and ask our old participants: "would you consent to being in our paper as a participant?" and many (most, in fact) said they wouldn't. This is because earnest accessibility work is about finding flaws, not solutioning. And I think that the various lawyers and marketing people who got wind of our work loved the free (or heavily subsidized) labor but didn't want the company or product to look bad in the process of a research paper coming out. I don't really have regrets in this regard, because the paper was still published. But at the time of our first rejection, I had doubts about whether I should just run a whole new study from scratch.

This project does say a bit about how research tends to, by the nature of how it operates as a

productive profession of its own, disincentivize certain kinds of work. I doubt many researchers in the academy would intentionally sign an NDA to run a study and gather data that they can't ever publish (even though this is largely how product research and many professional, non-academic researchers presently operate). But I was able to see the real inner workings of some complex visualization products and ecosystems that I wouldn't have been able to if my goal was to publish a paper from the start. But instead, my goal was to learn about how to make *Chartability* better and learn more about how people used it (which very much seems like the same framing as a research paper!). But because I wasn't actively planning to publish anything at the time, I gained collaborators that I wouldn't have had otherwise (some of whom I still chat and share ideas with to this day).

Another thing that was quite difficult about *Chartability* was finding people with expertise in visualization *and* accessibility *and* lived experience with a disability *and* who were working on a visualization project *and* willing to let me see what they were up to. Recruitment was tough for this. I wouldn't do it again, especially now that I am a researcher. When people first interacted with me, the spirit was generous: I'm giving them this resource and my time and they would give me their time and even potentially company secrets. But ironically, now that I am a researcher and my goal is open dissemination of knowledge, it is more difficult to work with certain people.

But *Chartability* sets the tone for this whole thesis: our work is for the data practitioners, whoever they are. I was one. And at first, *Chartability* was for myself. So, we carry this spirit into all of the projects of this thesis, including many of the risks involved when doing research in this way. *Skeleton* (in Chapter 6) unpacks this further as well: *action research* [72], when conducted true to its spirit, operates first and foremost from the problems, framings, and imaginations of a particular group or community first, and not solely from the outside perspective of a researcher. In that sense, this thesis earnestly engages the barriers that data practitioners face when they design, develop, analyze, and architect data experiences.

This chapter was adapted from our published paper:

F. Elavsky, C. Bennett, and D. Moritz, ‘How accessible is my visualization? Evaluating visualization accessibility with Chartability’, *Computer Graphics Forum*, vol. 41, no. 3, pp. 57–70, Jun. 2022.

4.2 Overview

26% of people in the United States self-report living with at least one disability [140]. Of those, 13.7% live with a mobility disability and 10.8% with a cognitive disability. Globally, the World Health Organization reports that 29% of the world lives with uncorrected or uncorrectable blindness, low vision, or moderate to severe visual impairment [142]. Access is a significant inclusion effort that has broad international impact, especially for data visualization.

Accessibility is the practice of making information, content, and functionality fully available to and usable by people with disabilities. As part of this process, practitioners need to be able to identify accessibility barriers. While general accessibility standards help, evaluating the inaccessibility of complex data systems can be a daunting and often expensive task. State-of-the-art automated compliance checkers only find 57% of accessibility errors [36], meaning accessible experiences must still be manually designed and checked for quality. And following standards may only account for up to half of the needs of people with disabilities [148] anyway. Additionally, the intended wide applicability of these general standards means they fall short for information-rich systems, such as data visualizations (which use size, color, angles, shapes, and other dimensions to encode information). These specific contexts, communities, and libraries that deal with data visualizations and information-rich interfaces often have their own tools and guidelines for use, but they seldom include accessibility. Finally, research at the intersection of data visualization and accessibility has yet to meaningfully permeate data visualization tools and communities and primarily focuses on blindness and low vision, neglecting diverse accessibility needs of people with other disabilities.

Synthesizing evolving accessibility standards, research findings, and artifacts from communities of practice into usable knowledge for a specific, evolving domain is a wicked problem. To address this, we present Chartability. Chartability is an accessibility evaluation system specific to data visualizations and interfaces which aims to help practitioners answer the question, “how accessible is my data visualization?” Chartability organizes knowledge from disparate bodies of work into testable heuristics based on the functional accessibility principles POUR (Perceivable, Operable, Understandable, and Robust) [205] and 3 novel principles CAF (Compromising, Assistive, and Flexible), which we added to attend to the unique qualities and demands of data visualizations. We refer to these 7 heuristic principles as POUR+CAF. Chartability is a community-contributed project that leverages the governance strategies of open source projects as a way to address the complex dual-evolution of both accessibility and data interaction practices.

We additionally present an initial, light evaluation of Chartability from the experience of practitioners using it. We set out to see if using Chartability reduces the barrier of entry into

this work for accessibility novices and if accessibility experts had any feedback to share about its use. We gave practitioners introductory material for Chartability and instructed them to use it according to their needs. We found that before using Chartability only accessibility experts believed auditing data visualizations to be somewhat easy or easy, while the other group believed auditing data visualizations to be somewhat hard or hard. All novice accessibility practitioners became more confident after using Chartability and believed auditing data visualizations for accessibility to be less difficult. Conversely while the expert accessibility practitioners were already confident in their ability to evaluate accessibility (and all unanimously had no change in their before and after evaluations), they were excited to adopt Chartability into their set of auditing resources.

Our work sets out to acknowledge that data practitioners face significant barriers when first making data visualizations, systems, and experiences accessible. While Chartability contributes to filling gaps and organizing knowledge, it also challenges visualization and data interaction researchers to explore new horizons of possibilities in this space. As such, we conclude with recommendations for future research at the crossroads of data visualization and accessibility.

4.3 Existing Work in Data Visualization and Accessibility

While recent works at the intersection of data visualization and accessibility are promising, they do not provide a consistent and unified methodology for designers to evaluate the accessibility of their work across the broad spectrum of disability considerations.

4.3.1 Research Advancements in Data Visualization and Accessibility

In parallel to Mack et al.’s “What do we mean by Accessibility Research?” [111] when we asked “What do we mean by data visualization accessibility research?” we found that nearly all topics of study were vision-related. Largely, access issues other than vision that affect data visualization (such as cognitive/neurological, vestibular, and motor concerns) are almost entirely unserved in this research space. Kim et al. found that 56 papers have been published between 1999 and 2020 that focus on vision-related accessibility (not including color vision deficiency), with only 3 being published at a visualization venue (and only recently since 2018) [101]. Marriott et al. found that there is no research at all that engages motor accessibility [120]. We have found 2 papers that engage cognitive/neurological disability in visualization and 1 student poster from IEEE Vis, which are all recent (specifically intellectual developmental disabilities [220] and seizure risk [185, 186]). We found no papers that engage vestibular accessibility, such as motion and animation-related accessibility. We also found that there is no research specific to low vision disabilities (not blindness or color vision deficiency) unless conflated with screen reader usage in data visualization. Blind and low vision people are often researched together, but in practice may use different assistive technologies (such as magnifiers and contrast enhancers) and have different interaction practices (such as a combination of sight, magnification, and screen reader use) [194].

Since the 1990s, the most prominent and active accessibility topic in visualization has been color vision deficiency [23, 107, 122, 138, 141]. Research projects that explore tactile sensory

substitutions have been a topic in computational sciences dating back to the 1983 [167], with tactile sensory substitutions being used for maps and charts as far back as the 1830s [68]. Sonification used both in comparison to and alongside visualization and tactile methods for accessibility dates as far back as 1985 [17, 32, 51, 116, 125, 223]. Some more recent work has explored robust screen reader data interaction techniques [61, 184], screen reader user experiences with digital, 2-D spatial representations, including data visualizations [164, 176], dug deeper into the semantic layers of effective chart descriptions [109], and investigated how to better understand the role of sensory substitution [29]. Jung et al. offer guidance that expands beyond commonly cited literature that chart descriptions are preferably between 2 and 8 sentences long, written in plain language, and with consideration for the order of information and navigation [96]. We find all of this emerging work promising and foundational.

Despite this promising work emerging, we also want to acknowledge a spectrum of other work that exists at the intersection of accessibility and data visualization that does not serve the goals of our project. There is significant research that explores automatic or extracted textual descriptions [5, 24, 25, 26, 105, 139, 149, 175] and haptic graphs and tactile interfaces [1, 4, 13, 18, 20, 56, 91, 92, 106, 167, 168, 179]. These research projects produce artifacts that are high-cost for individual use, some are not robust enough to interpret complex visualizations effectively, and several have not included people with disabilities. Since our goal is to synthesize knowledge for practitioner accessibility work, we also acknowledge that some of these projects did not follow standards during their research project and in their output, such as using Web Content Accessibility Guidelines [203] or The American Printing House for the Blind and Braille Authority of North America [6]. All of these challenges are factors that limit the generalizability of these artifacts and knowledge for practitioner use [110, 131, 176]. We encourage work to continue at the intersection of accessibility and visualization, but stress the importance of practical, disability-led research that either builds on or explicitly challenges standards.

4.3.2 Accessibility Practices in Data Visualization Tools and Libraries

Our research goals are to find what is already being done in data visualization and accessibility and to see if we can enhance that activity. To this aim, our background investigation includes a broad and comprehensive exploration of the field of practitioner and non-academic artifacts.

Some open source and industry contributions have pushed data visualization and related accessibility efforts. Libraries like Highcharts [77] or Visa Chart Components (VCC) [202] and tools like the Graphics Accelerator in SAS [161] have broad accessibility functionality built in, but their documentation is technically specific to their implementation. While these relatively accessible libraries and tools can be helpful for inspiration, their specific techniques and guidance materials are not easily transferrable to other environments or applications where data visualizations are created. Practitioners must reverse engineer and deconstruct many of the methods employed by these libraries, and with the exception of VCC (which is open source), this task requires significant effort, given their primarily closed-box nature.

In common charting tools and libraries (apart from those already mentioned) accessibility engineering is often not present, limited in scope, or has only recently become an effort. More established visualization libraries like matplotlib, ggplot2, d3js, R-Shiny, and Plotly have left most accessibility efforts to developers, with varying levels of documentation and difficulty in-

volved [21, 30, 53, 54, 208]. None of these major tools have a broad spectrum of accessibility options built in and documented.

Community contributors often must fight to make their tools and environments accessible (sometimes even against the design of the tools themselves) with little to no compensation for their contributions. For example, Tableau’s first accessible data table was built by a volunteer community member Toan Hoang as an extension [79]. Tableau users more broadly must resort to voting systems to gather attention to accessibility issues [35]. Semiotic’s accessibility features were added by community member Melanie Mazanec [124]. For Microsoft’s PowerBI, students have organized resources for how to make visualizations built with it more accessible [103] while non-profits like the City of San Francisco’s data team have had to build features like keyboard instructions from scratch [159]. Mapbox GL JS is an example of a popular mapping library (over 400,000 weekly downloads) [118] that has no built-in accessibility support by default. The accessibility module for Mapbox GL on GitHub was created and maintained by volunteers but has had less than 10 weeks of work with any activity invested since its first activity in late 2017 [117].

Many community-driven efforts are under-utilized, must be discovered outside of the primary environment’s ecosystem, have poor or no core, internal support, and are inconsistently and partially implemented. Accessibility is still an afterthought in data visualization and ad-hoc, specific solutions proposed have not led to widespread improvements.

4.3.3 Accessibility in Practice, Broadly

Accessibility in practice is largely motivated by standards work or assistive technology. We want to acknowledge that tactile and braille standards are robust [6], but have limited transferability to digital contexts currently. For example, whereas tactile graphics guidelines lend insight into information prioritization, layout, and fidelity, the assumption is they will be embossed onto paper or similar physical mediums [10, 110, 176].

In digital contexts, the most influential body for accessibility is the World Wide Web Consortium’s (W3C) Web Accessibility Initiative (WAI). WAI’s Web Content Accessibility Guidelines (WCAG) [203] influence accessible technology policy and law for more than 55% of the world’s population [87]. WAI and WCAG outline 4 types of functional accessibility principles: Perceivable, Operable, Understandable, and Robust, abbreviated as POUR [205]. POUR is the foundation that organizes all 78 accessibility testing criteria in WCAG.

4.3.4 Using Heuristics to Break Into Under-addressed Areas

To summarize the complex problem space to which this paper contributes: Research in data visualization primarily focuses on visual accessibility, accessibility standards focus on a broad range of disabilities but lack deep contextualization for data visualization, and practitioners seem to build a wide array of solutions to fill these gaps, most of which are poorly maintained or adopted. Any time a practitioner wants to embark on a journey learning how to evaluate the accessibility of a data visualization, they must collect and synthesize this complex space of knowledge themselves. We have included (with permission) an exemplary field artifact as an example of this type

of labor in our supplemental materials, which contributed to the United States Government’s project, “Improving Accessibility in Data Visualizations” [200, 201].

After gathering information with this breadth and complexity, a heuristic evaluation model was chosen as a way to deliver useful but flexible knowledge. Heuristic evaluation models have a long history in HCI and are cheap to use and require little expertise. They have been shown to be effective methods for practitioners compared to user testing, focus groups, or other evaluative methods that require existing expert knowledge or recruitment, moderation, and compensation of participants [15, 27, 58, 94, 122, 136, 137, 143, 160, 182]. Heuristics are also not new in visualization [31, 52, 156, 169] even among topics related to accessibility (color vision deficiency, specifically) [141, 160].

4.4 Making Chartability

We next present Elavsky’s work to develop Chartability as a real-world design process contribution to the larger research community. Our making process does not neatly fit into most design models that divide researchers from practitioners. In Gray’s different models of practitioner-researcher relations, our work is some variation of bubble-up, practitioner-led research [64]. This project was initiated by Elavsky while they were an industry practitioner, deeply situated in this work already.

Thus, the following description of Chartability’s 10-month creation is written from Elavsky’s perspective. The supplemental materials include the data from this stage of the process, a preview of which is available in [Table 4.1](#):

1. **Situate, Survey, and Select Problem Space:** I was situated within the context of accessibility evaluations of data visualizations. From personal experience, I recognized the prohibitively significant labor involved in ensuring I was effectively following accessibility standards while also attending to the complex design considerations of data visualizations. To improve this work both for myself and others in the future, I surveyed existing problems and challenges others faced and selected a solution that I felt equipped to address.
2. **Collect Existing Resources:** I set out to answer, “If evaluating the accessibility of data experiences is hard, what do existing standards miss?” I evaluated my seed knowledge (WCAG criteria) for shortcomings and gaps and collected other data relevant to my goal (academic and industry research, open-source libraries, tools, applications, data products, government guidelines, design guidelines, software documentation, university coursework, and practitioner articles).
3. **Code Resources:** After collating these resources (including relevant WCAG criteria), I loosely borrowed from thematic analysis [16] and qualitatively coded this data. I developed a set of 29 codes starting with WCAG’s POUR principles and expanded the codes to account for other concerns that came up in the resources, including what type of accessibility was being addressed (e.g., cognitive, visual), whether a solution was technology-specific or agnostic, and other categories (like “time-consuming” or “user-controlled”). I then divided the resources into codable segments with relatively distinct pieces of information and applied the 29 codes to the information segments. I grouped information with

codes in common, resulting in a representative 45 groups of related information segments.

4. **Synthesize Heuristics:** Since auditing depends on measurable heuristics, I adjusted each of the 45 groupings that resulted from the qualitative analysis into phrasing that could be verified by an evaluation. I then augmented each heuristic with known testing procedures, resources, and tools necessary for applying them in practice. 10 critical heuristics (these were determined top priorities through user feedback) are previewed in [Table 4.1](#), with the full version of this table (and more) provided in our supplemental materials.
5. **Group Heuristics into Higher-level Principles:** I linked each heuristic with relevant web accessibility standards and POUR principles to draw a familiar connection for users who might already be accessibility practitioners. 26 heuristics fit neatly back into Perceivable, Operable, Understandable, or Robust.
6. **Develop Remaining Themes into New Principles:** 19 remaining heuristics with complex codes and overlapping groups demanded new theorizing, as they either did not fit into POUR at all or could arguably belong to multiple principles at once. I analyzed these remaining complex heuristics and for similarities and organized them under 3 new themes, which we are contributing as new accessibility principles, Compromising, Assistive, and Flexible, defined below.

Heuristic title	Principle	Origin	Coding categories	
			POUR	Other
Low contrast	Perceivable	Standard	P	2
Small text	Perceivable	Research	P	2
Content is only visual	Perceivable	Standard	P, R	3
Interaction has only one input	Operable	Standard	O, R	3
No interaction cues/instructions	Operable	Standard	O, U	2
No explanation for how to read	Understandable	Research	U	1
No title, summary, or caption	Understandable	Research	U	1
No table	Compromising	Research	O, U, R	3
Data density inappropriate	Assistive	Research	P, U	4
User style change not respected	Flexible	Standard	P, O, R	6
... and 35 additional non-critical heuristics				

Table 4.1: Previewing Chartability’s 10 critical heuristics. The coding categories are split into two parts: which POUR principles contributed to each heuristic, and how many additional coding categories (“Other”) were assigned.

4.4.1 Compromising

Compromising is a principle that focuses on Understandable, yet Robust heuristics. These heuristics are based on providing alternative, transparent, tolerant, information flows with consideration

for different ways that users of assistive technologies and users with disabilities need to consume information.

Compromising challenges designs that only allow access to information through limited or few interfaces or processes. These heuristics focus on providing information at a low and high level (such as tables and summaries), transparency about the state of complex interactions, error tolerance, and that data structures can be navigated according to their presentation. Compromising designs have both information and system redundancies in place.

4.4.2 Assistive

Assistive is a principle that primarily builds off the intersection of Understandable and Perceivable principles but focuses on the labor involved in access. These heuristics include categories that encourage data interfaces to be intelligent and multi-sensory in a way that reduces the cognitive and functional labor required of the user as much as possible.

The Assistive principle focuses on what Swan et al. refer to as “adding value” [193] and what Doug Schepers meant by “data visualization is an assistive technology” [165]. We visualize because it is faster and more efficient than munging cell at a time through data. Assistive heuristics ensure that both visual and non-visual data representations add value for people with disabilities.

4.4.3 Flexible

Contrasted with Compromising (which focuses on robust understanding), flexible heuristics focus on robust user agency and the ability to adjust the Perceivable and Operable traits of a data experience. Flexible heuristics all have a tight coupling between a data experience and the larger technological context the user inhabits. The preferences that a user sets in lower-level systems must be respected in higher level environments.

Self-advocacy and interdependent agency are important sociotechnical considerations that engage the conflicting access needs that different users might have in complex technological interactions like data experiences [7, 115]. Some users might want specific controls or presentation, while others might want something else entirely. Designs must not be rigid in their opinions and ability assumptions and should be designed to be moldable by and adaptive to user needs [104, 218].

4.5 Using Chartability

All of Chartability’s tests are performed using Chartability’s workbook [40] alongside various tools and software (linked in the workbook). For the scope of this paper, we are not including an explanation for how to perform all of these. Both the workbook and supplementary materials with this paper give more details.

While a highly trained auditor may be able to casually evaluate an artifact in as little as 30 minutes or even hold heuristics in mind as they are doing their own creative work, those new to auditing may take anywhere between 2 and 8 hours to complete a full pass of Chartability. Professional audits, which can take weeks or months, often include multiple auditors and provide

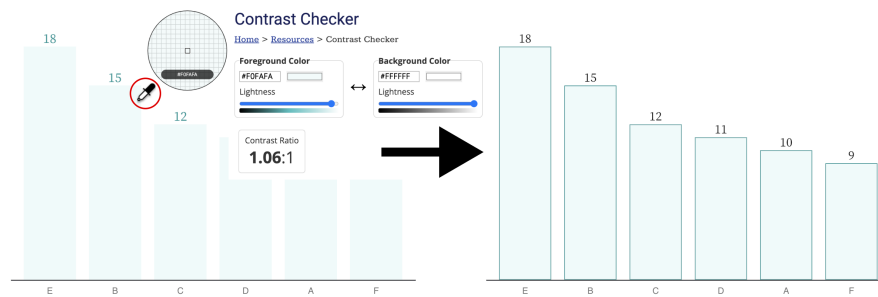


Figure 4.1: A low contrast chart (left) compared to a higher contrast version (right). A dropper tool is extracting the fill color of the bar and then a contrast ratio has been calculated. Note that the fill color is the same on both bars, but darker borders have been added to ensure the visualization passes contrast tests.

rigorous documentation and detailed recommendations for remediation, typically in the form of a report. Chartability is meant to serve both quick pass and deep dive styles of audits, so users are expected to leverage it as they see fit.

Below we give an example of what might be a quick pass audit, using Chartability. Which principles are applied in each of these stages are listed in parentheses in each heading.

4.5.1 Visual Testing (Perceivable)

Checking for contrast is the most common critical failure; 87.5% of tests (7 out of 8) from our user study involving this heuristic failed, which supports the WebAIM Million Report’s findings (83.9% of the top 1 million websites also fail contrast testing, more than any other WCAG criterion) [207]. In order to evaluate contrast, often a combination of automatic (code-driven) and manual tooling is performed. When manually auditing, practitioners typically use a dropper and a contrast calculator (Figure 4.1). Most auditors find this to be one of the easiest tasks to perform and accomplishes 3 different heuristics in Chartability: ensuring text/geometries have contrast, interactive states for elements have enough contrast change, and the keyboard focus indicator is easy to distinguish.

Perceivable heuristics also include tests and tools for color vision deficiency and ensuring that color alone isn’t used to communicate meaning (like the redundantly encoded textures in Figure 4.8). And another common, critical failure from Perceivable is text size. No text should be smaller than 12px/9pt in size.

4.5.2 Keyboard Probing (Operable, Assistive)

The next practice that most auditors should become comfortable with is using a keyboard to navigate and operate any functionality that is provided. Most assistive technologies, from screen readers to a variety of input devices (like switches, joysticks, sip and puffs, etc) use the keyboard API (or keyboard interface) to navigate content. If a data interface contains interactive elements (Figure 4.2, Figure 4.3), those elements (or their functionality) must be able to be reached and

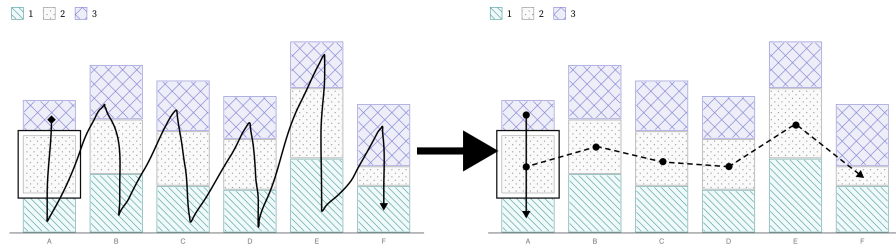


Figure 4.2: Keyboard navigation paths on a stacked bar chart. The left shows a serial navigation example, typically just a default of rendering order. The right shows both groups (the stack of bars) and categories (the color/texture shared among bars across stacks) as dimensions to explore laterally or vertically.

controlled using a keyboard alone. Auditors should be critical of how much work is involved in keyboard navigation, especially (Figure 4.7). All that is required to start is that the auditor begins pressing the tab key to see if anything interactive comes into focus. Arrow keys, spacebar, enter, and escape may be used in some contexts. Generally, instructions or cues should always be provided.

Using a keyboard provides an opportunity to evaluate many different heuristics: checking for multiple inputs (Figure 4.3), whether the data structure that is rendered is navigable according to its structure (Figure 4.2), and whether keyboard navigability across all elements in a data interface is even necessary (Figure 4.7).

4.5.3 Screen Reader Inspecting (Perceivable, Operable, Robust, Assistive)

Closely related to keyboard testing is testing with a screen reader. Some things may work with a screen reader that do not with a keyboard (and vice versa), so both must be evaluated.

Screen readers, unlike more basic keyboard input devices, read out content that is textual (including non-visual textual information like *alternative text*). Using a screen reader to audit is generally the hardest skill to learn. Keeping this in mind, testing whether the meaningful text provided in a visual (such as in Figure 4.4) is accessible with a screen reader is the easiest and most basic test that auditors should first perform.

Next, all valuable information and functionality in a data experience should be tested whether it is available to a screen reader. This includes the individual variables about a mark as well as whether that mark is interactive (Figure 4.5), whether status updates that reflect context change provide alerts (Figure 4.3), and whether summary textual information is provided about the whole chart (Figure 4.4) as well as statistically and visually important areas of that chart (Figure 4.7).

4.5.4 Checking Cognitive Barriers (Understandable, Compromising)

First, auditing for cognitive barriers generally involves checking the reading level and clarity of all available text using analytical tools. But Chartability also requires that all charts have basic text that provides a visually-available textual description and takeaway (Figure 4.4). This alone is

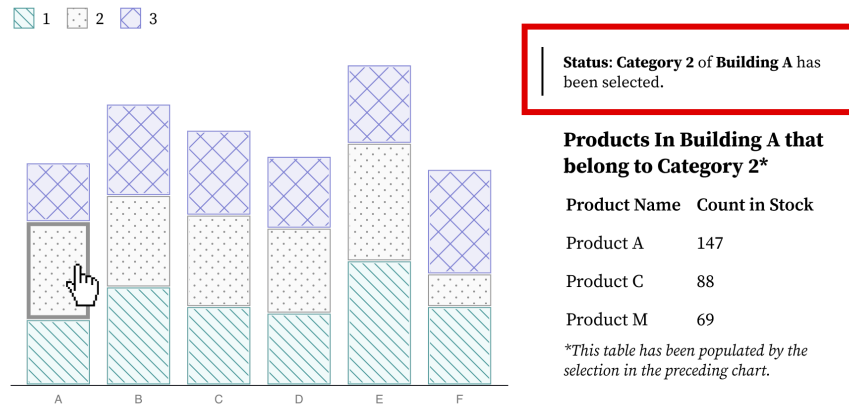


Figure 4.3: A mouse cursor is selecting a bar (left, shown with a thick indication border) in a stacked bar chart to filter a dataset (on the right). A system alert (red box) notifies the user of their interaction result. This selection capability must also be provided for the keyboard interface and the alert must be announced to screen readers.

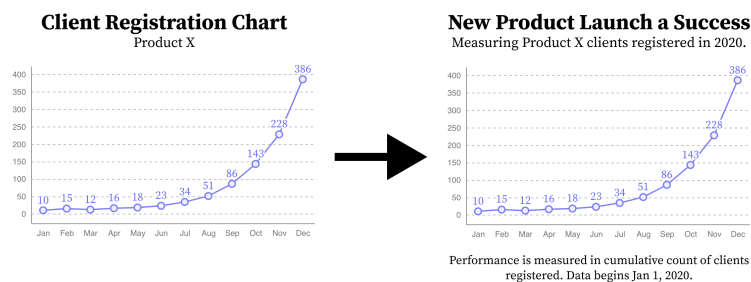


Figure 4.4: Charts must have a visually available textual explanation provided that summarizes the outcome. “Client Registration Chart” for “Product X” (left) is inaccessible while “New Product Launch a Success” (right) gives a clear takeaway.

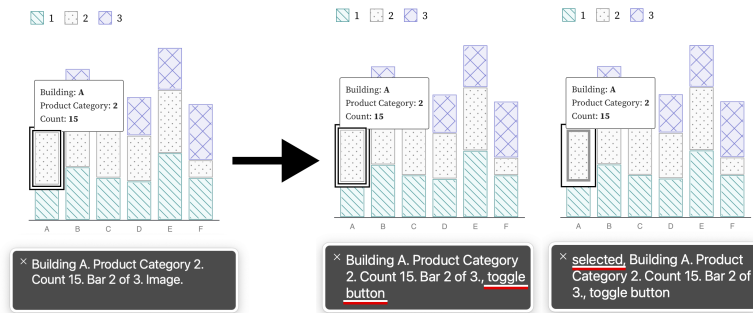


Figure 4.5: An interactive chart displaying only “Image” as semantic information with no feedback provided on selection. The robust semantics given to a screen reader, “toggle button” (middle) as well instant feedback, “selected” (right) are considered proper semantics for an interactive experience.



Figure 4.6: A line chart (left) with a single line and an accompanying data table (right). This line chart would not provide enough low-level information about each datapoint without the table provided. A table alone however would also be inaccessible. Providing both can satisfy conflicting accessibility needs for different audiences.

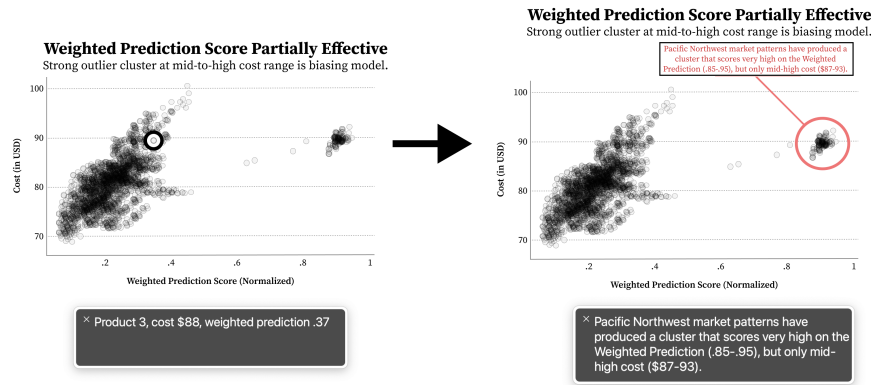


Figure 4.7: A scatterplot with many points, where a single point within the chart can be accessed by a screen reader (left). Navigating this data piece by piece is unnecessarily tedious, so an annotation callout is provided to help the reader focus on an outlier cluster (right). The callout is being accessed by a screen reader, which is displaying the annotation's summary as well.

one of the most important things to check for. In complex cases where a chart has a visual feature with an assumedly obvious takeaway, checking for annotations or textual callouts is important to help avoid interpretive issues [221] (Figure 4.7).

4.5.5 Evaluating Context (Robust, Assistive, Flexible)

The final series of checks an auditor should make involve thinking about the overall work in a design (as it intersects with other considerations) as well as the larger technical context where the user is situated.

Auditors should first try to change system settings (such as toggling high contrast modes) to see whether a data experience respects these settings (Figure 4.8), run automatic semantic evaluations as well as manually check for appropriate meaning (Figure 4.5), and check if dense or highly complex visuals have sonified, tactile, or textual summaries available (Figure 4.7). Auditors should also check whether system updates provide clear feedback textually (Figure 4.3) as well as checking if there are both high and low level representations of information available (Figure 4.6).

Auditors should be especially critical of static designs, such as those that either use textures by default or not (Figure 4.8), which are a high risk of compromising and assistive failure.

4.6 Validating Chartability

Next, Elavsky explains the preliminary user evaluation: I validated whether data practitioners felt more confident and equipped to make their own work accessible with Chartability. Additionally, I also wanted to interview expert accessibility practitioners (including those with disabilities) with the same questions, to see if Chartability had anything to offer in helping them understand and evaluate data experiences better.

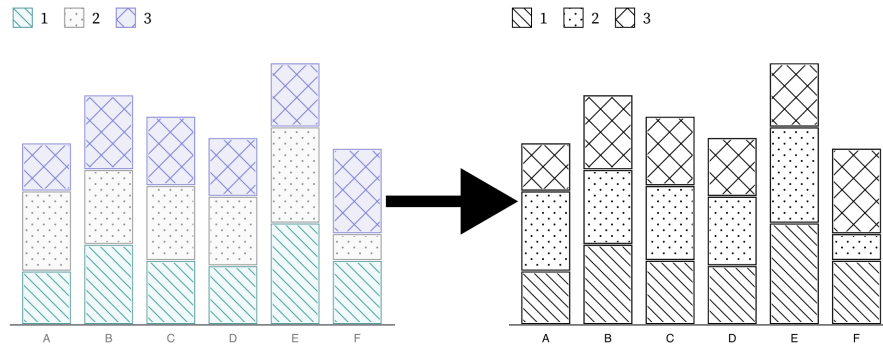


Figure 4.8: A bar chart with categories (left) shown not conforming to Windows High Contrast White Mode. High contrast mode on Windows requires limiting color palettes, using only black or white for most elements (shown on the right).

My secondary goal was to present a tool that can be helpful even in the wild on real projects (with all the weird design and engineering quirks that come with that). I wanted Chartability to be usable on things built with a tool like Tableau and fully bespoke, hand-coded visualizations, like those made with JavaScript and D3. To this secondary aim, I intentionally solicited participants who were working on a variety of different projects, each of their own design.

4.6.1 Pre-Validations and Flipped Roles: Participants Question *Me*

I performed several early, light validations of my work before soliciting and involving participants formally. My early pre-validations #2-4 (below) all focused on practitioners asking me questions and giving feedback.

My 4 pre-validations happened during the process of making Chartability, as well as introducing short iterations back into the making process:

1. **Beta Testing:** I performed several beta tests of Chartability during the process of making. I audited using versions that only had POUR principles, tried versions of Chartability that focused only on standards, and also tried out different iterations of the heuristics as I was forming them. This testing was important to perform early in the process because it helped me test the limits of various possible directions for this tool (standards-only, against standards, building off of standards, etc).
2. **Early Advice:** After the first full pass of making Chartability was complete, I sent Chartability via email to 4 accessibility experts and 6 interested people with disabilities familiar with auditing in order to solicit open feedback.
3. **Professional Workshop:** I held a half-day professional workshop via Zoom on auditing visualizations for accessibility and presented Chartability's heuristics to this select audience of 50 participants. I demonstrated how to audit and then had a chance for feedback and questions.
4. **Deep Feedback Session:** I presented Chartability to 14 experts on data visualization and accessibility, 5 of which are people with disabilities. I presented in two separate sessions

through 2-hour video calls on Zoom (roughly one hour was demonstration and one hour was discussion).

4.6.2 Discovering “Critical” Heuristics

These pre-validations helped me combine and divide some of the heuristics, adjust the language and phrasing, and label 10 specific tests as “Critical,” which can be seen in [Table 4.1](#). These critical tests were ones that community members stressed as an important priority for one or more of the following reasons:

- They are prohibitively expensive to fix late.
- The barriers they produce are too significant to ignore.
- They are among the most common type of accessibility failure.
- They affect many parts of a data experience.

All Critical heuristics are based on standards or research.

4.6.3 Selecting Participants and Projects

I was a practitioner and representing myself as a volunteer when I reached out to participants. At this stage in the project, I was still not affiliated with a research institution and was not interested in producing publishable knowledge. I intended to test Chartability in the wild and validate whether it achieved its aims. My priority was to collaborate with folks working on difficult problems or those who had a rare intersection of expertise between accessibility standards and interactive data experiences. To this end, I was highly permissive with potential collaborators in order to maximize the expertise of participants and breadth of environments for testing Chartability.

However, part of being permissive with participants meant that I was willing to collaborate on projects that I cannot share in a research publication and many of my participants must remain anonymous (including interview results that contain sensitive information about intellectual property). Given that auditing is a field of work about identifying failures, there was both a high demand for participation in the evaluation of Chartability in tension with a low motivation to make these failures known in a public venue.

A summary of our selection process:

- **Solicitation:** I reached out via email to 24 individuals in my network to participate in helping to evaluate Chartability. I mentioned that I wanted Chartability to be applied to a current project of theirs and was interested in performing some interviews about their experience before and after using Chartability. I mentioned up front that working with me would be uncompensated and potentially take multiple hours of their time (even multiple sessions) over Zoom meetings.
- **Response:** 16 individuals were interested and shared their project details (2 would require an NDA to be signed).
- **Selection:** I selected 8, based either on the expertise of the individuals, on the robustness of their project, and/or on the opportunity to get feedback about Chartability in team

environments (which I didn't anticipate, but 3 of the 8 represented team efforts).

- **Resulting Group:** I worked with 19 total participants across 8 environment spaces.
- **Publishable Group:** Due to intellectual property concerns, I can publish interview results from 6 participants and discuss the details of 4 audit environments.

Chris DeMartini: a multi-year Tableau Zen Master and recognized expert visualization practitioner. His dashboard of a coin flipping probability game dataset that he produced with his daughter was the subject of his audit [34]. His audit only included criteria labelled Critical in Chartability (which involves only 10 tests instead of the full 45) and his dashboard failed 7 of them. A full audit was later conducted on Chris's behalf. His full audit had a total of 26 failures, 11 of which were considered non-applicable.¹

Amber Thomas: a data storyteller and technologist credited on 30 of The Pudding's visual essays. Amber has had a growing interest in accessibility challenges related to her line of work designing and developing state of the art, bespoke visual essays. Her article The Naked Truth was still in the early design and development stages when it was fully audited [3]. It failed 22 out of 45 tests, including 6 out of 10 criteria considered Critical. 6 tests were considered non-applicable.¹

Sam (self-selected pseudonym): a recognized design practitioner in the visualization community who lives with disability. They were collaborating on an interactive data project that would be specifically made to be used by international participants with a broad spectrum of disabilities. Their interactive infographic failed 21 out of 45 tests, 5 of which were considered Critical. 10 tests were considered non-applicable.¹

Øystein Moseng: Core Developer and Head of Accessibility of Highcharts. Øystein was interested in taking one of Highchart's demo charts not specifically developed with accessibility features in mind [78] and testing it against a full Chartability audit to see how it held up. The demo failed 13 out of 45 tests, 3 of which were Critical. 10 tests were considered non-applicable.¹

Jennifer Zhang: a senior accessibility program manager at Microsoft with expertise working on enterprise data products.

Ryan Shugart: a blind, screen reader user and disability subject matter expert at Microsoft who has a strong expertise in collaborative accessibility for interactive data systems.

Both Shugart and Zhang were interested in applying Chartability internally and testing its effectiveness and potential with various projects. Their application and use of Chartability (including audits) are not available for publication, but their valuable interviews and evaluations are included with permission.

4.7 Study Results

I asked the 6 participants a series of qualitative and Likert-scale evaluation questions:

1. Have you ever performed an audit of a data experience before?

¹“Non-applicable:” any test in the auditing process that does not contain content relevant to the test, such as “Scrolling experiences cannot be adjusted or opted out of” for a visualization that does not have a scrolling input control

2. What stage of production is your project in? Analysis, design, prototyping, development, maintenance?
3. How confident are you in your ability to perform an audit of a data experience for accessibility issues? (1-5, 1 being not confident at all, 5 being fully confident.)
4. How difficult do you perceive auditing a data experience for accessibility issues is? (1-5, 1 being trivial, 5 being very difficult.)
5. (After using Chartability) How confident are you in your ability to perform an audit of a data experience for accessibility issues? (1-5, 1 being not confident at all, 5 being fully confident.)
6. (After using Chartability) How difficult do you perceive auditing a data experience for accessibility issues is? (1-5, 1 being trivial, 5 being very difficult.)
7. (After using Chartability) Do you intend to continue using Chartability?

Each of these questions had an open-ended question attached, “Is there anything else you would like to add?” Every participant provided additional input on questions 3 through 7.

None of the 3 participants who only consider themselves expert data practitioners had performed an audit before. All 3 of them reported that they believed auditing to be easier and that they are more confident in their ability to evaluate the accessibility of data experiences after using Chartability.

Of the 3 accessibility experts (all of whom have performed audits of data experiences before), their opinions on these measurements were unchanged after using Chartability. All 6 participants noted that they plan to use Chartability in their own work and would recommend it to their peers.

Below we overview some of the key insights Elavsky received from the open ended responses.

4.7.1 Real Access has more Considerations than Colorblindness

Among the data practitioners, DeMartini wrote after his audit, “I have read a lot about color blindness and could provide meaningful feedback to visualization developers on that topic, but I have come to realize that accessibility is so much more than this and I basically didn’t really know where to start when it came to the true scope of accessibility.” He ended his qualitative feedback with, “I think this could be a great tool for the masses and really look forward to the impact it can possibly have on the (inaccessible) data visualizations which are being created in huge numbers these days.”

4.7.2 Audits are Slow, but Help me Focus

Amber Thomas wrote, “It still takes a while to do a complete audit, but it’s not hard! For someone new to the space, all the possible options that can be used to make visualizations more accessible can be overwhelming. [Chartability] helped me to focus.” She finished her feedback with, “There aren’t really guidelines (at least to my knowledge) that exist to help data visualization creators to ensure their work is accessible... [Chartability] helps to direct users to the most common

accessibility problems with straightforward questions. It really helps to narrow the focus and prioritize efforts.”

4.7.3 Chartability Helps me Remember and Stay Consistent

Among the accessibility experts, Zhang wrote, “While I am skilled, depending on the day I might not remember everything I need to look at. I am more confident in consistency between different auditing sessions. For experts it’s a good reminder framework.” Moseng of Highcharts noted, “[Chartability] did a very good job of highlighting concerns that are often ignored or forgotten when auditing and designing/developing.” Shugart of Microsoft added along those lines, “I feel [Chartability] arranges a good set of questions in a user’s mind and makes it easier for them to determine if a visualization is accessible.”

4.7.4 Access is an Experience, not just Compliance

Zhang offered insight into the design intention of Chartability, “[it is] clearly going for above compliance and focusing on a good experience.” Sam expressed their need to make an excellent accessibility experience, “I am not just worried about compliance, but I want to make something really good. Nothing seems to help you go beyond? This is better than WCAG, I can already tell.”

4.7.5 Everyone wants More Evaluation Resources and Tools

For constructive feedback, all the data experts noted that they wanted more resources and materials related to learning the skills needed to conduct an audit. Shugart and Moseng both noted that they hope for more tooling and (in some cases) automated tests that can take the burden off the auditor and streamline the design and development process (much like Axe-core [37]). They both also agreed that automation and tooling would help novice practitioners perform this work faster and with more confidence. 2 of the 3 mentioned wanting more examples of failures as well as accessible data experiences. Sam wrote that they felt Chartability was overwhelming at first, but after focusing on just the Critical items, the rest of the framework “became easier.”

4.7.6 Experts: “Novices will Struggle.” Novices: “This was so helpful”

The accessibility experts all unanimously agreed that Chartability is helpful to their own work, but they are unsure how accessibility novices would do. They all believe that more training and resources are needed to help people who are new, with one noting that Chartability could even be “overwhelming” to someone who has not been exposed to accessibility work before. All of the novices remarked that Chartability was “so helpful,” “made this work so much clearer than before,” and “made a lot of hard problems not as hard.”

4.7.7 What about Auditors with Disabilities?

Shugart’s feedback was critical when discussing continuing to use Chartability, “I still feel as a screen reader user, the audit itself would have some unique challenges because I’d be missing a lot and would have problems determining things such as color.” He continued, “Auditing anything accessibility-wise as a screen reader user poses challenges because you don’t always know what you’re missing. In many cases there are workarounds to this but datavis is one area where this is really hard to do now.”

4.8 Extended Results

Following calls to ensure accessibility work has practical benefits that exceed publications [84], in April, 2021 Elavsky made Chartability openly available on GitHub. As new research and practices emerge and more community members get involved, Chartability will become an evolving artifact of consensus similar to existing standards bodies [206].

Projects like Turkopticon benefited from the discussion about how a community actually used their tool [88]. In the same vein, we are happy to report some valuable findings from within this last year that we think demonstrate (in a pragmatic way) that Chartability has some merit:

- **It is living and growing:** Chartability has received enough community feedback that it is now on Version 2, with more tests and background resources provided.
- **People are talking about it:** Chartability has been featured in 14 workshops, talks, and podcasts and at least 2 university courses.
- **People are using it:** Chartability has contributed to projects at Microsoft, Highcharts, Project Jupyter, Fizz Studio, FiveThirtyEight, Vega-Lite, UCLA, the City of San Francisco, the Missouri School of Journalism, a Fortune 50 company, two Fortune 500 companies, and community groups (like MiR).
- **It has breadth:** Chartability has evaluated static and interactive data experiences made with Microsoft’s Excel and PowerBI, Tableau, JavaScript (D3, Vega-Lite, Highcharts, Visa Chart Components), Python (Altair, Bokeh, and matplotlib), R (ggplot2), as well as design sketches and low/medium-fidelity artifacts (Illustrator, Figma, Sketch).

When considering the analysis by Hurst and Kane about high abandonment rates in assistive technology, [84] we wanted to make sure that we created an artifact (assistive technology or otherwise) that would at least survive its first year of use in the real world.

The greater community feedback as well as new research before and after open-sourcing Chartability has also led to 5 new heuristics being added since our test users performed audits and gave evaluations. The current version of Chartability (v2) has a total of 50 heuristics.

It is important to note that the work of Chartability did not begin and does not conclude with the publication of this manuscript. We want Chartability to become a living, community-driven effort that will adapt and grow as more resources, tools, and research become available.

4.9 Discussion

From our presentation of Chartability and a preliminary user evaluation with data visualization and accessibility practitioners, we learned that Chartability reduced the perception that working on accessibility is difficult and increased the confidence of those new to this work. Chartability shows promise as a useful framework for expert accessibility practitioners because it serves to produce consistency in contexts like the evaluation of dashboards, data science workflows, and other complex, data-driven interfaces.

While our practitioners with novice accessibility experience were initially concerned about doing the audit correctly, most of their audit results were reasonably comparable with that of the authors (although their time to complete was much longer).

We agree with experts that beyond Chartability, more resources are needed which provide examples of both inaccessible and accessible data visualizations as well as how to perform some of the more difficult parts of the auditing process (such as evaluating with a screen reader). We hope that keeping Chartability on GitHub will inspire future improvements to address this gap in examples, and will address future limitations, as we discover them.

Chartability is a valuable tool for auditing. But we also hope that it can inspire researchers to:

1. Examine which heuristics (in our supplemental materials) could use more research attention, particularly those labelled “community practice.”
2. Define constraints or requirements on novel projects, ensuring that new explorations still respect established standards, mitigating ethical risks.
3. Explore the intersections of disability in ways yet unaddressed in standards, such as the strong overlaps between understandability and operability (like keyboard navigation patterns across a data structure) or conflicts in understandability and flexibility (how some users need redundant encodings on charts while others find this overwhelming).
4. Consider access barriers in data experiences beyond those related to visual perception.
5. Engage the relationship between labor and access in computing, such as developing more measurements that demonstrate the imbalance of time and effort expected of users with disabilities (even in systems considered to provide “equal” access) and ways to evaluate who is contributing to accessibility efforts in a project (core team members, contractors, or volunteers).

We also want to caution researchers who are considering developing heuristics or auditing tools for use in practitioner environments to consider the tradeoffs between evaluation in rich, authentic professional settings and concerns such as intellectual property and corporate branding. We were able to apply our work in rich and collaborative practitioner settings because we were permissive with our potential participants. However, much of this work exists behind closed doors, similar to the downsides of industry research settings. More work may need to be done in order to encourage rich, cross-industry research projects, such as helping to anonymize the content of intellectual property and not just participants, while retaining data and findings.

4.10 Conclusion

The demand for accessible data experiences is long overdue. The Web Accessibility Initiative's (WAI) Web Content Accessibility Guidelines (WCAG) are over 22 years old and yet little work has been done to synthesize this large body of existing accessibility standards with research and inclusive design principles relevant to the fields of data communication, data science, data analysis, and visualization. Chartability begins to address unique accessibility best practice gaps in these domains with specific heuristics. This synthesis is meant to empower researchers, analysts, designers, developers, editors, and accessibility specialists with a framework to audit the accessibility of data experiences, interfaces, and systems to produce more inclusive environments for users with disabilities. The goal of Chartability is to make this work easier in order to encourage practitioners to regard current practices and resources, some of which have existed for decades.

In addition, Chartability opens the door to more work that remains to be explored in this space. Additional research is needed into many of the topic areas within Chartability's heuristic principles (POUR+CAF) as well as resources, examples, and tools provided for practitioners to perform this work more confidently and efficiently.

The changing landscape of visualization techniques and alternative interfaces (such as sonification and dynamic tactile graphics) may increase the demands for accessibility considerations in this space. The growing technological divide will become an even greater human rights issue as time moves on and we believe that tools like Chartability are necessary for the community of data practitioners to ensure they are including people with disabilities.

4.11 Postface

This chapter could easily produce the longest postface, and arguably the rest of this thesis *is* the postface following *Chartability*. But for the archival record, I (Frank) want to simply add that this project has afforded immense opportunities for future work (and still, even presently many threads are emerging).

On the one hand, *Chartability* was intended to be an in-depth set of heuristics that anyone could use to evaluate accessibility barriers in interactive data experiences. But on the other hand, the level of depth may also have introduced an unexpected outcome: many companies and organizations are interested in simply hiring an expert to help them remediate their systems. I wonder: “is it *too* deep?” But this unexpected outcome has been great for hands-on collaboration, funding opportunities, and for generating the focus areas of this thesis. However, it also points to a larger reality that many teams with the right motivations may simply be under-resourced.

Only a few practitioners in the world specialize in evaluating data visualizations and interactive data experiences exclusively, myself (Frank) being one of them. I helped to recruit among this group, led by Jonathan Schwabish, where we co-authored a report to better democratize the deeper knowledge required to do this work professionally [170]. While that project and my many speaking opportunities (nearly 100 focused on accessibility and visualization, largely thanks to *Chartability*) have been successful in helping to educate practitioners, we believe that significant gaps still remain. Practitioners need more, from somewhere, in order to feel independently empowered enough to begin, and gain expertise in, accessibility work in the context of interactive data experiences. Accessibility work itself still remains largely inaccessible.

And a wide range of recent work has begun to try to use *Chartability* as a prompt or design guardrail for agentic skills, or large-language model based or otherwise intelligent/automatic systems [28, 67, 112, 123, 145, 172]. The motivation here, as we interpret it, is to make *Chartability* more accessible to systems and automated or semi-automated design work. We intend to watch this work closely, because on one hand our goal for this project has been to empower practitioners themselves to make better decisions while they work. But the fundamental way that agentic systems operate is in delegation of agency and decision-making. Perhaps *Chartability*’s future lies in two spaces: narrow, but exceptionally deep human expertise in evaluation, or as guidance for automated systems whose context window has greater working-memory capabilities than an average interface designer or engineer.

However, the successful cases where *Chartability* has improved have been through socialization of the practice: shared expertise, working groups, collaborations, joint ventures, grant opportunities, talks, workshops, and more. Perhaps, as we unpack more in the conclusion of this thesis, what *Chartability* still requires lies upstream, not downstream or laterally: that policy, budgets, priorities, culture, and practice might be the new frontier for the success of this project.

Part III

Navigation: Making Data Structures Traversable

Chapter 6

Skeleton: Visual Authoring of Non-visual Data Experiences

6.1 Preface

When sighted practitioners author accessible data visualizations, they build navigation structures (the nodes, edges, and input bindings that govern how assistive technologies traverse an interface) entirely in code, with no visual representation. This invisibility makes navigation structures difficult to inspect, debug, and iterate on. To sighted practitioners, every other aspect of a visualization is iterated on because it is visible; navigation structure ships as a first draft, if at all, because it is not. Without a representation to react to, practitioners cannot develop judgment about what makes navigation good or bad, and the quality ceiling of non-visual experiences is set by the absence of a feedback loop. We address this problem through longitudinal co-design with practitioners across cartography, design systems, and open-source visualization, and make three contributions.

First, we introduce technical advancements for making the properties of accessible navigation structure visible and directly manipulable during authoring, grounded in two foundational pieces of infrastructure produced by our co-design work: an *Inspector* that renders navigation graphs as interactive node-link diagrams, and a *Dimensions API* that expresses navigation in terms of data dimensions rather than explicit graph construction.

Second, building on these, we present *Skeleton*, a direct-manipulation authoring environment in which the properties of an accessible navigation structure are translated into visual representations authors can observe and manipulate. Key techniques include a dual-view editor that simultaneously shows the system’s navigation model and the end user’s spatial experience, a scaffolding engine that automates spatial node placement by re-purposing a visualization rendering pipeline, a live label-template editor with real-time screen-reader-output preview, and a testing mode that makes traversal sequence visually trackable.

Third, we evaluate *Skeleton* through an in-situ study with 8 practitioners across visualization design, engineering, and research. Making navigation structure visible changed how practitioners engaged with accessible design: they reconsidered the architecture of their own visualizations, attended to a broader range of input modalities, and shifted from treating accessibility as a compliance task to treating it as a design problem.

6.1.1 Context

Data Navigator was successful enough to help us acquire practitioner-centered funding to apply it in real-world contexts. We worked with three different communities as a researcher-designer-engineer alongside the teams attempting to apply *Data Navigator* to their work. These projects, while ultimately responsible for *Skeleton* (spoiler alert), also quickly motivated a new hypothesis: that making data navigation experiences isn’t just *hard* for some generic reasons but that it may

be hard because it involves perceptual accessibility barriers for sighted practitioners (or in other words: we conjectured that sighted practitioners face barriers when working with non-visual design materials).

Data Navigator evolved and grew and found a place in real-world work after its release. However, *Data Navigator* eventually also became a vehicle to open up new research directions. Notably, we made improvements to the system from a technical standpoint and also identified new barriers that practitioners faced when applying *Data Navigator*, both of which we discuss further in this chapter.

The following work was proposed as the culmination of this thesis, and chronologically was the final work we completed for this body of work.

This chapter has also, notably, been modified from the submitted version to IEEE VIS, for the sake of this thesis’ overall narrative cohesion as well as the fact that this thesis can afford more space to expand in several areas beyond the page limits that VIS allows.

It is worth stating plainly, before the chapter proceeds, how we intend each artifact to be read. The two pieces of infrastructure described in later subsections, the *Inspector* and the *Dimensions API*, are engineering contributions in the ordinary sense: they are released, reusable, and already used in production-facing collaborations beyond this research. The integrated authoring environment we call *Skeleton* is something different. We built and studied it as a design probe [69, 86], an instrument whose purpose is to make a specific thing observable rather than to ship a finished workflow. What the probe is designed to make visible, up front, is twofold: first, the navigation structure itself, rendered as a manipulable visual object so that sighted practitioners can perceive and react to it; and second, the practitioners’ own relationship to that structure, what they notice, question, and revise once it stops being invisible. The first is a property of the artifact; the second is what the study is built to surface.

This framing has consequences for what we claim and how we evaluate. A production authoring tool would be judged on throughput, robustness, export fidelity, and task completion time. A probe is judged on what it reveals. Accordingly, our study (Section 6.4 onward) reports qualitative shifts in how practitioners engage with accessible design, not task performance against a baseline, and the prototype deliberately leaves production concerns such as deployable export out of scope (see the Limitations and Future Work section). Treating *Skeleton* as a probe is also why we study it first with sighted practitioners alone: the probe is positioned to test whether making structure visible changes sighted authors’ reasoning, which is a precondition for, not a substitute for, evaluation with the disabled users those structures ultimately serve. We return to the boundary between what the probe makes *designable* and what only end-user evaluation can establish as *good* in the Limitations and Future Work section.

This chapter was adapted from our paper, currently conditionally accepted with IEEE VIS:

F. Elavsky, C. Nnadozie, L. Nadolskis, P. Carrington, and D. Moritz, ‘*Skeleton: Visual Authoring of Non-visual Data Experiences*’, *IEEE Transactions on Visualization and Computer Graphics*, 2026.

6.2 Overview

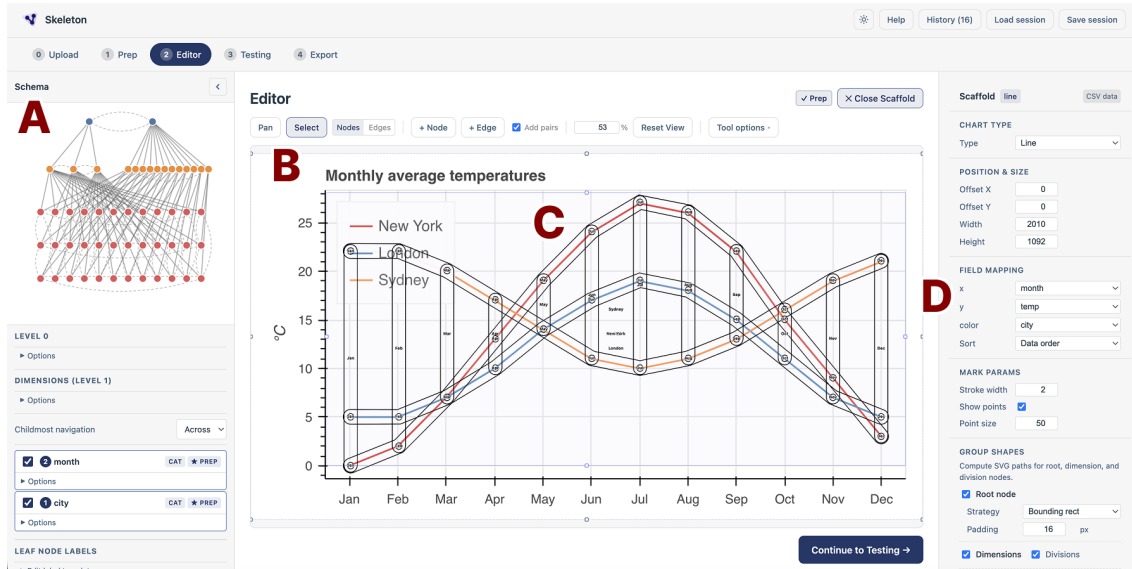


Figure 6.1: *Skeleton* being used to build a navigation structure over a line chart. **A**: a dual tree visualization of nodes and edges built for a line chart with 3 lines, spanning over 12 months. **B**: a direct-manipulation canvas that resides over a static png image of a line chart. **C**: a skeleton-like structure of nodes, edges, and group outlines that correspond to the schema shown in (A), overlaid on the visual space of the chart in (B). **D**: controls for rapidly scaffolding the nodes, edges, and their shapes shown in (C), using Vega’s visualization engine for automatically generating spatial properties.

We start this work with a provocation: How might the discipline of visualization help the discipline of accessibility?

Visualization has spent decades developing techniques for a specific class of problem: representing and interacting with information visually. Grammars of visual encoding [126, 162, 209, 212, 226], direct-interaction interfaces [44, 85, 181], and iterative feedback loops between representation and understanding [49, 73, 108] are all methods that enable abstraction and manipulation of the information that underlies the visual representation. We argue these methods have a direct application within the discipline’s own accessibility challenges.

Sighted practitioners who build accessible data visualizations face an unusual authoring problem. The non-visual navigation structures they construct (the nodes, edges, focus states, input bindings, and semantics that govern how assistive technologies traverse a chart) exist only as code. A practitioner can write a navigation hierarchy, but cannot see it, click on a node to inspect what will be announced, observe the spatial relationship between a navigation path and the chart it overlays, and then manipulate its properties through direct interaction. Every other aspect of a visualization has a visible, inspectable representation during authoring: the visual encodings are visible, the layout is visible, the interaction states are visible. And yet, navigation structure is not.

This invisibility has practical consequences. Without a way to see what they are building, sighted practitioners cannot easily catch structural errors, compare design alternatives, and iterate. Accessibility becomes downstream of every other design choice, not because practitioners choose to deprioritize it, but because the authoring conditions do not support anything else [95, 178]. The floor and ceiling of non-visual data experiences are constrained by what sighted authors can perceive of their own work.

We engage this space with the following research questions:

R1 (Qualitative, Exploratory): What challenges do sighted practitioners face when designing and engineering navigation structures for accessible visualizations?

R2 (Qualitative, Exploratory): How do sighted authors reason about the non-visual experiences that accompany their visualizations?

R3 (System, Design): How can we make the properties of accessible navigation structure visible and directly manipulable during authoring?

R4 (Qualitative): In what ways does a directly manipulable visual representation of navigation structure change how practitioners find errors and improve upon their designs?

We address these questions through longitudinal co-design with practitioners across cartography, design systems, and open-source visualization, following an action research orientation [72] in which the research team was embedded in each community’s active development work. This paper makes three contributions:

First, **we introduce technical advancements** for making the properties of accessible navigation structure visible and directly manipulable during authoring. These techniques are grounded in our co-design collaborations, which produced two foundational pieces of infrastructure: an *Inspector* that renders any navigation graph as an interactive node-link diagram, and a *Dimensions API* that formalizes a declarative grammar for expressing navigation in terms of data dimensions rather than explicit graph construction (**R1, R2, R3**).

Second, building on this infrastructure, **we present *Skeleton*, a direct-manipulation authoring environment** in which the topology, spatial mapping, semantics, and input logic of an accessible navigation structure are made visible and directly manipulable. *Skeleton* is built on Data Navigator [43], a code-based library for constructing interactive data navigation structures (**R3**). Our intention with *Skeleton* is to continue to develop it towards a usable, practical system beyond the scope of this research.

Third, **we contribute findings from an in-situ interview study** with 8 practitioners across visualization design, engineering, and research. We evaluate *Skeleton* as a design probe [69, 86] rather than a deployable system, focusing on qualitative shifts in practitioner engagement rather

than task performance. We find that making navigation structure visible shifted how participants engaged with accessible design: they reconsidered the architecture of their own visualizations, attended to a broader range of input modalities, and shifted from treating accessibility as a compliance task to treating it as a design problem (R1, R2, R4).

6.3 Related Work

6.3.1 Visualizing Non-visual Structure

Long before accessibility, computing established a practice of taking structures that are not inherently visual and giving them a visual form that can be seen and acted upon directly. Sutherland’s Sketchpad turned the constraints and topology of a drawing into a manipulable graphical object rather than coordinates in memory [192]; visual programming environments rendered computation as arrangements of on-screen objects [183], syntax-directed editors treated a program as a hierarchical structure edited through its visible derivation tree [195], and structured flowcharts gave control flow a fixed spatial form [135]. Myers later named the distinction this lineage had been drawing, separating *visual programming* from *program visualization*, depicting otherwise non-visual programs graphically [134]. A consequence is easy to overlook: many structures we now treat as inherently visual, including trees, graphs, flowcharts, and the document object model, are abstractions whose visual rendering has become so dominant that the representation now stands in for the structure.

Direct manipulation interfaces, in turn, provide a continuous representation of the objects of interest and give rapid, incremental feedback, letting people work with a structure they would otherwise hold in their heads or reach only through code [85, 180]; grammars of interactive graphics extend the same move to authoring [14, 162]. Yet across this tradition, one structure has remained conspicuously without a visual form to manipulate: the UI structure that a screen-reader traverses. This lack of visual representation is especially surprising because screen readers evolved in parallel with the emergence of graphical interfaces [197].

6.3.2 Non-visual Data Experiences

Blind people who rely on assistive technologies interact with data in fundamentally different ways than sighted users who use a direct pointer, like a mouse [101, 120, 176]. A substantial body of research has documented what these experiences look like across modalities, and what it takes to make them meaningful. In the context of “data experiences,” this paper focuses specifically on interactive navigation structures, but we briefly survey adjacent modalities to situate our contribution.

Alternative text and natural language. A dominant strand of this work concerns the generation and evaluation of textual descriptions of visualizations [96, 98, 102, 109]. More recent LLM-driven systems and Q/A approaches can caption charts with varying degrees of semantic depth, some at a risk of producing bias [41, 47, 100]. While alt text makes a visualization’s message available without sight, it is by nature static: a description conveys what a visualization says, but not how a user might explore or interact with it.

Sonification, haptics, and tactile rendering. Non-visual data experiences extend well beyond text. Sonification encodes data as sound [17, 76, 116], with declarative grammars emerging for authoring these experiences [99]. Haptic and tactile representations offer another channel through refreshable displays, 3D-printed graphics, and multimodal touchscreen interactions [20, 81, 121, 153]. Recent systems integrate multiple non-visual representations of the same data [29, 82, 173].

Interactive navigation structures. The primary focus of our work centers on the state of research related to structured navigation: the traversal of data points, groupings, and interface elements through assistive technologies and keyboard input [43, 184, 204, 225]. Existing systems and interfaces have demonstrated that going beyond static descriptions to support hierarchical, traversable data structures meaningfully improves how blind users can explore and reason about charts [198, 225]. Giving users control over the textual tokens surfaced during navigation improves comprehension and agency [93]. And more recent work has found that perceptually congruent navigation structures for charts and diagrams can improve goal-driven exploration [127].

6.3.3 Authoring Non-visual Data Experiences

Accessible visualization has historically centered the experiences of disabled users, but a parallel body of work examines the experiences of visualization practitioners working on accessibility.

Practitioner challenges. Research consistently finds that sighted visualization practitioners struggle with accessibility [46, 95, 178]. Most visualizations in the wild are inaccessible and designers themselves report lacking guidance, especially for complex and interactive graphics [95, 178]. And screen reader users experience the downstream effects of these gaps: inconsistent structure, poor keyboard support, and information that is present visually but absent in the accessibility tree [46, 176]. The pattern across this work is consistent: the practitioners who build visualizations lack the tools and feedback mechanisms to make non-visual experiences effective, useful, and good.

Across practitioner-centered literature, a recurring finding is that non-visual experiences are treated as downstream of visual design choices, added after the visual representation is finished rather than designed in parallel [95, 110, 178, 227]. This sequencing has consequences: what is navigable and how it is structured is constrained by whatever visual decisions came first.

Authoring-oriented systems. A distinct line of work has focused on building authoring tools and libraries that give practitioners more tractable paths to accessible output. Few visualization tools support the kinds of interactive navigation structures that assistive technology users most benefit from [102]. Of those that do, most rely on code-based approaches [12, 43, 175, 177]. *Umwelt* [227] takes a different and notable approach: it is a structured editing environment where authors specify representations *across* modalities (sonification and visualization) in an integrated interface, where navigation is made available over the visualization using *Olli* [12]. The latest work in this space is *Arboretum*, a tool that provides automatic conversion of diagrams to a tabular structure, navigable structure, and tactile representation [216]. In *Arboretum*, the input visual is treated as the ground truth for the navigation structure, which is treated as downstream output.

Communicating visually, authoring invisibly. There is a revealing irony across this body of related work: research about navigation structures almost invariably communicates those

structures visually. Papers such as “rich screen reader experiences” [225], *ChartReader* [198], *Benthic* [127], and *Data Navigator* [43] each use visual node-link diagrams and hierarchical schematics to explain navigation paths to their readers. The same pattern holds in adjacent domains that involve structuring data for navigation, such as PDF remediation [132].

Yet, most other non-visual modalities are not only communicated but *authored* in visual and non-visual modalities. Sonification is designed in editors that are both visual and auditory [99]. Tactile graphics are composed by transcribers on a graphical canvas before being physically embossed or printed [13].

Inspecting accessibility structure. In authoring-oriented systems, none provide a visual interface through which practitioners can interactively inspect and manipulate navigation structures as a first-class design material. Structure output is either downstream of code or static visuals. Structure is always *derived* or *specified*; indirectly manipulated. Additionally, verification of the structure across all of these systems requires developers to manually navigate using a screen reader after the structure has been authored and rendered, before returning to the upstream visual design space or code. Navigation structure is thus understood and communicated visually by sighted researchers and designers, yet built entirely without visual feedback by developers; this is the gap we address.

6.4 Co-design Foundations

Our research follows an *action research* orientation [72], in which knowledge is generated by engaging with a community to solve a real problem in-situ, alongside them rather than studying it from the outside. These collaborations started from a shared motivation: practitioners needed to make their existing systems accessible to navigational assistive technologies, using *Data Navigator* [43] as a foundation. Across three projects, we worked with 12 individuals outside our research team. Three blind co-designers shaped the work throughout: CD1 and CD2 (anonymous) and a co-author on this paper, [REDACTED]. CD1’s contributions are noted in [Section 6.4.1](#), CD2’s are noted in [Section 6.5.3](#). [REDACTED] contributed to early ideation, problem formation, and framing for the project as a whole, helping define the authoring challenges that *Skeleton* addresses, in addition to feedback on study design.

The co-design literature on accessibility has centered people with disabilities as primary design partners [32, 33, 150, 198, 224, 225], and rightly so. Our co-design takes sighted practitioners as primary partners because the authoring challenges we address happen on the sighted side of the process: it is sighted authors who cannot see what they are building.

Two themes converged across all three engagements, and we present them here before describing the individual projects that surfaced them. First, **practitioners communicated and reasoned about accessible navigation using visual representations**: while designing for language, sound, and structure, our collaborators drew on paper, built wireframes of nodes and edges, and reflected on the design space using visual artifacts. Even when collaborating with blind co-designers, a visual medium was the first language of sighted authors. Second, **development that followed visual design work faced severe iteration barriers**: verifying a navigation experience required building a working code prototype and manually navigating it with a screen reader. The gap between visual design and code-based scaffolding with manual testing produced

repeated mistakes, misinterpretations, and abandoned prototypes.

These themes did not arrive as abstractions; each project below surfaced them as concrete friction, and together the three engagements produced five design goals that *Skeleton* was built to satisfy. We state them here and, in the subsections that follow, mark where each emerged:

DG1 Make structure visible and inspectable. Render the navigation structure as a manipulable object during authoring, not only as code, so that practitioners can perceive and react to it.

DG2 Specify navigation at the level of data. Provide a higher-level, data-driven abstraction so practitioners describe traversal in terms of data dimensions rather than hand-wiring every node and edge.

DG3 Shorten the iteration loop. Let practitioners verify a design without a full code build or a manual screen-reader pass.

DG4 Support image-only and bespoke visualizations. Do not require a single underlying dataset, format, or recognizable chart type, since the cases that most need accessible tooling often have neither.

DG5 Reach beyond keyboard input. Accommodate input modalities other than assuming keyboard navigation, including touch and text.

6.4.1 Geologic Map

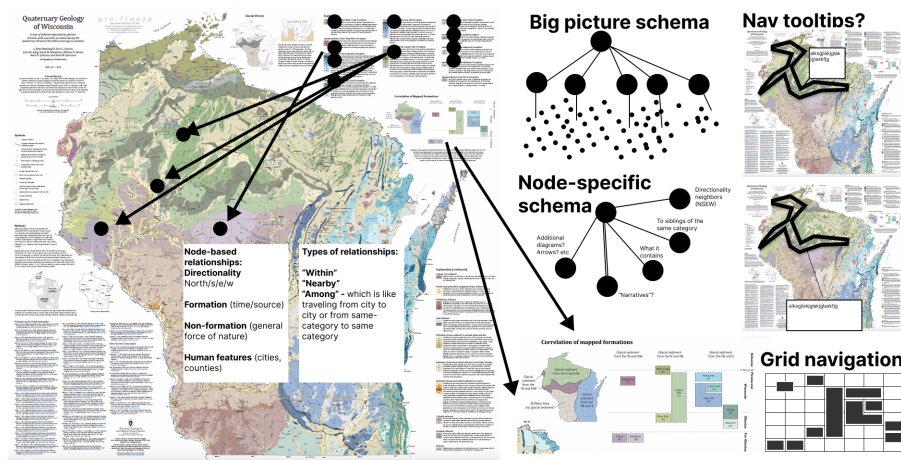


Figure 6.2: Our visual design work in Figma over a static geologic infographic map of Wisconsin. We use visual forms and illustrations over and beside the map to communicate flows, structure, navigation styles, and interaction patterns.

Our longest engagement spanned just over two years, with a cartographer, accessibility consultant, and blind co-designer (CD1) building an interactive version of a quaternary geologic map of Wisconsin [151]. The map combined dozens of irregular geographic regions organized categorically by a legend.

Early design sessions took place in Figma (Figure 6.2), where we laid out node-link diagrams of how a screen reader user might traverse the map, legend, and peripheral information. We

annotated each node with text a screen reader would announce and connected them to relevant regions. Drawing the structure made it possible to discuss design choices, catch dead ends, and debate traversal strategies. We were informally doing what *Skeleton* later formalized.

The friction began when we moved from design to implementation. We had no way to verify that our designed structure would be good or ideal, and scaffolding the project into Data Navigator was arduous: the translation from even simple navigational designs to functional code was too complex to hand off or meaningfully iterate on. The collaboration also surfaced a design-space boundary: for within-map spatial navigation, an egocentric audio-game approach [9] fit better than a node-link graph, a distinction CD1 helped surface. Reaching this boundary was only possible because we could see the structure we were trying to build (DG1). And the map’s lack of a single tabular dataset made clear that a useful navigation tool should not assume structured data or a grammar of graphics (DG4).

6.4.2 Design System Library

Our second engagement, spanning 7 months, was with 6 engineers and designers on Adobe’s React Spectrum Charts library, an open-source chart component system. We worked in their codebase and in Miro on navigation design for bar charts, clustered and stacked bars, line charts, and related types. Because we needed generalized, reusable patterns, our design problems differed from the geologic map: we needed to account for use, re-use, and edge cases across chart types.

Our Miro sessions produced two kinds of artifacts: diagrams of navigation structure for specific chart *instances* and *schema* diagrams capturing generalized patterns in dimensional terms. The concept of a “dimension” emerged naturally: common transformations on Data Navigator’s graph structure corresponded to properties within a dataset, such as a “categorical” dimension or a “numerical” dimension, where traversal took place within collections of grouped siblings. This dimensional thinking directly foreshadowed the Dimensions API (Section 6.4.4).

The dominant friction was iteration speed. We could sketch and converge on a schema in Miro in an hour, but verifying the design in a functional example required embedding Data Navigator into a large codebase, implementing changes, rebuilding, and manually testing with a screen reader. The collaboration also surfaced a limitation: mobile screen reader navigation uses swipe gestures rather than keyboard input, and our keyboard interaction model did not account for it. Each of these frictions pointed somewhere: the dimension concept toward a higher-level abstraction (DG2), the cost of verifying each change toward a faster loop that does not require a full build or screen reader (DG3), and the mobile-gesture gap beyond keyboard input (DG5).

6.4.3 Open Source Visualization Library

Our third collaboration, spanning nearly two years, was with Quansight Labs and contributors to Bokeh, a Python-based open-source visualization library. We performed an accessibility audit [45] based on Chartability [42] and identified that Bokeh visualizations with interactive chart elements needed to be navigable by assistive technologies.

Unlike Adobe, Bokeh has no native chart types. Its API operates at the level of glyphs, renderers, and data sources, which users assemble freely. There was no standard unit around

which to anchor a navigation pattern, and any grammar or tooling would need to accommodate an open-ended range of encoding combinations. The iteration gap from Adobe was present in a more severe form: making library-wide contributions to a fully open-source project required incremental tooling for the most common cases. For Bokeh, we needed to test functional, data-driven navigation abstractions without fully embedding Data Navigator into the library. A library with no native chart types required the abstraction to be data-driven and chart-type-agnostic rather than tied to fixed templates, sharpening **DG2** into a concrete constraint.

6.4.4 Infrastructure from Practice

The three collaborations converged on the design goals above. Two of them demanded new infrastructure before *Skeleton* could exist: a way to visually render and inspect navigation structures (**DG1**), and a higher-level, data-driven abstraction for specifying navigation without hand-wiring every node and edge (**DG2**). We built two pieces of infrastructure to address these, described next; the remaining goals (**DG3**, **DG4**, **DG5**) are met by *Skeleton*'s authoring environment (Section 6.5). These infrastructural improvements as well as *Skeleton* are open source on [our GitHub repo](#).

6.4.4.1 An Inspector Gadget

We built an *Inspector* (`@data-navigator/inspector`) to render any Data Navigator structure as an interactive node-link graph using D3, with an accompanying console for debugging. Hierarchical structures are colored by level; edges are drawn as directed links; the entry point is visually marked. The *Inspector*'s graph can itself be navigated using Data Navigator, with visual focus tracking during navigation, allowing practitioners to manually verify structure and reachability.

This made structural verification immediate: a practitioner could generate a structure and check at a glance whether the hierarchy had the right levels, whether circular extents produced expected wrap-around edges, or whether a particular path was reachable. The interactive console logs API information and underlying data when nodes are activated, and hovering or focusing logged information highlights the corresponding node in the graph.

The *Inspector* remains a developer tool, however. It requires code familiarity to attach and renders structure as an abstract graph with no connection to the spatial layout of the underlying visualization. It shows topology but not instantiated geometry: a practitioner can see that two nodes are connected but not where their focus indicators will appear on-screen. This gap between navigable structure and its spatial instantiation over a rendered chart motivated *Skeleton*.

6.4.4.2 Alternative Dimensions

The original Data Navigator library requires explicit graph construction: practitioners specify nodes, edges, and navigation rules by hand. This is general but scales poorly. Our *Dimensions API* addresses this.

Our new vocabulary mirrors how practitioners already think about their data, much as a grammar of graphics lets a designer write a visualization at the level of data fields rather than primitive

marks. Rather than specifying a graph directly, a practitioner describes the *dimensions* of their data, the meaningful axes along which a user might want to navigate, and a single build step constructs the full node-link structure automatically. Notably, this enables systems to programmatically construct navigable structures as long as dimensions are configurable. Each dimension declares a type (categorical or numerical) and a `dimensionKey` naming the field it traverses, together with a `behavior` block that governs traversal (how navigation behaves at group boundaries, and whether leaf nodes are reachable laterally across divisions) and an `operations` block that governs how the data is shaped into the hierarchy (sorting, filtering, and binning numerical ranges). [Section 6.4.4.3](#) gives a deeper look into the API’s parameters.

This data-dimensions framing both aligns with and departs from *Olli* [12], which similarly lets authors express a navigable structure over a chart from a higher-level specification rather than hand-built nodes. Both share the premise that fields, not graph primitives, are the right authoring unit. The difference is what the specification produces and where it lives: *Olli* compiles a chart specification into a traversable tree that an author does not subsequently manipulate as a graph, whereas the *Dimensions API* generates an explicit Data Navigator node-link structure whose every node, edge, and rule remains addressable, so that the declarative output is itself the object a practitioner can then inspect in the *Inspector* and edit directly in *Skeleton*.

6.4.4.3 Dimensions API Reference

Each dimension in the *Dimensions API* ([Section 6.4.4](#)) declares a type (categorical or numerical, inferred from the data when omitted) and a `dimensionKey` naming the data field it traverses. Two further groups of properties govern behavior. A `behavior` block controls traversal: `extends` sets boundary behavior (`terminal` stops at the edges of a group, `circular` wraps around, and `bridgedCousins` carries focus across to the nearest element of an adjacent group rather than stopping or wrapping), and `childmostNavigation` determines whether leaf-level nodes are reachable laterally across a dimension’s divisions (`across`) or only within a single division before returning to a parent (`within`, the default). An `operations` block controls how the data is shaped into the hierarchy: a `sortFunction` orders divisions and leaves, a `filterFunction` selects which data participate, and `createNumericalSubdivisions` bins a numerical dimension into a chosen number of ranges.

The generated structure is a multi-level hierarchy: each dimension produces a root node, below which division nodes group the data, below which leaf nodes represent individual data points. Multiple dimensions over the same dataset share leaf nodes, so users navigating via different dimensions reach the same data through different paths.

To make the input and output concrete, the examples below use this small four-row dataset of average temperatures, with an explicit `id` on each row:

```
const data = [
  { id: '_0', month: 'Jan', city: 'NYC', temp: 0 },
  { id: '_1', month: 'Jul', city: 'NYC', temp: 25 },
  { id: '_2', month: 'Jan', city: 'Sydney', temp: 22 },
  { id: '_3', month: 'Jul', city: 'Sydney', temp: 8 }
];
```

With the Dimensions API, bar chart navigation that would otherwise require constructing every node and edge by hand is expressed as a single dimension declaration:

```
dimensions: {
  values: [
    {
      dimensionKey: 'month',
      type: 'categorical',
      behavior: { extents: 'circular' }
    }
  ]
}
```

From this declaration, the build step does three things automatically. It assembles the nodes and the parent-child and sibling edges that connect them, including, when multiple dimensions are declared, the cross-dimension edges that let a user move between hierarchies over shared leaves. It applies the boundary and childmost rules to decide what happens at each edge of the structure. And it assigns keyboard navigation rules to each dimension, drawing from a pool of directional key pairs; because that pool is finite, the API recommends keeping a structure to six dimensions or fewer and asks for explicit navigation rules beyond that point.

This bounds the structures the abstraction expresses by default: it generates the regular, dimensionally-organized hierarchies that recur across common chart types, and structures that do not decompose into a small set of data dimensions (or that exceed the directional-key budget) fall back to explicit graph construction or manual editing in the editor.

The abstraction is otherwise chart-type-agnostic: bar charts, scatter plots, line charts, and layered charts all use the same vocabulary, with different combinations producing different navigation topologies. This directly addressed Bokeh's problem, where no native chart types exist to anchor a pattern: the API mirrors data fields and encoding choices as a set of dimensions, so a thin wrapper can map an arbitrary glyph assembly onto a handful of recurring dimensional shapes ([Section 6.4.3](#)).

The vocabulary composes to richer topologies simply by adding dimensions. Declaring two categorical dimensions over the same dataset, for instance a temperature series recorded by month and by city, produces a dual hierarchy in which left and right traverse one axis and up and down traverse the other, with both sharing the same leaf nodes; here the month dimension wraps at its boundaries while the city dimension stops:

```
dimensions: {
  values: [
    {
      dimensionKey: 'month',
      type: 'categorical',
      behavior: {
        extents: 'circular',
        childmostNavigation: 'across'
      }
    },
  ],
}
```

```

    {
      dimensionKey: 'city',
      type: 'categorical',
      behavior: {
        extents: 'terminal',
        childmostNavigation: 'across'
      }
    }
  ]
}

```

This two-dimension declaration assigns a keyboard control to each axis. [Table 6.1](#) lists the controls Data Navigator generates for it; the arrow keys move between siblings along each dimension, Enter drills toward the data, and each dimension receives its own drill-out key from the default key pool (here W and J).

Table 6.1: Keyboard controls generated for the two-dimension (month, city) structure over the sample dataset. Movement along month wraps (circular); movement along city stops at the ends (terminal).

Key	Action
← / →	Move to previous / next month (wraps Dec–Jan)
↑ / ↓	Move to previous / next city (stops at ends)
Enter	Drill in (dimension → division → data)
W	Drill out along the month grouping
J	Drill out along the city grouping
Esc	Exit the structure

Finally, to show what the build step emits, the two-dimension declaration above, applied to the four-row dataset, expands into the nodes and edges below. Each dimension produces a dimension node carrying its `dimensionKey`, a set of division nodes (one per distinct value, each tagged with the field it was derived from), and the shared leaf nodes that carry the original rows. A node’s edges list the edges it participates in; each edge names a source, a target, and the `navigationRules` that traverse it. Crucially, a single edge is bidirectional: the left/right pair on one edge means pressing left moves toward its source and right toward its target, so the month and city axes at the leaf level each collapse to one edge with two rules (abbreviated here for space):

```

// nodes (excerpt: one division + one leaf per axis)
{
  month: {
    id: 'month',
    data: { dimensionKey: 'month' },
    edges: ['month-Jan', 'any-exit']
  },

```

```

city: {
  id: 'city',
  data: { dimensionKey: 'city' },
  edges: ['city-NYC', 'any-exit']
},
'Jan': {
  id: 'Jan',
  derivedNode: 'month',
  data: { month: 'Jan' },
  edges: [
    'Jan-Jul',
    'month-Jan',
    'Jan-_0'
  ]
},
'NYC': {
  id: 'NYC',
  derivedNode: 'city',
  data: { city: 'NYC' },
  edges: [
    'NYC-Sydney',
    'city-NYC',
    'NYC-_0'
  ]
},
_0: {
  id: '_0',
  data: { month: 'Jan', city: 'NYC', temp: 0 },
  edges: [
    'leaf-_0-_1',      // month axis (l/r)
    'leaf-_0-_2',      // city axis  (u/d)
    'Jan-_0', 'NYC-_0',
    'any-exit'
  ]
}
// Jul, Sydney, and _1/_2/_3
// follow the same pattern
}

// edges
{
  'month-Jan': {
    source: 'month',
    target: 'Jan',

```

```

    navigationRules: ['drill-out_month', 'drill-in']
  },
  'city-NYC': {
    source: 'city',
    target: 'NYC',
    navigationRules: ['drill-out_city', 'drill-in']
  },
  'Jan-Jul': {
    source: 'Jan',
    target: 'Jul',
    navigationRules: ['left', 'right']
  },
  'NYC-Sydney': {
    source: 'NYC',
    target: 'Sydney',
    navigationRules: ['up', 'down'] },
  'leaf-_0-_1': {
    source: '_0',
    target: '_1',
    navigationRules: ['left', 'right']
  },
  'leaf-_0-_2': {
    source: '_0',
    target: '_2',
    navigationRules: ['up', 'down']
  },
  // ...etc
}

// navigationRules
{
  left: { key: 'ArrowLeft', direction: 'source' },
  right: { key: 'ArrowRight', direction: 'target' },
  up: { key: 'ArrowUp', direction: 'source' },
  down: { key: 'ArrowDown', direction: 'target' },
  'drill-in': { key: 'Enter', direction: 'target' },
  'drill-out_month': {
    key: 'KeyW',
    direction: 'source'
  },
  'drill-out_city': {
    key: 'KeyJ',
    direction: 'source'
  }
}

```

}

Every node, edge, and rule in this output remains individually addressable: this is the structure the *Inspector* renders and that practitioners select and edit directly in *Skeleton*, rather than a compiled artifact they cannot reopen.

6.5 *Skeleton*: System Design

With a visual structure renderer (the *Inspector*) and a declarative abstraction for producing navigation structures (the *Dimensions API*), the remaining problem was to bring these capabilities into an integrated, direct-manipulation authoring environment where practitioners could not only see structure but manipulate, test, and iterate on it with immediate feedback. *Skeleton* is that environment. It integrates both and extends them with a guided preparation phase, a spatial rendering canvas, and a live testing mode, reversing the *Inspector*'s code-first direction: practitioners design a navigation structure visually and inspect the code representation as a consequence of that design. Each of its authoring techniques makes visible a specific property of non-visual interaction that sighted practitioners otherwise cannot see during authoring: the topology of what is navigable, the spatial mapping of where navigation lives over the chart, the semantics of what is announced, and the temporal sequence in which a user encounters nodes.

A principle runs through the environment: everything *Skeleton* generates is an editable default, not a fixed output. The *Dimensions API* proposes a topology, the scaffold proposes spatial positions and group outlines, and the preparation wizard proposes labels and rules, but each is materialized as concrete nodes, edges, positions, and text that a practitioner can then select and override by direct manipulation, or bypass entirely by constructing nodes and edges manually over an image. The declarative and automated layers lower the cost of a reasonable starting point; they do not constrain what the final structure can be.

6.5.1 Staging: Input and Preparation for Authoring

The authoring workflow proceeds through four stages: upload, prepare, edit, and test. Making a visualization accessible involves decisions about what is navigable, how navigation is triggered, and where focus indicators appear in space. These decisions are related but distinct, and collapsing them into a single undifferentiated interface, as code-only workflows effectively do, makes each one harder to reason about. The stage architecture surfaces them as separate concerns. It also preserves a direct correspondence between what practitioners see in the interface and the structure of the Data Navigator API: each stage maps onto a distinct layer of the API, lowering the barrier to moving from visual authoring into code when production deployment requires it.

Upload. The upload phase is deliberately permissive. Practitioners can bring a dataset, a chart image, both, or neither. When a dataset is present, *Skeleton* parses its fields and infers dimension types automatically, producing a default, starting configuration for the *Prepare* step. When only an image is present, practitioners proceed directly to editing and construct nodes and edges manually over the image. Because dimensions are derived from whatever fields a dataset actually carries rather than from a fixed, pre-registered schema, the approach does not require the data to be known in advance: a different dataset simply yields a different set of inferred

dimensions, and a visualization with no tabular dataset at all is handled through the image-only path. This was motivated by our geologic map co-design work: many bespoke visualizations do not have a single underlying dataset, and any tool that requires structured data as a precondition excludes the cases that need it most. *Skeleton* can be applied to any 2D image surface, not only to visualizations in the conventional sense.

Prepare. The prepare stage addresses a hard authoring bottleneck: not placing nodes, but deciding what structure to build at all. A practitioner who has never designed a navigation structure faces an open configuration space with no obvious entry point. The prep stage presents a four-chapter Q&A wizard that moves through authoring decisions sequentially: (1) whether the chart should have a root node and what it should announce, (2) which data fields should be navigable dimensions and how those dimensions should behave at their boundaries, (3) which keyboard interactions each dimension should be assigned to, and (4) what text labels each level of the hierarchy should produce. Each chapter is accompanied by an illustrative schematic diagram of which part of the hierarchy is being edited as well as a diagram showing examples of what these decisions look like when showing on a chart. The wizard’s output populates a configuration in the editor that practitioners can then inspect, refine, and revise.

6.5.2 Edit: Interacting with Topology, Layout, and Semantics

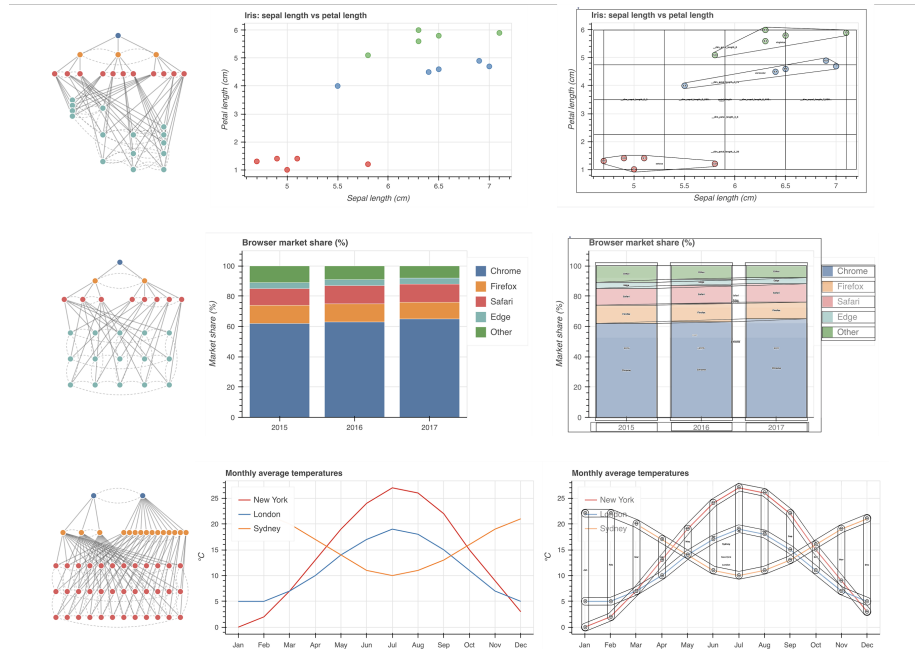


Figure 6.3: Input data transformed into a navigable structure using the *Dimensions API* and visualized with our *Inspector* gadget (left). The input chart (middle). The navigable structure is transformed and drawn over the chart using the *Scaffolding Engine* (right).

Seeing the system, seeing the experience. The editor is *Skeleton*’s primary authoring environment (??) and presents two interlinked representations of the same navigation structure. A

schema panel shows the structure as an abstract hierarchical tree layout that makes levels and parent-child relationships immediately readable. A graph canvas shows the same structure rendered as geometric elements positioned over the uploaded chart image, representing what an end user would encounter spatially. These two representations are bidirectionally linked: selecting a node in either view propagates the selection to the other, so practitioners can simultaneously hold in mind both the abstract topology of what is navigable and the spatial instantiation of where that navigation will live. The dual-view design makes visible a real conceptual divide between the system’s model of navigation and a user’s experience of it, one that practitioners recognize once they can see it, even without prior vocabulary for it.

Leveraging visualization as a scaffolding engine. Manually positioning leaf nodes over each data mark is the most mechanical step in the authoring workflow, and it scales poorly with dataset size. In our early pilot sessions, actually placing nodes in the canvas space was the slowest and most tedious part of the process. To address this, *Skeleton* includes a scaffold tool (Figure 6.3) that automates spatial placement by repurposing Vega [162] as a layout engine.

The scaffold renders a Vega chart specification to a hidden, off-screen container and extracts node positions via the library’s internal scale and view APIs, using the rendering engine purely as a coordinate computation step: no actual chart is ever shown to the practitioner. Coincidentally, none of our co-designers were crafting visualizations using Vega or Vega-Lite (or derivatives), yet the Vega rendering engine could faithfully reconstruct every necessary mark position over the underlying, inaccessible data visualization provided to *Skeleton*.

Additionally, positions and outline strategies for category-level group nodes are computed from the leaf positions using geometric algorithms: a union of child node paths, a convex hull, a grid over numerical bounds, or a bounding rectangle. These designs were consistently produced as ideal treatments during co-design, so we chose them for our starting set of outline strategies.

The scaffold is optional and works by generating synthetic placeholder positions that practitioners can adjust manually. It dramatically improved authoring speed: in light pilot tests, a research team member completed a three-dimension structure without scaffolding in 8 minutes 22 seconds and with scaffolding in 56 seconds. A co-designer completed the task incorrectly (failing to account for one dimension’s division nodes) in 13 minutes 7 seconds without scaffolding, and correctly in 2 minutes 44 seconds with it.

Specifying token patterns, editing instances. Selecting any node populates a properties panel with spatial properties (position, size, shape) and semantic properties (ARIA role, description, and a label template editor; Figure 6.4). The label template allows practitioners to assemble the text a screen reader will announce at that node from tokens drawn from data fields, including aggregate statistics at group-level nodes and precise data values at leaf nodes [93]. Editing labels can apply to all nodes of the same type or to a single instance.

At the bottom of the semantic section, a live preview displays the full assembled announcement string in the exact form a screen reader would produce: role, semantics, group membership, and label combined into a single rendered output that updates in real time. Prior to this preview, understanding what would be announced at a given node required running a screen reader and navigating to it sequentially. In deeply nested structures, this meant spending several seconds listening to labels and drilling in. This was consistently the main point where bugs were produced and missed during our co-design work.

The preview makes text announcements inspectable as visible, editable objects. This surfaces

AGGREGATE SUMMARIES FROM CHILDREN

☐ Count of children

☐ Min and Max of temp

☐ Sum of temp

☒ Average of temp

Trend direction and R²

X variable

Y variable

month
temp

☒ Trend direction
 ☐ R² value

Label template

{value:"city"}, trend for {key:"temp": {trend:"
Clear

LABEL

PREVIEW:

New York, trend for temp: flat, average temp: 13.78

City 3 of 8.

Figure 6.4: Group label pattern builder, including an array of aggregate summary options, template formatter field, and preview.

a class of highly specific, low-level problems that code-only workflows leave invisible: redundancies in announced text, missing contextual framing, and label ordering and punctuation that affects comprehension and reading speed. Of all the authoring decisions *Skeleton* exposes, label templates involve the most degrees of freedom and have the most direct bearing on the quality of the non-visual experience. Getting the structure right ensures navigability; getting the labels right determines whether navigation communicates anything meaningful.

6.5.3 Test: Debugging Interaction Interactively

The testing stage allows practitioners to navigate the structure they have built using the same keyboard input and navigation rules that assistive technologies would use, without leaving the tool. When a practitioner enters testing, the three Data Navigator modules are instantiated in sequence: the structure module rebuilds the navigation graph from the current configuration, the rendering module creates an HTML layer positioned over the chart image at each node's spatial coordinates, and the input module registers keyboard listeners for all navigation rules. The result is a live, keyboard-navigable structure. An event log records navigation events in order, letting practitioners verify that all nodes are reachable and that label sequences make sense when encountered serially rather than read simultaneously.

The abstract graph continues to display during testing. As the practitioner navigates, the focused node is highlighted simultaneously in the canvas (showing its spatial position over the chart image) and in the abstract graph (showing its structural position in the hierarchy). Practitioners

can verify at a glance both where focus is and what role it occupies. A mobile-friendly text-chat mode is also available, in which practitioners navigate by typing natural language commands, motivated by the Adobe collaboration ([Section 6.4.2](#)).

Practice-based validation. The testing stage also served, during development, as the primary debugging interface for *Skeleton*’s own data pipeline. Additionally, CD2, a subject matter expert who professionally evaluates interfaces for screen reader access and is familiar with other visualization navigation systems, used *Skeleton*’s testing stage to evaluate navigation output across several chart types and dimension configurations: line charts (3 configurations), bar charts (2), stacked bar charts (3), and scatterplots (4). This evaluation followed a manual, systematic approach combining standards-based criteria with expert screen reader testing. Scatterplots required the most iteration, surfacing bugs in Data Navigator’s core library that were then fixed. CD2 also recommended that we use list-based navigation while in the editor (not in testing) in case users build themselves into a keyboard trap.

6.5.4 System Implementation: Technical Details

This appendix details the three technical components behind *Skeleton*’s spatial authoring ([Section 6.5](#)): how the scaffold repurposes a visualization rendering engine to approximate mark positions, how group outlines are computed from those positions, and how a lightweight, non-ML computer vision pass aligns the scaffold to an uploaded chart image. We intentionally chose deterministic math approaches at any opportunity, for speed and inspectability benefits.

6.5.4.1 Vega as a Deterministic Layout Engine

The scaffold ([Section 6.5](#), “Leveraging visualization as a scaffolding engine”) treats a visualization grammar as a coordinate oracle rather than a rendering target. From the active configuration, which carries a chart type, field-to-channel mappings (`xField`, `yField`, and an optional `colorField`), mark parameters, and an outer plot box, *Skeleton* assembles a *Vega-Lite* [162] specification and renders it through `vega-embed` into a hidden, off-screen container using the SVG renderer. The container is never attached to the visible document; it exists only so that *Vega* will compile a full scenegraph and, with it, the scale functions that map data values to pixels.

Positions are then read directly from the compiled view rather than parsed from the rendered output. For each leaf node, *Skeleton* evaluates the view’s `x` and `y` scales on the corresponding data values: a band scale supplies a bar’s horizontal position and, through its `bandwidth`, its width, while a linear scale maps a quantitative value to a pixel height with the baseline taken at the scale’s zero. Stacked bars accumulate per-segment offsets in data space before scaling; clustered bars add the sub-band offset; scatter and line marks evaluate both scales at each point. Because the scale functions are pure, this path needs no DOM measurement and is fully deterministic: identical configurations yield identical coordinates. A secondary path, used when scale evaluation does not apply, parses the hidden SVG and reads mark bounding boxes instead.

The configuration distinguishes the inner mark area from the outer box. *Vega-Lite*’s `width` and `height` describe the inner area in which marks are drawn, while a `padding` object reserves space for axes and labels; *Skeleton* treats `plotWidth` and `plotHeight` as the outer, border-box dimensions and derives the inner area by subtracting padding. A mark at normalized position

f within the inner area therefore lands at image coordinate $o + p + f w_{\text{in}}$, where o is the plot offset, p the leading padding, and w_{in} the inner extent. This relation is what later makes padding recoverable.

A practical subtlety motivates the rest of this appendix. The engine that produced the uploaded image is generally unknown and is rarely *Vega*; our co-designers authored charts in other tools entirely (Section 6.4). What transfers across engines is the *relative* geometry of the marks, the spacing of bars or the placement of points within the data rectangle, not the absolute pixel offsets, which depend on how much room each particular chart reserved for its axes, ticks, labels, and title. The scaffold reconstructs the relative geometry faithfully; reconciling it with the absolute placement in the image is the task taken up by the padding estimation described below.

6.5.4.2 Computing Group Outlines

Once leaf positions exist, the group nodes above them (divisions, dimensions, and the root) receive a visible outline computed purely from the geometry of their descendants. Each leaf contributes a rectangle (for bars) or a circle (for points), and a chosen strategy turns the collection into a single SVG path string. These functions live in the Data Navigator core (`geometry.ts`) and are side-effect free, taking arrays of rectangles or circles and an optional padding term and returning a path `d` attribute, so they can be reused outside *Skeleton* and packaged for export.

Four strategies cover the cases that recurred during co-design (Section 6.4). A *bounding rectangle* encloses all descendants in the single axis-aligned box given by their combined extent. A *union* emits one rectangular subpath per mark and relies on the SVG fill rule to render disconnected regions, which suits groups whose marks are spatially separated. A *convex hull* collects the corner points of every rectangle (or sixteen sampled points around every circle) and runs a Graham scan [63], optionally pushing each hull vertex radially outward from the centroid to add padding; this gives a tight, convex boundary appropriate to scatter clusters and bar groups. For line series, an *offset path* traces the polyline of points at a fixed perpendicular distance on each side, using per-vertex averaged normals for miter joins and semicircular arc caps at the ends, producing a closed band that follows the line. Numerical dimensions additionally support a grid drawn over the dimension’s value bounds.

Because every outline derives from the same leaf coordinates the scaffold computed, changing a mark position, a chart type, or the padding propagates automatically to the group shapes, and authors edit outline strategy and padding as ordinary node properties rather than drawing shapes by hand.

6.5.4.3 Estimating Plot Padding from the Image

The scaffold’s layout is internally consistent but free floating: it knows where marks sit relative to one another, not how the uploaded image split its canvas between the data rectangle and the axis space, which authors originally recovered by dragging the four padding values until the scaffold marks settled onto the real marks. Our most recent work automates it with a small, fast, dependency-free computer vision pass that runs entirely in the browser, in the spirit of chart reverse-engineering that recovers marks from bitmap images [147, 163] but reduced to coarse mark localization, since the scaffold already supplies the data.

Detection proceeds on a downscaled copy of the plot region. The image is drawn into an off-screen canvas with its longer side capped near one thousand pixels, and a binary foreground mask is built by thresholding the pixel data [144]. The key observation is that data marks are almost always rendered in color while chart chrome (axis lines, gridlines, tick labels, titles, and legend text) is grayscale. A saturation threshold therefore isolates the marks and prevents a dark axis line from bridging adjacent bars into a single blob; for the rare monochrome chart the pass falls back to a dark-ink mask. Connected-component labeling [155] with eight-connectivity groups the mask into candidate marks, each carrying a bounding box, centroid, area, and mean color. Two filters clean the result: components far smaller than the median are discarded as legend swatches, and for line and area charts, where the mark is one long stroke rather than discrete glyphs, the endpoints of the dominant component are used in place of separate marks. The pass returns the extreme detected marks along each axis (leftmost and rightmost, topmost and bottommost) as points in image space.

Padding then follows in closed form. Recall that a scaffold mark at normalized inner position f sits at $o + p + f w_{\text{in}}$, and that f is invariant to padding because it is fixed by the data and the scale. Taking the two extreme scaffold marks on an axis, with inner positions f_{lo} and f_{hi} , and the two detected image positions d_{lo} and d_{hi} they should align to, the inner extent and the two padding values follow while the outer box (o, W) is held fixed:

$$w'_{\text{in}} = \frac{d_{\text{hi}} - d_{\text{lo}}}{f_{\text{hi}} - f_{\text{lo}}}, \quad p'_{\text{lo}} = d_{\text{lo}} - o - f_{\text{lo}} w'_{\text{in}}, \quad p'_{\text{hi}} = W - w'_{\text{in}} - p'_{\text{lo}}.$$

The two axes are solved independently. Several guards keep the result trustworthy: an axis whose two marks are nearly coincident in f is ill-conditioned and is skipped, negative padding is clamped, and a solved inner extent larger than the outer box signals that the box itself is too small and is reported rather than forced. The pass runs once automatically when an image, a chart type, field mappings, and at least two scaffold marks are all present, and on demand thereafter from a control in the scaffold panel; after applying the padding it re-measures the residual between detected and scaffold marks as a one-shot verification.

In practice this recovers padding consistent with the original chart across the four canonical types we tested (bar, stacked bar, line, and scatter), shown in Figure 6.5. On the bar example, for instance, the solver recovers a left padding close to the width of the chart’s y -axis label gutter and a small right padding, with no manual adjustment. The approach is deliberately heuristic and assumes colored marks over light chrome; its single entry point makes a heavier detector (for example an `OpenCV` build) substitutable without changing the padding solver. We regard this as promising but early, and a fuller evaluation across more diverse chart images remains future work.

6.6 User Study

Our co-design work was motivated by the problems of the communities the research team was embedded in, and our co-designers were deeply familiar with the problem space, having spent months or years working on accessible navigation. We still needed to understand what happens when practitioners who are *not* embedded in this process design, author, and debug navigation

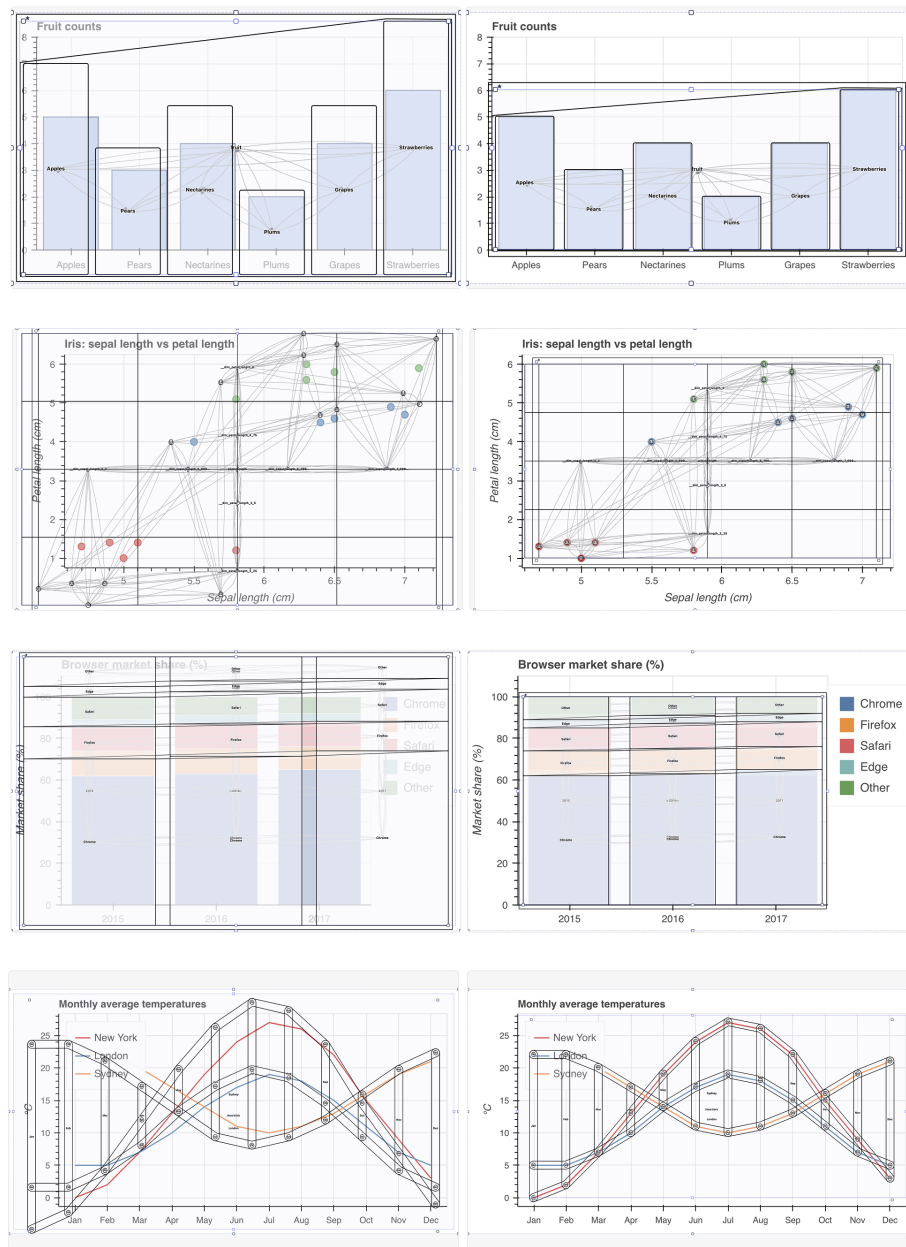


Figure 6.5: Computed layout for marks via Vega's visualization engine plus group outlines (left) and with padding estimation using our non-ML computer vision approach (right) for various types of common visualizations.

structure visually: whether the representations we built are legible to them, whether the techniques change how they reason about accessible design, and what new questions or problems emerge when navigation structure becomes visible. These are empirical questions that required a study.

To evaluate how *Skeleton* influences the way practitioners engage with accessible design, we conducted an in-situ interview study with 8 participants across visualization design, engineering, and research. The study was conducted remotely over video call, took approximately 45–60 minutes per session, and was approved by our IRB. Participants provided verbal consent at the start of each session. Video and audio were not recorded, however some participants consented to share the data/image they brought to the study as well as screenshots of their workflow; data collection was note-based throughout.

6.6.1 Participants

Participants were recruited through snowball sampling within the visualization and accessibility community, and through referrals from co-designers involved in our earlier collaborations. We asked each participant to self-report their primary work role (engineering, research, design, or student) and their existing level of accessibility expertise on a 1–5 Likert scale. We also asked whether the visualizations they build are ever bespoke, that is, custom rather than instances of a recognizable, standard chart type. This distinction mattered because bespoke visualizations represent an especially underserved case in accessible design tooling: no library pattern applies, and every navigation structure must be designed from scratch. Participants were not compensated.

6.6.2 Procedure

Each 45-minute session proceeded in four phases. Before the session, all participants were asked to prepare a chart image they were currently working on or had recently built, for use in the third phase.

Phase 1: Introduction and demographics (5 minutes). After obtaining verbal consent and recording a pseudonym, we collected self-reported role, accessibility expertise level, and whether the participant regularly builds bespoke visualizations.

Phase 2: Generic chart think-aloud [2] (10 minutes). Participants used *Skeleton* on a provided bar chart and dataset of fruit counts (Apples, Pears, Nectarines, Plums, Grapes), asked to design an accessible navigation experience for a screen reader user with no instructions on how the tool worked. The tool loaded with a default structure having both a categorical dimension (`fruit`) and a numerical dimension (`count`) active. This default was intentionally problematic for two reasons: numerical navigation sorts by count value and groups data into subdivided ranges, producing a traversal order different than the visual layout an additional, largely unhelpful level in the hierarchy for such a simple chart. We observed how participants reasoned about what they saw and whether and how they noticed this extra dimension.

Phase 3: Own-chart think-aloud (15 minutes). Participants loaded their own chart image and attempted the same task, except they were also asked to explain their graphic to the research team (purpose, role, data, and domain). This phase was open-ended: charts ranged from standard

types to bespoke visualizations, and the goal was to observe how participants reasoned about navigation structure when the context was their own work.

Phase 4: Reflective interview (15 minutes). We conducted a semi-structured interview in which participants reflected on their decisions in Phase 2 versus Phase 3, their experience with the generic versus their own chart, and their assessment of the tool’s capabilities and limitations. We asked what felt possible or impossible, what they wanted to do that they could not, and what they found themselves thinking about that they had not considered in Phase 2.

6.6.3 Analysis

Notes from each session were compiled into a shared document. Participant quotes reported in the results are reconstructed from these researcher notes, not verbatim transcripts. We analyzed the data using a combination of thematic analysis [16] and affinity diagramming [70], iterating across both methods to surface recurring patterns while preserving the specificity of individual participant experiences. Analysis attended particularly to differences in how participants engaged with accessible design before and after using the tool, the range of input modalities and user scenarios they considered, and moments when participants reconsidered or wished to redesign their own visualizations.

6.7 Results

We organize our findings into five themes that emerged from thematic analysis and affinity diagramming across all eight sessions. Each theme captures a qualitative pattern in how practitioners engaged with accessible navigation design when its structure was made visible and manipulable. We report these findings descriptively and ground them in specific participant moments; interpretation follows in the Discussion.

6.7.1 Seeing Navigation Made Structural Problems Legible as Design Problems

The generic bar chart in Phase 2 loaded with an intentionally problematic default: both a categorical dimension (`fruit`) and a numerical dimension (`count`) were active, producing overlapping navigation structures with different traversal orders over the same data. This configuration is a poor design choice, arguably a design failure, but one that would be difficult to detect in code alone (R1, R4).

Participants varied widely in how quickly they recognized the problem. P1 turned off the numerical dimension within seconds of seeing the editor, without commenting on it. Most participants, however, initially struggled to understand what they were seeing, remarking on the unfamiliar structure: “what is this? what are these?” when encountering the numerical divisions for the first time. P8 spent time trying to guess what the extra divisions represented but did not remove them during Phase 2, only realizing during Phase 3 that the additional dimension was “probably bad.” P2 and P5 expressed suspicion early: P5 asked, “Is this too much data? This

seems like way too much to just navigate through,” and P2 noted, “I feel like a lot of data points would be bad, yeah? Like, too many at once is bad?”

The testing stage (Section 6.5.3) proved critical for resolution. P4, P5, and P7 each removed the extra dimension only after navigating the structure with keyboard input in testing mode, where the traversal sequence made the redundancy experientially apparent. In total, five of eight participants resolved the problem during the session: P1 and P2 during editing, and P4, P5, and P7 after testing.

Beyond the intentional default, participants identified other problems through visual inspection. P8 reacted to a generated node name: “Okay `dim_fruit` node...that is horrible, what is that?” During Phase 3, P7 looked at the edges of their multi-line chart and asked, “are all these bad? Is it bad that I don’t even really know what the takeaway of this [structure] is?” P3, seeing a full hierarchy for their own simple six-item bar chart (during Phase 3), concluded “I should just skip the root and grouping and go straight to the data. This seems like too many steps.” In each case, the visual representation of their navigation structure motivated judgment about potential negative design qualities.

6.7.2 Practitioners Developed a Designerly Interest in What Constitutes Good Navigation

The most pervasive pattern across sessions was that participants began asking design questions about navigation quality, unprompted by any instruction or guidance from the research team (R2). These questions went beyond identifying errors: participants wanted to know what *good* navigation would be for their charts.

P2, working through the bar chart in Phase 2, deliberated over boundary behavior: “Loop back or stop? I don’t think there is a right way. I will just pick *fruit* for now and *loop* and see what this does.” P6, who brought a bespoke flower visualization, wondered how to translate the affective quality of their chart: “I think my visualization should be more about the vibes, but I don’t know how to make the alt text have good vibes. What is *fun*?” P4 asked fundamental questions about the interaction model itself: “Why do screen readers and keyboards have to work this way? Do people like that?... why do we navigate?” And later after testing, P4 concluded, “I bet we should make this *faster*” before cutting out the additional numerical dimension in Phase 2.

Several participants engaged with the concept of narrative and flow. P8 articulated this as a question about the goal of the visualization: “sometimes I want a big picture, not precision. I may want to drill down a little...” and observed that “we think too much in terms of components... sometimes accuracy isn’t the actual goal, it’s getting a general sense of something.” P4, upon discovering that *Skeleton* supports text-based input, asked, “How do you make that good, though? Like chatGPT, or do people want to, like, interact with the chart [elements]?”

During the interview, several participants explicitly requested guidance. P1, P2, P3, and P6 wanted to see examples of well-designed navigation experiences. P1, P3, P4, P5, P6, and P7 wanted embedded guidelines within the tool. P2 and P4 actually used web search to look for “chart navigation for accessibility guidelines” (P2) and “accessible viz screen reader design” (P4). P3 was interested in automation and heuristics that could suggest reasonable defaults.

These requests are consistent with the pattern (R2): practitioners who could see the design space wanted orientation within it.

6.7.3 Iteration Was Substantive, Self-directed, and Concentrated on Semantics

Every participant iterated on their navigation designs, and this iteration was neither perfunctory nor prompted by the research team (R4). Participants revisited decisions, revised configurations, and in some cases restructured their entire approach after encountering their design in a new stage of the tool.

The most sustained iteration concerned labels. Every participant spent the largest share of their authoring time in the label template editor (Figure 6.4), editing the text that a screen reader would announce at each node. This editing was granular: participants wrote full sentences, rearranged the order of data tokens, debated whether to include field keys alongside values or values alone, experimented with how to name groups and individual elements, and considered the length and density of the resulting announcements. At the division and dimension levels, some participants added multiple aggregate statistics (count, sum, average, range, trend) and then returned to trim them. P2, for instance, edited data point labels, left them, and returned to revise them two additional times: “This label is way too complicated, I think.”

A second, distinct pattern of iteration emerged around testing. The editor displays all navigation nodes simultaneously, showing the full structure at a glance. The testing stage (Section 6.5.3), by contrast, shows only the currently focused node, highlighting it in the canvas one at a time as the practitioner navigates. This difference consistently prompted participants to revise. Several returned to the editor after testing to adjust group node outlines, because outline strategies that looked distinct when displayed simultaneously (such as bounding rectangles vs. convex hulls) became harder to differentiate when encountered one at a time. Others adjusted their dimension configurations: P3 and P5 restructured their dimensions after testing, and P4, P6, and P8 experimented with different key bindings and navigation rules. P2 used the testing stage specifically to identify labels that needed revision at the dimension and division levels.

The most extensive iteration came from P1, who returned to the preparation wizard after reaching the testing stage and re-took the entire wizard from scratch. Seeing the full structure in testing motivated them to re-evaluate their earliest decisions. They reduced their navigation from three dimensions to one, producing a navigation experience they described as deliberately minimal. In the reflective interview, P1 explained that they had been thinking about trimming excess, joking that they were “working on a data-to-word ratio.”

The scaffolding tool shaped the pace and character of these iterations. Participants who used the scaffold (Figure 6.3) generally required only minor manual adjustment of node positions, spending one to two minutes refining placements in-aggregate before moving on. Iteration on spatial layout was primarily about the appearance of group-level outlines rather than individual node positions: because the scaffold computed leaf positions from the chart’s encoding, the main remaining spatial decision was how division and dimension nodes should visually indicate their grouping.

6.7.4 Seeing Navigation Prompted Practitioners to Reconsider the Architecture of Their Own Charts

An unexpected finding was that working with *Skeleton* prompted several participants to question the design of the visualization they had brought, not just the navigation structure they were building over it (R4). Five participants (P4, P5, P6, P7, P8) expressed a desire to simplify their own charts after seeing what the corresponding navigation structure required. Three (P1, P5, P6) considered whether a different chart type might serve the same communicative goal with a simpler navigational architecture. And in 2 cases (P1, P6), participants questioned whether a chart was the right medium at all.

P5, who brought a scatter plot with over two thousand data points, initially tried to build a navigation structure over the full dataset, recognized that the result was unmanageable, and then asked: “do I even need visualization?” They went on to wonder whether the information could be communicated as “a few sentences or like, some data [users] can prompt” (referring to using a large-language model). P6, who brought a bespoke flower visualization in which each petal encoded a different variable, initially tried to express the chart’s full complexity in navigation (grouping by flower and then navigating by petal) but then reversed course: “okay, what if we actually just treat this like a bar chart?” Using the scaffold tool, they produced a simple list-style navigation that worked well for their data, and reflected: “my visualization has a lot, but actually, this could be pretty easy to navigate, I think.” P6 also wondered whether there was a non-chart way to “tell their story,” and in further discussion described something closer to a scrollytelling article or guided walkthrough than a single interactive chart.

P8’s case was the most striking. They brought a voronoi pie chart (Figure 6.6) used to communicate to students that 26 assignments make up 45% of their grade, the largest slice of the pie. Working through *Skeleton*, P8 first considered making all 26 voronoi cells navigable, then considered making only the three main slices of the pie navigable, and then questioned whether navigation was needed at all. The “point of this,” as they described it, was to communicate a single ratio; students did not need to traverse individual cells. P8 concluded that well-written alternative text was sufficient for the screen reader experience and chose not to build any structure. They also reflected on the design of the visualization itself, concluding that the voronoi treatment served a visual purpose (it is visually striking) that was separable from the informational purpose (communicating a grade breakdown). They kept the visual design and simplified the non-visual experience accordingly.

6.7.5 Experiencing Keyboard Navigation Surfaced a Broader Range of Users and Input Technologies

The testing stage (Section 6.5.3) provided what was, for most participants, their first experience of keyboard navigation through a data visualization. Only three participants (P1, P2, P8) reported prior experience using a screen reader, and only two (P1, P8) had previously navigated a visualization using keyboard input alone. Yet during the study, every participant navigated using a keyboard.

Several participants responded to this experience with genuine engagement. P4 reacted with enthusiasm: “Woah, this is incredible. Wait, what other visualizations can do this?” and later,

Editor

Draw nodes and edges to define the navigation graph.



Click and drag to pan the canvas — Escape to exit

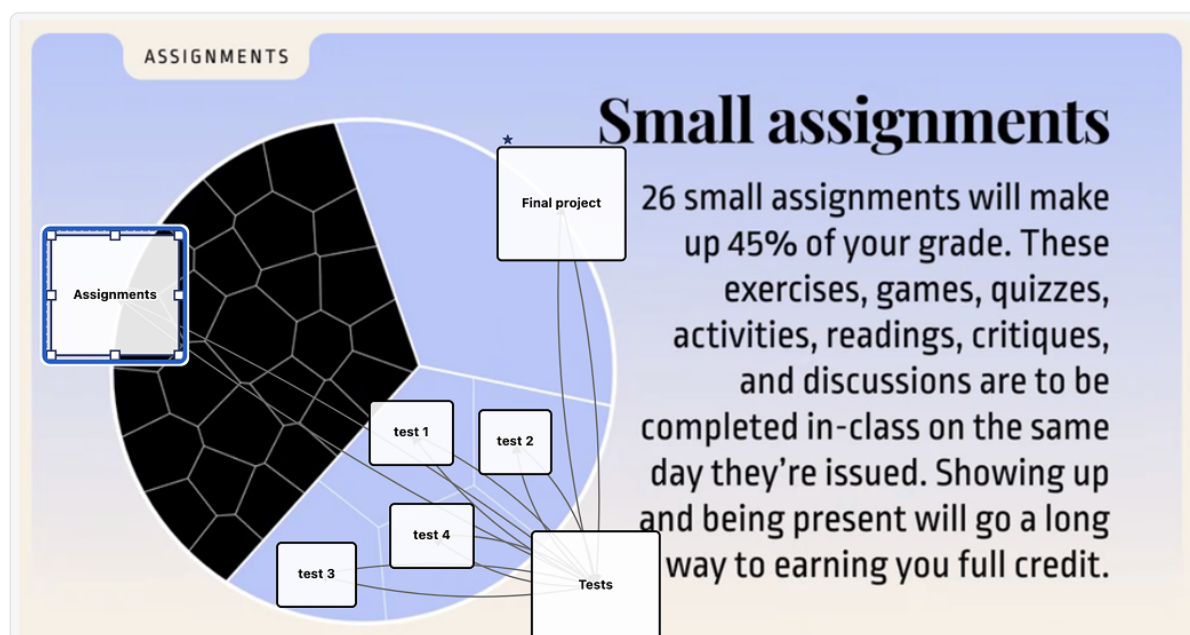


Figure 6.6: Re-creation of P8’s moment of realization, placing nodes manually: not every element in their voronoi pie chart *should* be navigable.

“Maybe you could make a visualization game, where you can move around the data?” P7 said, “I love this. I want to make charts with this.” P5, who had not previously encountered keyboard-navigable charts, said, “I didn’t know you could do this.”

The experience also expanded participants’ sense of who these structures serve (R2, R4). P8 adjusted a node’s spatial dimensions and said, “Let’s make the hitbox as big as possible here... for my neighbor with Parkinson’s to click these,” treating the navigation structure as relevant to motor accessibility, not only screen reader access. P3, observing the visual focus indicators that appeared during scaffold-based placement, remarked that “this is for more people than [someone] blind,” recognizing that sighted keyboard users and users with partial vision also rely on visible focus states. During the interview, P4 asked sustained questions about what kinds of technologies different people with disabilities use, and P4 and P7 both asked why focus indication was designed to be visible rather than invisible.

P4’s curiosity extended to alternative input modalities. Upon learning that *Skeleton* supports text-based navigation and (due to leveraging Data Navigator) also supports a wide array of other input modalities, they asked, “Can you talk at it, too? Is that what some people do?” and followed up with, “How cool is that? How do you make that good, though?” P8 raised a concern about discoverability in the current interaction model: “I’ve never used j or w to drill out, always shift arrow or option arrow...” and worried that a user might not know how to exit a nested level. These observations reflect a broad mental model of the user, from an abstract “screen reader

user” to a person with multiple capabilities, preferences, and interaction patterns (R2).

6.8 Discussion

6.8.1 Visibility as a Precondition for Iteration

The central finding of this work is not that *Skeleton* taught sighted practitioners how to design accessible navigation, but that it gave them something to react to. When navigation structure was invisible, our co-designers would only seek to verify whether navigation followed our design documentation. Once visible, questions shifted to whether it was good, why, how it could be improved, and what else it could do. Visibility encouraged iteration, and iteration is enabled continuous design improvements.

This mechanism is consistent with Schön’s account of reflective practice [171]: designers iterate by externalizing a representation, perceiving its properties, and responding to what they see. The representation talks back, and the designer adjusts. But this loop requires a representation. In our co-design work, we assumed that the gap was hand-off between design and development, and the slow process of manual verification. Instead, the gap is that development was not participating in, and encouraging, further design reflection. In a developer’s code-only workflow, navigation structure has no externalization that supports this kind of perceptual engagement. A practitioner can read the code that specifies a navigation graph, but they cannot see the graph, cannot perceive its topology at a glance, cannot notice that a label is redundant or that a hierarchy is too deep by looking at it. The reflective loop is disrupted and occluded at the first step.

Skeleton encourages this loop by rendering navigation structure in a form that supports the same kind of perceptual judgment that visualization practitioners already apply to every other aspect of their work. The results show what happened when this loop was available: every participant iterated, substantively and self-directedly (Section 6.7.3). They revised labels repeatedly, restructured dimensions after testing, adjusted group outlines when sequential traversal revealed problems that simultaneous display had hidden. P1 restarted the entire preparation wizard after seeing their structure in testing mode. These are not the behaviors of practitioners following a specification; they are the behaviors of practitioners negotiating with a design material.

The co-design work (Section 6.4) provides complementary evidence: in each collaboration, sighted practitioners reached for visual representations when reasoning about navigation, through node-link diagrams in Figma, schema sketches in Miro, and annotated wireframes on paper. They were already thinking visually about non-visual structure; the development tooling simply had not caught up. The practical implication is broad: if sighted authors depend on visibility, then any authoring workflow that keeps navigation structure invisible limits design iteration. Making structure visible does not guarantee good design, but it is a precondition for the kind of sustained, judgment-driven refinement that good design requires.

6.8.2 From Compliance to Design

A consistent pattern across the accessibility literature is that practitioners frame accessibility as a compliance problem: a set of requirements to satisfy, a checklist to complete, a legal or insti-

tutional obligation to meet [42, 95, 178]. The framing matters because compliance and design orient practitioners toward fundamentally different activities. Compliance asks: “does this pass?” Design asks: “is this good?” Our results suggest that *Skeleton* produced a partial shift from the first orientation to the second. Participants’ initial instincts clustered around compliance-oriented responses: provide alternative text, follow guidelines, ask an expert (Section 6.7.2). But alongside that desire for guidance, they began doing something compliance framing does not typically produce: they encountered complexity and then *iterated* (Section 6.7.3). This sustained, self-directed refinement is the behavioral signature of design, not compliance. As P4 put it: “I’d love to have someone blind actually just with me while I make this, but I also understand that I should learn what makes a good experience too.”

The shift extended beyond the non-visual experience itself. As reported in Section 6.7.4, five participants reconsidered the design of the visualization they had brought, not just the navigation structure. Making the accessibility consequences of visual design choices visible prompted practitioners to question whether a different chart type, a simpler encoding, or a non-chart medium might better serve their communicative goals. This interrelation between non-visual and visual design suggests that the widespread treatment of accessibility as a compliance activity may be partly a consequence of tooling that offers no legible, manipulable design surface. Auditing frameworks are valuable, but they are *evaluative* tools, not *authoring* tools. The field needs both.

6.8.3 Bespoke Visualizations as an Unaddressed Accessibility Research Problem

Diagrams, infographics, and data-driven illustrations are often one-off, custom designs with bespoke symbols, layouts, and visual languages. These representations are increasingly common in journalism, scientific communication, personal projects, art, and public-facing data work, and they represent the cases where accessibility-focused tools are needed most and available least. *Skeleton*’s image-based workflow (upload any 2D image, place nodes manually) provides a starting point, but the study made clear that bespoke visualizations need more than node placement. They need support for reasoning about what navigational structure (if any) is appropriate when no template applies, a research problem that remains largely unexplored. It is worth being precise about where generality holds and where it stops. Because the *Dimensions API* derives structure from whatever fields a dataset carries rather than from a fixed schema, the data side of the approach already extends to datasets not known in advance; what does not yet generalize is the prior, harder question of which dimensions, if any, a bespoke visual should expose when its visual form was never organized around a tabular structure to begin with. Generalizing the abstraction is therefore not primarily a matter of supporting more schemas but of supporting the design judgment that precedes choosing a schema at all.

6.8.4 What Visualization Owes Accessibility

Our approach, to make visual non-visual experiences, should not be limited to data visualization’s own accessibility challenges. Navigation structure is a foundational component of accessible experience across domains: PDF and document reading order, web page structures, and

software application layouts. In each of these areas, sighted practitioners author non-visual experiences without visual feedback, and in each, the same gap between design intent and verifiable outcome constrains quality. Testing is slow, error prone, and requires expertise in assistive technology use. Visual tooling for authoring, inspecting, and debugging non-visual structure ([R3](#)) is a tractable and high-value problem across application domains.

There is also a deeper question about what visibility can accomplish in principle. Work on multi-modal authoring environments has argued for de-centering visual representation and treating modalities as equal partners in the design process [227], an important ethical commitment. *Skeleton* does not do this: it re-centers visual representation as the medium through which sighted practitioners engage with non-visual structure. We believe this is justified pragmatically, because sighted authors need to articulate navigation design in their own perceptual language before they can reason about it at all, and this paper provides evidence that they do. But articulating a design in one’s own language is not the same as understanding how it will be experienced in someone else’s. The structures participants built during our study were never evaluated by blind users, and visibility alone cannot substitute for that evaluation. The risk we want to name is that making non-visual structure visible to sighted practitioners could be mistaken for making it *understood*, when in fact it makes it *designable*, a real but bounded gain. The fuller design process requires collaboration with disabled users, not as an occasional supplement but as a regular practice. *Skeleton* can make that collaboration more productive by giving both parties a shared representation or space of translation between representations, but it cannot replace it.

What visualization owes accessibility, then, is not simply better output but authoring tools that better stimulate reasoning, both individually and collaborative, about design.

6.9 Limitations and Future Work

Skeleton surfaces design questions but does not answer them: several participants asked what constitutes good navigation, and the tool had nothing authoritative to offer. As ?? argues, our study evaluated whether making structure visible stimulated design consideration, not whether the resulting designs were good for the people who will use them, which is a distinct question that remains open. CD2’s expert screen reader evaluation of *Skeleton*’s navigation output ([Section 6.5.3](#)) provided practice-based validation for several common chart types and surfaced concrete bugs, but this evaluation was neither comprehensive nor controlled: many configurations remain untested, and expert review is not a substitute for evaluation with a broader population of end users.

Our approach using *Skeleton* as a design probe only with sighted participants is a limitation and, we believe, the right sequencing: *Skeleton* improves the intentionality and iterability of what sighted practitioners produce, which is a necessary precondition for a subsequent evaluation or future collaborative design work between sighted and blind authors. Further research and evaluation should close this loop, ideally to engage how mixed-ability teams co-design multi-modal data experiences.

A further boundary concerns the kind of visualization *Skeleton* targets. Highly interactive or dynamic visualizations, where selecting an element reloads the data, where marks animate, or where the navigable structure itself changes in response to user interaction, are not yet addressed.

We see this as an important future direction rather than a fundamental obstacle: the same argument for making structure visible applies with more force when there are several structures to coordinate, and an inspectable representation may be precisely what makes a dynamic navigation experience tractable to author.

Skeleton is also a prototype with substantial work remaining. Not within the scope of the paper, but our participants provided ample feedback on the functionality of the prototype itself. The most urgent gap is export functionality: practitioners can design and inspect navigation structures in the tool but cannot yet produce deployable output.

6.10 Conclusion

Accessible navigation structure has long occupied an awkward position in visualization practice: known to matter, difficult to design, and invisible to the people responsible for building it. Without a way to see what they were making, sighted practitioners could not catch errors, could not iterate, and could not develop the kind of considered judgment that good design requires; accessibility remained downstream of every other decision not because of any single failure, but because the authoring conditions did not support anything else. *Skeleton* demonstrates that those conditions can be changed. Making navigation structure visible and manipulable (as an interactive graph rendered over the spatial layout of a real visualization, with live label previews and testable traversal) shifted how practitioners engaged with and reasoned about accessible design, prompting them to ask whether the design of their structure was *good* rather than merely whether it passed.

The implication generalizes beyond our tool: wherever a non-visual structure or user experience is authored without a visible representation, building one the author can react to and manipulate invites them to design. What the paper leaves open is more than what it closes. We used *Skeleton* as a design probe with sighted practitioners; we did not evaluate the structures they produced with the end users those structures are meant to serve, and the broader disciplinary conversation about what visualization research owes accessibility, and what methods might transfer between them, has more questions than answers. We offer *Skeleton* not as a solution to these problems but as evidence that engaging them directly, with the full weight of visualization's methodological tradition, is both possible and fruitful.

6.11 Postface

While this chapter did not take the liberty of making a strong position statement, it is clear to us now that when viewed through the social model of disability (where disability is produced between human interaction with systems and environments) practitioners who are not traditionally considered “people with disabilities” in the medical sense may still face barriers to access when using an interface or system. *Systems can disable any user.*

As another note worth making, we also attempted to “vibe-code” the prototype for this work as well, and encountered (ironically) many barriers while developing, both that we personally faced and that were unfortunately also produced for end users. Remediating our prototype has taken a non-trivial amount of manual and agentic effort.

Large-language models are not necessarily skilled enough out of the box, at the time of this writing, to make highly complex research prototypes that focus on cutting-edge accessibility functionality. The overall work spent vibe-coding now vastly outpaces whatever work we would have invested if we hadn’t vibe-coded at all, because we will need to virtually rebuild this entire prototype from scratch before deploying it live for real-world use. It is, in this sense, a throwaway research prototype in its current condition. But at the same time, we did learn that vibe-coding in contexts with significant system complexity also takes from its user an opportunity to solve the engineering problems themselves, which is a challenge that we enjoy and believe helps us to better understand our own work.

Perhaps in the future, we will submit an experience report or write a workshop paper on the *self* and *social* barriers that vibe-coding introduced into our accessibility-centered prototyping work.

Part VI

Conclusion

Chapter 9

Findings

The five projects in this thesis were presented as five chapters, each with its own results and its own conclusion. Read separately, they contribute a heuristic resource for evaluation, a structural toolkit for navigation, an authoring environment for non-visual design, an interaction technique and device for blind analysis, and a system and study of end-user personalization. This chapter states what they show *together*. Chapter 2 closed on a lopsided field: a breadth of empirical work and exemplary prototypes on one side, and a near absence of practitioner tools in production environments and practitioner action on the other, with the hardest remaining challenges concentrated in navigation, interaction, and personalization. The findings below are what this thesis learned by working inside that gap.

Each finding is a synthesis across chapters, and each is traceable to results reported in the body of this thesis. We have written them for two audiences at once: practitioners who build, evaluate, and maintain data experiences, and researchers who study how that work happens. Where a promising idea outran the evidence, it is not here; it is in the next chapter, with the rest of the discussion and future work.

9.1 Access Barriers Are Produced for Practitioners, Both With and Without Disabilities

The social model holds that disability is not located in a body but produced in the interaction between a person and a human-made system [119]. This thesis began by applying that lens to end users: visualizations, like curbs, produce barriers. The cross-cutting finding of this thesis is that the same lens applies, with the same force, to the practitioners doing accessibility work, and that the materials of that work are among the systems producing the barriers.

Every project surfaced a version of this. In *Chartability*, the guideline landscape itself was a barrier: standards and research were scattered, often high-level, and difficult to apply to complex interactive data interfaces, and practitioners had little way to judge the quality of what they gathered [95, 178]. Guidelines written to remove barriers for end users limited what practitioners could reasonably understand and act on, and an expert visualization practitioner in our study admitted he did not know where to start. In *Skeleton*'s co-design foundations, *Data Navigator* made better end-user experiences possible, but because its structures were entirely non-visual and code-bound, it produced severe iteration barriers for sighted practitioners: they could not perceive what they were building, and prototypes were misinterpreted or abandoned. And in *Cross-perception*, the screen reader, the most important enabling technology for blind computer use, also limited what blind practitioners could reasonably do analytically: its serial, discrete traversal kept participants in an access-oriented mode that structurally separated interaction from information, with measurable costs in time, completion, and the number of questions they could ask of their data [176].

The operationalized version of this finding: when accessibility work stalls, look for barriers

in the practitioner’s tools and materials before attributing the failure to motivation or knowledge. None of the practitioners in these studies lacked care. What they lacked, in each case, was a working relationship with materials that had been shaped without them in mind, and this is true whether the practitioner is sighted, blind, disabled, or not. The same observational stance the field has learned to take toward end users belongs in the study of the workers, too.

9.2 Tools Can Reduce the Work Required to Make Things Accessible

Chapter 2 argued that guidance cannot substitute for means. The constructive half of that argument is that supplying means measurably changes what practitioners can do, and three chapters provide the evidence.

Chartability reduced the perceived difficulty of evaluation and increased the confidence of practitioners new to accessibility, whose audit results were reasonably comparable to the authors’ own; beyond the study, it has been applied by policy and governmental organizations and more than 100 companies, across toolchains from Tableau and Excel to D3 and Figma. *Data Navigator* gave previously inaccessible rendering formats a navigable structure: a static raster image gained full multi-input navigation with no automatically detectable conformance failures, and a deterministic ingestion pipeline recreated, and then improved on, an existing toolkit’s navigation scheme for rendering modes that previously had none. *Skeleton*’s scaffolding collapsed the spatial authoring of a navigation structure to one or two minutes of refinement, and within single sessions every participant iterated substantively on designs that, in a code-only workflow, our co-design collaborators had struggled to iterate on at all.

None of this is evidence that tools solve access; the boundaries of that claim get their own finding below. It is evidence of something narrower and more useful: the labor of access work is sensitive to tooling, and reductions in that labor are achievable, measurable, and repeatable across very different kinds of tools, from a workbook of heuristics to a rendering-independent toolkit to an authoring interface.

9.6 The Hardest Barriers Required Intervention at the Substrate

Chapter 2 characterized navigation, interaction, and personalization as a missing structure, a missing technique, and a missing infrastructure. Having now worked in all three domains, we can state what they share: in each, the barrier sat one level below where practitioners are ordinarily able to act, in the substrate their work is built on, and in each, outcomes changed when the intervention went to that level rather than adding features above it. This finding is held heavily in tension with existing research that tends towards higher levels of software and interaction abstraction, not lower. A massive wave of new research has emerged that essentially asks, “what new layers or wrappers can we create around existing interfaces and user experience problems, using large language models and agents?” While our thesis doesn’t engage the effectiveness or

intellectual value of those projects, instead we hold this thesis in fundamental contrast to their approach: that some of our most difficult gaps are best engaged below our present levels of software abstraction, not above.

The evidence is consistent across the three parts of this thesis. No amount of feature work on top of canvas-rendered pixels can recover semantics that the rendering substrate never carried, and forcing complex, non-hierarchical data relationships into HTML’s hierarchical substrate limited what was possible to build. This is precisely what a large and passionate resistance to “overlay” technologies engages within both the practical and academic accessibility communities [39, 66, 71, 114]. There may simply be a limit to how much good data, if at all, can be generated from bad or missing data [11, 146].

Data Navigator changed outcomes by supplying a structural substrate of its own, a graph independent of any renderer. This requires underlying data that is supplied with sufficient quality, what Doug Schepers calls an “underlay” rather than an overlay [166].

No software feature on existing consumer hardware approximates persistent physical state and simultaneous perception of input and output, the properties analytical interaction depends on [108]; the *cross-feeler* changed the hardware substrate, and the character of blind analysis changed with it, from accessing data toward interrogating it, at nearly three times the query rate.

And per-chart personalization features fail the effort-to-outcome test that end users themselves applied; what every participant and practitioner in the *Software* study named instead were system-class needs [50]: persistence, profile sharing, interoperability, and standards beneath any individual chart.

This is why the substrate matters as a space of intervention: in all three domains, the ceiling on what a motivated, well-informed practitioner could make accessible was set by material beneath their reach, so guidance, effort, and even exemplary prototypes hit that ceiling without moving it. The work in this thesis moved outcomes exactly where it changed the material itself, and the same evidence shows the stages of this work leaning on one another: structure without authoring support went unused, authoring raised evaluative questions practitioners could not answer alone, and personalization without accessible defaults and persistent infrastructure shifts burden onto the people it is meant to serve. Researchers and toolmakers deciding where to spend effort in these three domains have, in this thesis, a consistent answer about where the leverage was found.

Chapter 10

Discussion & Future Work

10.2 “Applied” accessibility work and *low research*

I came into an academic research environment already as an engineer. I had questions to big problems, especially about how tools might shape human behavior and how we can use that towards ethically good outcomes. But *Chartability* was a project I made first for myself, to engage the problems I, personally, was facing.

And *Data Navigator* was motivated by my existing experience making visualizations navigable. I knew we had to build better tools, because continually trying to make a hierarchical substrate like HTML work for complex, non-hierarchical relationships simply limited what was possible and easy to do.

These two projects then led to many opportunities to test them in applied contexts. Our CZI grant was motivated towards the application of both, Adobe was interested in *Data Navigator*, and the University of Wisconsin’s GIS folks wanted to figure out how to make a complex map navigable, *Data Navigator* or not. Yet, as a researcher I wasn’t incentivized to pursue these projects. In fact, they all proved to have no immediately apparent novelty. It was a risk to spend my time in these spaces.

Applied work is structurally and socially devalued in academic research environments, seen often as a lower form of research and design compared to more “pure” forms of knowledge production. (We conjecture that in part this is due to the scope of knowledge that practitioners produce: they work to take broad knowledge and apply it to specific, small problem spaces. Foundational research tends to assume that broad knowledge does not then bubble up from this work, that it is a one-way flow.)

In my first year as a PhD student I was given the advice by a senior researcher not to hold my experience as a practitioner too highly because, “practitioners often don’t know what they are doing.” But I did, of course, to some degree. Yet, this attitude is pervasive in some spaces and is a problem because it limits what kinds of knowledge we care to attend to.

Initially, I felt that with *Chartability* I had lucked out. But after four more collaborations which resulted in funding and two research publications, we can now argue that this method has been reliable. In *Softerware* and *Skeleton*, pursuing practical problems in applied environments *did* serve as a fruitful source of generalizable knowledge and advancements in the state of the art. But these projects, in addition to *Chartability*, were all motivated by immediate, non-novel problems: people knew their charts were inaccessible and wanted some design guidance or engineering solutions.

So perhaps “low research” works.

Yet, some research communities, especially *ASSETS*, have yet to reckon with “applied” work like this. If you trace the lineage of *action research* from Hayes [72] (which is how we frame our applied work in *Skeleton*), it leads into fantastic areas of participatory and community-based work, often with or by marginalized people and people with disabilities [7, 83, 110, 115, 187, 190, 191, 199, 215]. We would even argue that this lineage of work, which prioritizes and centers

the lived experience of a person, is also why we have seen so many successful auto-biographical and auto-innovation styles of research as well recently [9, 57, 59, 80, 173, 213, 214].

And yet, action research largely does not touch on how practitioners (particularly those without disabilities) do applied work in accessibility [95, 178]. We believe that significant gaps exist in the space between traditional “high” knowledge production and applied, “lowly” spaces. Researchers tend to pursue novelty and the lifecycle of research production is highly risk averse. And yet, relatively little work has been done within the accessibility research community to intellectually address the actual social and cultural risks involved in producing decades of useless, lost, and forgotten prototypes and design guidelines [84, 89]. We would conjecture that research-producing academics are overly concerned with academic readership relative to the myriad of ways that their knowledge production actually is used by the rest of the world [129]. We would challenge *ASSETS* and the AT/accessibility communities of *CHI* in particular to take seriously the lack of applied (industry, government, and non-academic) work it produces as a community.

10.3 Who is responsible for repair?

Lastly, we want to revisit one of this thesis’s opening points, where we argue that the *tool-makers* are first responsible for repair. This is true. However, the most pressing issue we have faced in recent years is mostly unmentioned across these research projects: tool-makers might be responsible, but this is because they are the only ones who have the *power* to make things accessible. Does this always need to be the case? Can we imagine an artifact’s authority over the user’s ability to access being designed towards self-subversion [62] or de-centralized agency [22, 97, 130], instead? What might that look like, concretely?

In *Softerware*, we begin to engage this larger problem in terms of an idealized state where a user can repair or re-design their own experiences. But to me, this self-repair is like laying down train tracks for yourself as you move a locomotive, but then lifting up your own tracks behind you as you go. You’re the only one helping yourself. This is not ideal, for you or others.

What we need are broad, lasting, infrastructural changes. On the web, this problem becomes quite difficult to solve. A personal computer or device? Again, someone can auto-design their interfaces into a better state. But back when we started *Chartability*, the WebAIM Million’s report showed more than 95% of the top one million website home pages contain at least one critical accessibility error. And now, more than six years later, that proportion is unchanged [207].

Some had imagined that generative AI would solve the massive infrastructural repair problems we face. But unfortunately, the latest WebAIM Million report shows that since 2020, ARIA usage has increased and correlates to more errors, while use of `tabindex` on a page has increased nearly 300% and also correlates to more errors on a page. If anything, during the age of generative AI, we have seen existing bad patterns worsen in prevalence and complexity.

We firmly believe that a tools-based approach is not enough on its own. Tool-making cannot be the *only* intervention on inaccessibility. Tools and tool-making, as our thesis argues, have a powerful role to play. But we simply can’t tool our way out of failed infrastructure and inadequate policy when someone else *owns* the tools and tool-making. Visiting a website is like going into someone else’s home: arranged according to their effort, tastes, and so on. If you can’t access their home, you essentially need to request that they let you in personally. Website repair always

falls to the owner and maintainer of a website, and they largely don't take any meaningful action.

Sidewalks outside of homes are a good parallel to this problem. Sidewalk accessibility is a massive infrastructural problem [158], and yet localities treat sidewalk maintenance in different ways: some, like where I, Frank, presently live in the south hills of Pittsburgh, put the onus on the homeowner whose house and property the sidewalk touches. In other places, sidewalks are considered a public path, similar to a roadway, and are maintained through public tax and resource management. To no surprise, privately-maintained, public-access sidewalks are worse for people in pretty much every way than publicly-maintained ones [219]. This is because private homeowners don't care about sidewalk maintenance unless the city manages to fine them or they get sued.

And the web is a collection of private spaces that you visit privately. There is no truly shared, universally democratic, public space on the web. Centralization is partly to blame: sharing space while scaling leads to consolidation.

So our future work will continue to wrestle with the same tensions of scale, repair, and anti-consolidation of power, motivated by the same WebAIM Million report. But now we look to questions of *democratic* and *radical* access to accessibility repair. The barriers we hope to tackle in the future are political and infrastructural. Perhaps tool-making will participate in this work, but it seems clear now from our work that the upstream technical problems and socio-political conditions that tools inherit, will likely not be addressed by tools alone.

What does “democratic” and “radical” infrastructure work look like? It will probably be an extension of *Softerware*, to some degree. We imagine future research that explores public-first spaces, ones where access is socially negotiated and repair belongs to all of us. Is this an autonomous space, like an autonomous zone [8] separate from the web? Above it, looking down into it, like shared annotation tools but capable of sharing the manipulation of websites [157]? A space with ambient co-repair, modeled after projects that bring people together [174] or that allow community “fixing” of misinformation [90]? Perhaps feminist thought on the ethics of care can help us [75, 113]? Or maybe it will be something else entirely; I'm not yet sure. But what made the web fantastic years ago is long gone; most of it has been hedged into corporate spaces that are controlled, maintained, and repaired by corporate power. And these entities are notoriously bad at repair. What we imagine in the future involves reclaiming a sense that the web is *ours*, belongs to *us*, and that ultimately *we* are responsible for making it accessible.

References

- [1] Frances Aldrich. Talk to the Hand: An Agenda for Further Research on Tactile Graphics. In Gem Stapleton, John Howse, and John Lee, editors, *Diagrammatic Representation and Inference*, Lecture Notes in Computer Science, page 344–346, Berlin, Heidelberg, 2008. Springer, Springer Berlin Heidelberg. ISBN 9783540877301. doi: 10.1007/978-3-540-87730-1_31. URL https://doi.org/10.1007/978-3-540-87730-1_31. 4.3.1
- [2] Obead Alhadreti and Pam Mayhew. Rethinking thinking aloud: A comparison of three think-aloud protocols. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI ’18, page 1–12. ACM, April 2018. doi: 10.1145/3173574.3173618. URL <https://doi.org/10.1145/3173574.3173618>. 6.6.2
- [3] Ofunne Amaka and Amber Thomas. The Naked Truth. <https://pudding.cool/2021/03/foundation-names>, 2021. Accessed: 2021-08-20. 4.6.3
- [4] Catherine M. Baker, Lauren R. Milne, Ryan Drapeau, Jeffrey Scofield, Cynthia L. Bennett, and Richard E. Ladner. Tactile graphics with a voice. *ACM Transactions on Accessible Computing*, 8(1):1–22, January 2016. ISSN 1936-7236. doi: 10.1145/2854005. URL <https://doi.org/10.1145/2854005>. 4.3.1
- [5] Abhijit Balaji, Thuvaarakkesh Ramanathan, and Venkateshwarlu Sonathi. Chart-Text: A Fully Automated Chart Image Descriptor. *arXiv Preprint*, December 2018. doi: 10.48550/ARXIV.1812.10636. Accessed: 2021-09-06. 4.3.1
- [6] BANA. Guidelines and Standards for Tactile Graphics. Braille standard, Braille Authority of North America, 2010. Accessed: 2021-09-03. 4.3.1, 4.3.3
- [7] Cynthia L. Bennett, Erin Brady, and Stacy M. Branham. Interdependence as a Frame for Assistive Technology Research and Design. In *Proceedings of the 20th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’18, page 161–173, New York, NY, USA, October 2018. Association for Computing Machinery, ACM. doi: 10.1145/3234695.3236348. URL <https://doi.org/10.1145/3234695.3236348>. Accessed: 2021-09-03. 1.1, 4.4.3, 10.2
- [8] Hakim Bey. *TAZ: The temporary autonomous zone, ontological anarchy, poetic terrorism*. Autonomedia, 1991. 10.3
- [9] Brandon Biggs, Christopher Toth, Tony Stockman, James M. Coughlan, and Bruce N. Walker. Evaluation of a non-visual auditory choropleth and travel map viewer. In *Proceedings of the 27th International Conference on Auditory Display (ICAD 2022)*, pages 82–90.

International Community for Auditory Display, June 2022. doi: 10.21785/icad2022.027. URL <https://doi.org/10.21785/icad2022.027>. PMID: PMC10010675. 6.4.1, 10.2

- [10] Jeffrey P. Bigham, Chandrika Jayant, Hanjie Ji, Greg Little, Andrew Miller, Robert C. Miller, Robin Miller, Aubrey Tatarowicz, Brandyn White, Samuel White, and Tom Yeh. Vizwiz: nearly real-time answers to visual questions. In *Proceedings of the 23rd annual ACM symposium on User interface software and technology*, UIST '10, page 333–342, New York, NY, USA, October 2010. ACM. ISBN 978-1-4503-0271-5. doi: 10.1145/1866029.1866080. URL <https://doi.org/10.1145/1866029.1866080>. Accessed: 2021-09-06. 4.3.3
- [11] Abeba Birhane, Vishnu Boddeti, Sanghyun Han, Sasha Luccioni, and Vinay Prabhu. Into the laion’s den: Investigating hate in multimodal datasets. In *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, page 21268–21284. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023. doi: 10.52202/075280-0930. URL <http://dx.doi.org/10.52202/075280-0930>. 9.6
- [12] Matt Blanco, Jonathan Zong, and Arvind Satyanarayan. Olli: An Extensible Visualization Library for Screen Reader Accessibility. In *IEEE VIS Posters*, 2022. URL <https://vis.csail.mit.edu/pubs/olli>. Accessed: 2026-03-30, <https://vis.csail.mit.edu/pubs/olli>. 6.3.3, 6.4.4.2
- [13] Jens Bornschein, Denise Prescher, and Gerhard Weber. Collaborative Creation of Digital Tactile Graphics. In *Proceedings of the 17th International ACM SIGACCESS Conference on Computers & Accessibility - ASSETS '15*, ASSETS '15, page 117–126, New York, NY, USA, October 2015. Association for Computing Machinery, ACM Press. doi: 10.1145/2700648.2809869. URL <https://doi.org/10.1145/2700648.2809869>. 4.3.1, 6.3.3
- [14] M. Bostock, V. Ogievetsky, and J. Heer. D³ data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, December 2011. ISSN 1077-2626. doi: 10.1109/tvcg.2011.185. URL <https://doi.org/10.1109/tvcg.2011.185>. 1.2, 6.3.1
- [15] Denis Boudreau. Unlocking Accessibility for UX/UI Designers. <https://www.24a11y.com/2018/unlocking-accessibility-for-ux-ui-designers/>, December 2018. Accessed: 2021-09-07. 4.3.4
- [16] Virginia Braun and Victoria Clarke. Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2):77–101, January 2006. ISSN 1478-0895. doi: 10.1191/1478088706qp063oa. URL <https://doi.org/10.1191/1478088706qp063oa>. 3, 6.6.3
- [17] S. Brewster. Visualization tools for blind people using multiple modalities. *Disability and Rehabilitation*, 24(11-12):613–621, January 2002. ISSN 1464-5165. doi: 10.1080/09638280110111388. URL <https://doi.org/10.1080/09638280110111388>. 4.3.1, 6.3.2
- [18] Craig Brown and Amy Hurst. Viztouch: automatically generated tactile visualizations

- of coordinate spaces. In *Proceedings of the Sixth International Conference on Tangible, Embedded and Embodied Interaction*, TEI '12, page 131–138, New York, NY, USA, February 2012. ACM. ISBN 978-1-4503-1174-8. doi: 10.1145/2148131.2148160. URL <https://doi.org/10.1145/2148131.2148160>. Accessed: 2021-09-03. 4.3.1
- [19] Jennifer L. Burke, Matthew S. Prewett, Ashley A. Gray, Liuquin Yang, Frederick R. B. Stilson, Michael D. Covert, Linda R. Elliot, and Elizabeth Redden. Comparing the effects of visual-auditory and visual-tactile feedback on user performance: a meta-analysis. In *Proceedings of the 8th international conference on Multimodal interfaces*, ICMI06, page 108–117. ACM, November 2006. doi: 10.1145/1180995.1181017. URL <https://doi.org/10.1145/1180995.1181017>. 1.1
- [20] Matthew Butler, Leona M Holloway, Samuel Reinders, Cagatay Goncu, and Kim Marriott. Technology developments in touch-based accessible graphics: A systematic review of research 2010-2020. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, page 1–15. ACM, May 2021. doi: 10.1145/3411764.3445207. URL <https://doi.org/10.1145/3411764.3445207>. 4.3.1, 6.3.2
- [21] Silvia Canelón and Liz Hare. Revealing Room for Improvement in Accessibility within a Social Media Data Visualization Learning Community. <https://silvia.rbind.io/talk/2021-05-04-data-viz-accessibility/>, 2021. Accessed: 2021-09-06. 4.3.2
- [22] Katherine Casey. Radical decentralization: Does community-driven development work? *Annual Review of Economics*, 10(1):139–163, August 2018. ISSN 1941-1391. doi: 10.1146/annurev-economics-080217-053339. URL <http://dx.doi.org/10.1146/annurev-economics-080217-053339>. 10.3
- [23] Alex Chaparro and Maria Chaparro. Applications of Color in Design for Color-Deficient Users. *Ergonomics in Design: The Quarterly of Human Factors Applications*, 25(1): 23–30, November 2016. ISSN 1064-8046. doi: 10.1177/1064804616635382. URL <https://doi.org/10.1177/1064804616635382>. Accessed: 2021-09-06. 4.3.1
- [24] Charles Chen, Ruiyi Zhang, Sungchul Kim, Scott Cohen, Tong Yu, Ryan Rossi, and Razvan Bunescu. Neural caption generation over figures. In *Adjunct Proceedings of the 2019 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2019 ACM International Symposium on Wearable Computers*, UbiComp/ISWC '19 Adjunct, pages 482–485, New York, NY, USA, September 2019. Association for Computing Machinery, ACM. doi: 10.1145/3341162.3345601. Accessed: 2021-09-06. 4.3.1
- [25] Charles Chen, Ruiyi Zhang, Eunye Koh, Sungchul Kim, Scott Cohen, and Ryan Rossi. Figure Captioning with Relation Maps for Reasoning. In *2020 IEEE Winter Conference on Applications of Computer Vision (WACV)*, page 1526–1534. IEEE, March 2020. doi: 10.1109/WACV45572.2020.9093592. URL <https://doi.org/10.1109/wacv45572.2020.9093592>. ISSN: 2642-9381. 4.3.1
- [26] Jinho Choi, Sanghun Jung, Deok Gun Park, Jaegul Choo, and Niklas Elmqvist. Visualizing for the Non-Visual: Enabling the Visually Impaired to Use Visualization. *Computer*

Graphics Forum, 38(3):249–260, 2019. doi: 10.1111/cgf.13686. Accessed: 2021-09-03. 4.3.1

- [27] Ngip Khean Chuan, Ashok Sivaji, and Wan Fatimah Wan Ahmad. Usability Heuristics for Heuristic Evaluation of Gestural Interaction in HCI. In Aaron Marcus, editor, *Design, User Experience, and Usability: Design Discourse*, Lecture Notes in Computer Science, page 138–148, Cham, 2015. Springer International Publishing, Springer International Publishing. ISBN 9783319208862. doi: 10.1007/978-3-319-20886-2_14. URL https://doi.org/10.1007/978-3-319-20886-2_14. 4.3.4
- [28] Pramod Chundury. Building multimodal chart accessibility for blind and low-vision individuals in the prompt-driven era. *XRDS: Crossroads, The ACM Magazine for Students*, 32(1):34–37, September 2025. ISSN 1528-4980. doi: 10.1145/3778055. URL <http://dx.doi.org/10.1145/3778055>. 4.11
- [29] Pramod Chundury, Biswaksen Patnaik, Yasmin Reyazuddin, Christine Tang, Jonathan Lazar, and Niklas Elmqvist. Towards understanding sensory substitution for accessible visualization: An interview study. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1084–1094, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114829. URL <https://doi.org/10.1109/tvcg.2021.3114829>. 4.3.1, 6.3.2
- [30] RStudio Community. Making Shiny apps accessible for all humans. <https://community.rstudio.com/t/making-shiny-apps-accessible-for-all-humans/8458>, May 2018. Accessed: 2021-09-06. 4.3.2
- [31] B. Craft and P. Cairns. Beyond guidelines: What can we learn from the visual information seeking mantra? In *Ninth International Conference on Information Visualisation (IV’05)*, page 110–118. IEEE, July 2005. doi: 10.1109/IV.2005.28. URL <https://doi.org/10.1109/iv.2005.28>. ISSN: 2375-0138. 4.3.4
- [32] Clare Cullen and Oussama Metatla. Co-designing inclusive multisensory story mapping with children with mixed visual abilities. In *Proceedings of the 18th ACM International Conference on Interaction Design and Children*, IDC ’19, pages 361–373. ACM, June 2019. doi: 10.1145/3311927.3323146. URL <https://doi.org/10.1145/3311927.3323146>. 4.3.1, 6.4
- [33] Lilian de Greef, Dominik Moritz, and Cynthia Bennett. Interdependent variables: Remotely designing tactile graphics for an accessible workflow. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’21, page 1–6. ACM, October 2021. doi: 10.1145/3441852.3476468. URL <https://doi.org/10.1145/3441852.3476468>. 6.4
- [34] Chris DeMartini. Coin flip game. <https://public.tableau.com/views/CoinFlipGame/CoinFlipGame>, 2019. Accessed: 2021-08-20. 4.6.3
- [35] Chris DeMartini. A Tableau Accessibility Journey - Part IV - Keyboard Accessibility. <https://www.datablick.com/blog/2021/8/10/a-tableau-accessibility-journey-part-iv-keyboard-accessibility>, 2021. Accessed: 2021-09-03. 4.3.2
- [36] Deque. Automated Testing Identifies 57 Percent of Digital Accessibility Issues. Technical report, Deque, March 2021. Accessed: 2021-09-06. 4.2

- [37] Deque. Axe-core, August 2021. Accessed: 2021-08-20. [4.7.5](#)
- [38] Johanna Drucker. *Graphesis*. metaLABprojects. Harvard University Press, London, England, July 2014. ISBN 9780674724938. [4.1.1](#)
- [39] Niklas Egger, Gottfried Zimmermann, and Christophe Strobbe. *Overlay Tools as a Support for Accessible Websites – Possibilities and Limitations*, page 6–17. Springer International Publishing, 2022. ISBN 9783031086458. doi: 10.1007/978-3-031-08645-8_2. URL http://dx.doi.org/10.1007/978-3-031-08645-8_2. [9.6](#)
- [40] Frank Elavsky. Chartability, 2021. URL <https://chartability.github.io/POUR-CAF/>. Accessed: 2021-09-07. [4.5](#)
- [41] Frank Elavsky and Cindy Xiong Bearfield. Playing telephone with generative models: “verification disability,” “compelled reliance,” and accessibility in data visualization. In *2025 IEEE Workshop on Accessible Data Visualization (AccessViz)*, page 14–24. IEEE, November 2025. doi: 10.1109/accessviz68666.2025.00008. URL <https://doi.org/10.1109/accessviz68666.2025.00008>. [6.3.2](#)
- [42] Frank Elavsky, Cynthia Bennett, and Dominik Moritz. How accessible is my visualization? evaluating visualization accessibility with chartability. *Computer Graphics Forum*, 41(3):57–70, June 2022. doi: 10.1111/cgf.14522. URL <https://doi.org/10.1111/cgf.14522>. [6.4.3](#), [6.8.2](#)
- [43] Frank Elavsky, Lucas Nadolskis, and Dominik Moritz. Data navigator: An accessibility-centered data navigation toolkit. *IEEE Transactions on Visualization and Computer Graphics*, page 1–11, 2023. ISSN 2160-9306. doi: 10.1109/tvcg.2023.3327393. URL <https://doi.org/10.1109/tvcg.2023.3327393>. [6.2](#), [6.3.2](#), [6.3.3](#), [6.4](#)
- [44] A. Endert, L. Bradel, and C. North. Beyond control panels: Direct manipulation for visual analytics. *IEEE Computer Graphics and Applications*, 33(4):6–13, July 2013. ISSN 0272-1716. doi: 10.1109/mcg.2013.53. URL <https://doi.org/10.1109/mcg.2013.53>. [6.2](#)
- [45] Pavithra Eswaramoorthy, Frank Elavsky, Tania Allard, and gabalafou. Quansight-Labs/bokeh-a11y-audit: v1.0.0, February 2025. URL <https://doi.org/10.5281/zenodo.14923642>. [6.4.3](#)
- [46] Danyang Fan, Alexa Fay Siu, Hrishikesh Rao, Gene Sung-Ho Kim, Xavier Vazquez, Lucy Greco, Sile O’Modhrain, and Sean Follmer. The accessibility of data visualizations on the web for screen reader users: Practices and experiences during covid-19. *ACM Transactions on Accessible Computing*, 16(1):1–29, March 2023. ISSN 1936-7236. doi: 10.1145/3557899. URL <https://doi.org/10.1145/3557899>. [1.1](#), [6.3.3](#)
- [47] Ali Mazraeh Farahani, Peyman Adibi, Mohammad Saeed Ehsani, Hans-Peter Hutter, and Alireza Darvishy. Automatic chart understanding: A review. *IEEE Access*, 11: 76202–76221, 2023. ISSN 2169-3536. doi: 10.1109/access.2023.3298050. URL <https://doi.org/10.1109/access.2023.3298050>. [6.3.2](#)
- [48] Stephen Few. Visual Business Intelligence - Data Visualization and the Blind. <https://www.perceptualedge.com/blog/?p=1756>, 2013. [Accessed 05-04-2025].

1.1

- [49] Danyel Fisher, Igor Popov, Steven Drucker, and m.c. schraefel. Trust me, i'm partially right: incremental visualization lets analysts explore large datasets faster. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '12, page 1673–1682. ACM, May 2012. doi: 10.1145/2207676.2208294. URL <https://doi.org/10.1145/2207676.2208294>. 6.2
- [50] Chris Fleizach and Jeffrey P. Bigham. System-class accessibility: The architectural support for making a whole system usable by people with disabilities. *Queue*, 22(5): 28–39, October 2024. ISSN 1542-7749. doi: 10.1145/3704627. URL <http://dx.doi.org/10.1145/3704627>. 9.6
- [51] John H. Flowers, Dion C. Buhman, and Kimberly D. Turnage. Cross-Modal Equivalence of Visual and Auditory Scatterplots for Exploring Bivariate Data Samples. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, 39(3):341–351, September 1997. doi: 10.1518/001872097778827151. Accessed: 2021-09-03. 4.3.1
- [52] Camilla Forsell and Jimmy Johansson. An heuristic set for evaluation in information visualization. In *Proceedings of the International Conference on Advanced Visual Interfaces*, AVI '10, page 199–206, New York, NY, USA, May 2010. Association for Computing Machinery, ACM. doi: 10.1145/1842993.1843029. URL <https://doi.org/10.1145/1842993.1843029>. Accessed: 2022-03-06. 4.3.4
- [53] Plotly Community Forum. Are plotly tables accessible? <https://community.plotly.com/t/are-plotly-tables-accessible/8263>, February 2018. Accessed: 2021-09-06. 4.3.2
- [54] Plotly Community Forum. Datatables and Accessibility. <https://community.plotly.com/t/solved-datatables-and-accessibility/31085>, November 2019. Accessed: 2021-09-06. 4.3.2
- [55] Steven L. Franconeri, Lace M. Padilla, Priti Shah, Jeffrey M. Zacks, and Jessica Hullman. The science of visual data communication: What works. *Psychological Science in the Public Interest*, 22(3):110–161, December 2021. ISSN 1539-6053. doi: 10.1177/15291006211051956. URL <https://doi.org/10.1177/15291006211051956>. 1.2
- [56] Alberto Gallace and Charles Spence. To what extent do Gestalt grouping principles influence tactile perception? *Psychological bulletin*, 137(4):538–561, 2011. ISSN 0033-2909. doi: 10.1037/a0022335. URL <https://doi.org/10.1037/a0022335>. 4.3.1
- [57] Paula Gardner and Jess Rauchberg. *Feminist, Postcolonial, and Crip Approaches to Human-machine Communication Methodology*. Sage Publishing, 2023. 10.2
- [58] Josefina Isabel Gil Urrutia, Eric Brangier, Véronique Senderowicz, and Laurent Cessat. Beyond “usability and user experience”, towards an integrative heuristic inspection: From accessibility to persuasiveness in the ux evaluation. *Advances in Intelligent Systems and Computing*, pages 460–470, June 2017. doi: 10.1007/978-3-319-60492-3_44. URL https://doi.org/10.1007/978-3-319-60492-3_44. 4.3.4
- [59] Kate S Glazko, Momona Yamagami, Aashaka Desai, Kelly Avery Mack, Venkatesh

- Potluri, Xuhai Xu, and Jennifer Mankoff. An autoethnographic case study of generative artificial intelligence’s utility for accessibility. In *The 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’23, page 1–8. ACM, October 2023. doi: 10.1145/3597638.3614548. URL <http://dx.doi.org/10.1145/3597638.3614548>. 10.2
- [60] Michele E. Gloede and Melissa K. Gregg. The fidelity of visual and auditory memory. *Psychonomic Bulletin & Review*, 26(4):1325–1332, April 2019. ISSN 1531-5320. doi: 10.3758/s13423-019-01597-7. URL <https://doi.org/10.3758/s13423-019-01597-7>. 1.1
- [61] A. Jonathan R. Godfrey, Paul Murrell, and Volker Sorge. An accessible interaction model for data visualisation in statistics. In *Lecture Notes in Computer Science*, page 590–597. Springer International Publishing, 2018. ISBN 9783319942773. doi: 10.1007/978-3-319-94277-3_92. URL https://doi.org/10.1007/978-3-319-94277-3_92. 4.3.1
- [62] David Graeber and Charlie Rose. Never cower to authority, 2006. URL <https://www.youtube.com/watch?v=qt5op6wft-8>. Accessed [2026]. 10.3
- [63] R.L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133, 1972. ISSN 0020-0190. doi: 10.1016/0020-0190(72)90045-2. URL [http://dx.doi.org/10.1016/0020-0190\(72\)90045-2](http://dx.doi.org/10.1016/0020-0190(72)90045-2). 6.5.4.2
- [64] Colin M. Gray, Erik Stolterman, and Martin A. Siegel. Reprioritizing the relationship between hci research and practice. In *Proceedings of the 2014 conference on Designing interactive systems*, DIS ’14, pages 725–734, New York, NY, USA, June 2014. Association for Computing Machinery, ACM. doi: 10.1145/2598510.2598595. 4.4
- [65] David Gray Widder, Sarah West, and Meredith Whittaker. Open (for business): Big tech, concentrated power, and the political economy of open ai. *SSRN Electronic Journal*, 2023. ISSN 1556-5068. doi: 10.2139/ssrn.4543807. URL <https://doi.org/10.2139/ssrn.4543807>. 1.2
- [66] Karl Groves. Overlay fact sheet: What it is and how to make it work for you - karl groves, March 2023. URL <https://karlgroves.com/overlay-fact-sheet-what-it-is-and-how-to-make-it-work-for-you/>. 9.6
- [67] Péter Ferenc Gyarmati, Dominik Moritz, Torsten Möller, and Laura Koesten. Structured visualization design knowledge for grounding generative reasoning and situated feedback, 2025. URL <https://doi.org/10.48550/arxiv.2512.20306>. 4.11
- [68] Jen Hale. Extensive digitization of tactile map collection. <https://www.perkins.org/extensive-digitization-of-tactile-map-collection/>, 2016. Perkins Archives Blog, Perkins School for the Blind. July 2016. Accessed: 2022-02-23. 4.3.1
- [69] Noor Hammad, Frank Elavsky, Sanika Moharana, Jessie Chen, Seyoung Lee, Patrick Carrington, Dominik Moritz, Jessica Hammer, and Erik Harpstead. Exploring the affordances of game-aware streaming to support blind and low vision viewers: A design probe study. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*,

- ASSETS '24, page 1–13. ACM, October 2024. doi: 10.1145/3663548.3675665. URL <https://doi.org/10.1145/3663548.3675665>. 6.1.1, 6.2
- [70] Gunnar Harboe and Elaine M. Huang. Real-world affinity diagramming practices: Bridging the paper-digital gap. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, CHI '15, page 95–104. ACM, April 2015. doi: 10.1145/2702123.2702561. URL <https://doi.org/10.1145/2702123.2702561>. 6.6.3
- [71] Parker Hartman and Tim Gorichanaz. Evaluating ai-powered website accessibility overlays. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '25, page 1–4. ACM, October 2025. doi: 10.1145/3663547.3759761. URL <http://dx.doi.org/10.1145/3663547.3759761>. 9.6
- [72] Gillian R. Hayes. The relationship of action research to human-computer interaction. *ACM Transactions on Computer-Human Interaction*, 18(3):1–20, July 2011. ISSN 1073-0516. doi: 10.1145/1993060.1993065. URL <https://doi.org/10.1145/1993060.1993065>. 4.1.1, 6.2, 6.4, 10.2
- [73] Jeffrey Heer and Maneesh Agrawala. Design considerations for collaborative visual analytics. In *2007 IEEE Symposium on Visual Analytics Science and Technology*, page 171–178. IEEE, October 2007. doi: 10.1109/vast.2007.4389011. URL <https://doi.org/10.1109/vast.2007.4389011>. 6.2
- [74] Jeffrey Heer and Dominik Moritz. Mosaic: An architecture for scalable & interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, page 1–11, 2023. ISSN 2160-9306. doi: 10.1109/tvcg.2023.3327189. URL <https://doi.org/10.1109/tvcg.2023.3327189>. 1.2
- [75] Ana O Henriques, Anna R. L. Carter, Beatriz Severes, Reem Talhouk, Angelika Strohmayer, Ana Cristina Pires, Colin M. Gray, Kyle Montague, and Hugo Nicolau. A feminist care ethics toolkit for community-based design: Bridging theory and practice. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, page 1–26. ACM, April 2025. doi: 10.1145/3706598.3713950. URL <http://dx.doi.org/10.1145/3706598.3713950>. 10.3
- [76] Thomas Hermann, Andy Hunt, and John G Neuhoff, editors. *The Sonification Handbook*. Logos Verlag Berlin, Berlin, Germany, December 2011. ISBN 9783832528195. ISBN: 978-3-8325-2819-5. 6.3.2
- [77] Highsoft. Highcharts - interactive charting library for developers. <https://www.highcharts.com/>, 2009. [Online]. 4.3.2
- [78] Highsoft. Highcharts accessibility module: information and demos. <https://highcharts.com/docs/accessibility/accessibility-module>, 2018. Accessed: 2021-09-06. 4.6.3
- [79] Toan Hoang. The TableauMagic DataTables Extension. <https://tableau.toanhoang.com/the-tableau-magic-datatables-extension-now-available/>, September 2018. Accessed: 2021-09-03. 4.3.2

- [80] Megan Hofmann, Devva Kasnitz, Jennifer Mankoff, and Cynthia L Bennett. Living disability theory: Reflections on access, research, and design. In *Proceedings of the 22nd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '20, page 1–13. ACM, October 2020. doi: 10.1145/3373625.3416996. URL <http://dx.doi.org/10.1145/3373625.3416996>. 10.2
- [81] Leona Holloway, Peter Cracknell, Kate Stephens, Melissa Fanshawe, Samuel Reinders, Kim Marriott, and Matthew Butler. Refreshable tactile displays for accessible data visualisation, 2024. URL <https://doi.org/10.48550/arxiv.2401.15836>. 6.3.2
- [82] Leona M Holloway, Cagatay Goncu, Alon Ilisar, Matthew Butler, and Kim Marriott. Infosonics: Accessible infographics for people who are blind using sonification and voice. In *CHI Conference on Human Factors in Computing Systems*, CHI '22, page 1–13. ACM, April 2022. doi: 10.1145/3491102.3517465. URL <https://doi.org/10.1145/3491102.3517465>. 6.3.2
- [83] Stacy Hsueh, Beatrice Vincenzi, Akshata Murdeshwar, and Marianela Ciolfi Felice. Crip-ping data visualizations: Crip technoscience as a critical lens for designing digital access. In *The 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '23, page 1–16. ACM, October 2023. doi: 10.1145/3597638.3608427. URL <http://dx.doi.org/10.1145/3597638.3608427>. 1.1, 10.2
- [84] Amy Hurst and Shaun Kane. Making "making" accessible. In *Proceedings of the 12th International Conference on Interaction Design and Children*, IDC '13, pages 635–638, New York, NY, USA, June 2013. Association for Computing Machinery, ACM. doi: 10.1145/2485760.2485883. Accessed: 2021-09-03. 4.8, 10.2
- [85] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. Direct manipulation interfaces. *Human–Computer Interaction*, 1(4):311–338, December 1985. ISSN 1532-7051. doi: 10.1207/s15327051hci0104.2. URL https://doi.org/10.1207/s15327051hci0104_2. 6.2, 6.3.1
- [86] Hilary Hutchinson, Wendy Mackay, Bo Westerlund, Benjamin B. Bederson, Allison Druin, Catherine Plaisant, Michel Beaudouin-Lafon, Stéphane Conversy, Helen Evans, Heiko Hansen, Nicolas Roussel, and Björn Eiderbäck. Technology probes: inspiring design for and with families. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI03, page 17–24. ACM, April 2003. doi: 10.1145/642611.642616. URL <https://doi.org/10.1145/642611.642616>. 6.1.1, 6.2
- [87] Web Accessibility Initiative. Web Accessibility Laws & Policies. <https://www.w3.org/WAI/policies/>, August 2021. Accessed: 2021-08-19. 4.3.3
- [88] Lilly C. Irani and M. Six Silberman. Turkopticon: Interrupting worker invisibility in amazon mechanical turk. In *Proceedings of the SIGCHI conference on human factors in computing systems*, CHI '13, page 611–620, New York, NY, USA, April 2013. Association for Computing Machinery, ACM. doi: 10.1145/2470654.2470742. URL <https://doi.org/10.1145/2470654.2470742>. 4.8

- [89] Liz Jackson, Alex Haagaard, and Rua Williams. Disability Dongle. <https://blog.castac.org/2022/04/disability-dongle/>, 2022. 10.2
- [90] Farnaz Jahanbakhsh, Amy X. Zhang, Karrie Karahalios, and David R. Karger. Our browser extension lets readers change the headlines on news articles, and you won’t believe what they did! *Proceedings of the ACM on Human-Computer Interaction*, 6 (CSCW2):1–33, November 2022. ISSN 2573-0142. doi: 10.1145/3555643. URL <http://dx.doi.org/10.1145/3555643>. 10.3
- [91] Yvonne Jansen, Pierre Dragicevic, Petra Isenberg, Jason Alexander, Abhijit Karnik, Johan Kildal, Sriram Subramanian, and Kasper Hornbæk. *Opportunities and Challenges for Data Physicalization*, page 3227–3236. CHI ’15. ACM, New York, NY, USA, April 2015. doi: 10.1145/2702123.2702180. URL <https://doi.org/10.1145/2702123.2702180>. Accessed: 2021-09-03. 4.3.1
- [92] Chandrika Jayant, Matt Renzelmann, Dana Wen, Satria Krisnandi, Richard Ladner, and Dan Comden. Automated tactile graphics translation: in the field. In *Proceedings of the 9th international ACM SIGACCESS conference on Computers and accessibility*, ASSETS07, page 75–82, New York, NY, USA, October 2007. ACM. ISBN 978-1-59593-573-1. doi: 10.1145/1296843.1296858. URL <https://doi.org/10.1145/1296843.1296858>. Accessed: 2021-09-03. 4.3.1
- [93] Shuli Jones, Isabella Pedraza Pineros, Daniel Hajas, Jonathan Zong, and Arvind Satyanarayan. “customization is key”: Reconfigurable textual tokens for accessible data visualizations. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI ’24, page 1–14. ACM, May 2024. doi: 10.1145/3613904.3641970. URL <https://doi.org/10.1145/3613904.3641970>. 6.3.2, 6.5.2
- [94] Ger Joyce, Mariana Lilley, Trevor Barker, and Amanda Jefferies. Mobile Application Usability: Heuristic Evaluation and Evaluation of Heuristics. In Ben Amaba, editor, *Advances in Human Factors, Software, and Systems Engineering*, Advances in Intelligent Systems and Computing, page 77–86, Cham, 2016. Springer International Publishing, Springer International Publishing. ISBN 9783319419350. doi: 10.1007/978-3-319-41935-0_8. 4.3.4
- [95] Shakila Cherise S Joyner, Amalia Riegelhuth, Kathleen Garrity, Yea-Seul Kim, and Nam Wook Kim. Visualization accessibility in the wild: Challenges faced by visualization designers. In *CHI Conference on Human Factors in Computing Systems*, CHI ’22, page 1–19. ACM, April 2022. doi: 10.1145/3491102.3517630. URL <https://doi.org/10.1145/3491102.3517630>. 6.2, 6.3.3, 6.8.2, 9.1, 10.2
- [96] Crescentia Jung, Shubham Mehta, Atharva Kulkarni, Yuhang Zhao, and Yea-Seul Kim. Communicating visualizations without visuals: Investigation of visualization alternative text for people with visual impairments. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1095–1105, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114846. URL <https://doi.org/10.1109/tvcg.2021.3114846>. 4.3.1, 6.3.2
- [97] Os Keyes, Josephine Hoy, and Margaret Drouhard. Human-computer insurrection: Notes

- on an anarchist hci. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–13. ACM, May 2019. doi: 10.1145/3290605.3300569. URL <http://dx.doi.org/10.1145/3290605.3300569>. 10.3
- [98] Gyeongri Kim, Jiho Kim, and Yea-Seul Kim. “explain what a treemap is”: Exploratory investigation of strategies for explaining unfamiliar chart to blind and low vision users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–13. ACM, April 2023. doi: 10.1145/3544548.3581139. URL <https://doi.org/10.1145/3544548.3581139>. 6.3.2
- [99] Hyeok Kim, Yea-Seul Kim, and Jessica Hullman. Erie: A declarative grammar for data sonification. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI '24, page 1–19. ACM, May 2024. doi: 10.1145/3613904.3642442. URL <https://doi.org/10.1145/3613904.3642442>. 6.3.2, 6.3.3
- [100] Jiho Kim, Arjun Srinivasan, Nam Wook Kim, and Yea-Seul Kim. Exploring chart question answering for blind and low vision users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–15. ACM, April 2023. doi: 10.1145/3544548.3581532. URL <https://doi.org/10.1145/3544548.3581532>. 6.3.2
- [101] N. W. Kim, S. C. Joyner, A. Riegelhuth, and Y. Kim. Accessible visualization: Design space, opportunities, and challenges. *Computer Graphics Forum*, 40(3):173–188, June 2021. doi: 10.1111/cgf.14298. URL <https://doi.org/10.1111/cgf.14298>. 4.3.1, 6.3.2
- [102] N. W. Kim, G. Ataguba, S. C. Joyner, Chuangdian Zhao, and Hyejin Im. Beyond alternative text and tables: Comparative analysis of visualization tools and accessibility methods. *Computer Graphics Forum*, 42(3):323–335, June 2023. ISSN 1467-8659. doi: 10.1111/cgf.14833. URL <https://doi.org/10.1111/cgf.14833>. 6.3.2, 6.3.3
- [103] Rebecca Klein. Power BI Accessibility Best Practices. <https://rklein324.github.io/PowerBIAccessibility/>, 2020. Accessed: 2021-09-06. 4.3.2
- [104] Richard E. Ladner. Design for user empowerment. *Interactions*, 22(2):24–29, February 2015. ISSN 1558-3449. doi: 10.1145/2723869. URL <https://doi.org/10.1145/2723869>. Accessed: 2022-03-07. 4.4.3
- [105] Chufan Lai, Zhixian Lin, Ruike Jiang, Yun Han, Can Liu, and Xiaoru Yuan. *Automatic Annotation Synchronizing with Textual Description for Visualization*, page 1–13. CHI '20. ACM, New York, NY, USA, April 2020. doi: 10.1145/3313831.3376443. URL <https://doi.org/10.1145/3313831.3376443>. Accessed: 2021-09-06. 4.3.1
- [106] Susan J. Lederman, Georgie Thorne, and Bill Jones. Perception of texture by vision and touch: Multidimensionality and intersensory integration. *Journal of experimental psychology. Human perception and performance*, 12(2):169–180, 1986. ISSN 0096-1523. doi: 10.1037//0096-1523.12.2.169. URL <https://doi.org/10.1037//0096-1523.12.2.169>. 4.3.1
- [107] Bongshin Lee, Eun Kyoung Choe, Petra Isenberg, Kim Marriott, and John Stasko. Reach-

- ing broader audiences with data visualization. *IEEE Computer Graphics and Applications*, 40(2):82–90, March 2020. ISSN 1558-1756. doi: 10.1109/MCG.2020.2968244. URL <https://doi.org/10.1109/mcg.2020.2968244>. 4.3.1
- [108] Zhicheng Liu and Jeffrey Heer. The effects of interactive latency on exploratory visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):2122–2131, December 2014. ISSN 1077-2626. doi: 10.1109/tvcg.2014.2346452. URL <https://doi.org/10.1109/tvcg.2014.2346452>. 6.2, 9.6
- [109] Alan Lundgard and Arvind Satyanarayan. Accessible visualization via natural language descriptions: A four-level model of semantic content. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1073–1083, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114770. URL <https://doi.org/10.1109/tvcg.2021.3114770>. 4.3.1, 6.3.2
- [110] Alan Lundgard, Crystal Lee, and Arvind Satyanarayan. Sociotechnical considerations for accessible visualization design. In *2019 IEEE Visualization Conference (VIS)*, page 16–20. IEEE, October 2019. doi: 10.1109/visual.2019.8933762. URL <https://doi.org/10.1109/visual.2019.8933762>. 4.3.1, 4.3.3, 6.3.3, 10.2
- [111] Kelly Mack, Emma McDonnell, Dhruv Jain, Lucy Lu Wang, Jon E. Froehlich, and Leah Findlater. What do we mean by “accessibility research”? A literature survey of accessibility papers in chi and assets from 1994 to 2019. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI ’21, page 1–18, New York, NY, USA, May 2021. ACM. ISBN 978-1-4503-8096-6. doi: 10.1145/3411764.3445412. URL <https://doi.org/10.1145/3411764.3445412>. Accessed: 2022-02-22. 4.3.1
- [112] Shalini Madan, Sreelakshmi Surabiyil Bindu, and Venkatesh Potluri. Vibe check: Accessibility heuristics for vibe coding interfaces. In *Proceedings of the 23rd International Web for All Conference*, W4A ’26, page 229–241. ACM, April 2026. doi: 10.1145/3800424.3800455. URL <http://dx.doi.org/10.1145/3800424.3800455>. 4.11
- [113] Els Maeckelberghe. Feminist ethic of care: A third alternative approach. *Health Care Analysis*, 12(4):317–327, December 2004. ISSN 1573-3394. doi: 10.1007/s10728-004-6639-6. URL <http://dx.doi.org/10.1007/s10728-004-6639-6>. 10.3
- [114] Tlameo Makati, Garreth W. Tigwell, and Kristen Shinohara. The promise and pitfalls of web accessibility overlays for blind and low vision users. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’24, page 1–12. ACM, October 2024. doi: 10.1145/3663548.3675650. URL <http://dx.doi.org/10.1145/3663548.3675650>. 9.6
- [115] Jennifer Mankoff, Gillian R. Hayes, and Devva Kasnitz. Disability studies as a source of critical inquiry for the field of assistive technology. In *Proceedings of the 12th international ACM SIGACCESS conference on Computers and accessibility*, ASSETS ’10, page 3–10, New York, NY, USA, October 2010. Association for Computing Machinery, ACM. doi: 10.1145/1878803.1878807. URL <https://doi.org/10.1145/1878803.1878807>. Accessed: 2021-09-06. 4.4.3, 10.2

- [116] Douglass L. Mansur, Merra M. Blattner, and Kenneth I. Joy. Sound graphs: A numerical data analysis method for the blind. *Journal of Medical Systems*, 9(3):163–174, June 1985. ISSN 1573-689X. doi: 10.1007/bf00996201. URL <https://doi.org/10.1007/bf00996201>. 4.3.1, 6.3.2
- [117] Mapbox. Mapbox gl accessibility. <https://github.com/mapbox/mapbox-gl-accessibility>, August 2021. Accessed: 2021-09-06. 4.3.2
- [118] Mapbox. Mapbox-gl js. <https://www.npmjs.com/package/mapbox-gl>, 2022. Accessed: 2021-09-03. 4.3.2
- [119] Deborah Marks. Models of disability. *Disability and Rehabilitation*, 19(3):85–91, January 1997. ISSN 1464-5165. doi: 10.3109/09638289709166831. URL <https://doi.org/10.3109/09638289709166831>. 1.1, 9.1
- [120] Kim Marriott, Bongshin Lee, Matthew Butler, Ed Cutrell, Kirsten Ellis, Cagatay Goncu, Marti Hearst, Kathleen McCoy, and Danielle Albers Szafrir. Inclusive data visualization for people with disabilities: a call to action. *Interactions*, 28(3):47–51, April 2021. ISSN 1558-3449. doi: 10.1145/3457875. URL <https://doi.org/10.1145/3457875>. 4.3.1, 6.3.2
- [121] Kim Marriott, Matthew Butler, Leona Holloway, William Jolley, Bongshin Lee, Bruce Maguire, and Danielle Albers Szafrir. From vision to touch: Bridging visual and tactile principles for accessible data representation. *IEEE Transactions on Visualization and Computer Graphics*, 32(1):659–669, January 2026. ISSN 2160-9306. doi: 10.1109/tvcg.2025.3634254. URL <https://doi.org/10.1109/tvcg.2025.3634254>. 6.3.2
- [122] Rubén Alcaraz Martínez, Mireia Ribera Turró, and Toni Granollers Saltiveri. Methodology for heuristic evaluation of the accessibility of statistical charts for people with low vision and color vision deficiency, February 2021. URL <https://doi.org/10.21203/rs.3.rs-156959/v1>. 4.3.1, 4.3.4
- [123] Michele Mauri, Elena Aversa, Anna Cattaneo, and María De Los Ángeles Briones Rojas. Ddenso: an evaluation-driven methodology for information visualisations. In *Proceedings of DRS*, DRS2026. Design Research Society, June 2026. doi: 10.21606/drs.2026.1396. URL <http://dx.doi.org/10.21606/drs.2026.1396>. 4.11
- [124] Melanie Mazanec. Semiotic. <https://semiotic.neract.io/guides/accessibility>, 2019. Accessed: 2021-09-03. 4.3.2
- [125] David K. McGookin and Stephen A. Brewster. Soundbar: Exploiting multiple views in multimodal graph browsing. In *Proceedings of the 4th Nordic conference on Human-computer interaction: Changing roles*, NORDICHI06, page 145–154, New York, NY, USA, October 2006. Association for Computing Machinery, ACM. doi: 10.1145/1182475.1182491. URL <https://doi.org/10.1145/1182475.1182491>. Accessed: 2021-09-03. 4.3.1
- [126] Andrew M. McNutt. No grammar to rule them all: A survey of json-style dsls for visualization. *IEEE Transactions on Visualization and Computer Graphics*, page 1–11, 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2022.3209460. URL <https://doi.org/10.1109/tvcg.2022.3209460>. 4.3.1

- [127] Catherine Mei*, Josh Pollock*, Daniel Hajas, Jonathan Zong, and Arvind Satyanarayan. Benthic: Perceptually congruent structures for accessible charts and diagrams. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '25, page 1–17. ACM, October 2025. doi: 10.1145/3663547.3746342. URL <https://doi.org/10.1145/3663547.3746342>. 6.3.2, 6.3.3
- [128] Mia Mingus. Access intimacy: The missing link. <https://leavingevidence.wordpress.com/2011/05/05/access-intimacy-the-missing-link/>, May 2011. Blog post, *Leaving Evidence*. 1.1
- [129] Ehsan Mohammadi, Mike Thelwall, Stefanie Haustein, and Vincent Larivière. Who reads research articles? an altmetrics analysis of `scp;mj/scp;endeley` user categories. *Journal of the Association for Information Science and Technology*, 66(9):1832–1846, April 2015. ISSN 2330-1643. doi: 10.1002/asi.23286. URL <http://dx.doi.org/10.1002/asi.23286>. 10.2
- [130] Giles Mohan and Kristian Stokke. Participatory development and empowerment: The dangers of localism. *Third World Quarterly*, 21(2):247–268, April 2000. ISSN 1360-2241. doi: 10.1080/01436590050004346. URL <http://dx.doi.org/10.1080/01436590050004346>. 10.3
- [131] Priscilla Moraes, Gabriel Sina, Kathleen McCoy, and Sandra Carberry. Evaluating the accessibility of line graphs through textual summaries for visually impaired users. In *Proceedings of the 16th international ACM SIGACCESS conference on Computers & accessibility - ASSETS '14*, ASSETS '14, page 83–90, New York, NY, USA, October 2014. Association for Computing Machinery, ACM Press. doi: 10.1145/2661334.2661368. URL <https://doi.org/10.1145/2661334.2661368>. 4.3.1
- [132] Peya Mowar, Aaron Steinfeld, and Jeffrey P. Bigham. iTagPDF: Towards finally automating PDF accessibility. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI '26, page 1–17. ACM, April 2026. doi: 10.1145/3772318.3790289. URL <https://doi.org/10.1145/3772318.3790289>. Best Paper Award. 6.3.3
- [133] Tamara Munzner. *Visualization Analysis and Design*. A K Peters/CRC Press, December 2014. ISBN 9780429088902. doi: 10.1201/b17511. URL <http://dx.doi.org/10.1201/b17511>. 4.1.1
- [134] Brad A. Myers. Taxonomies of visual programming and program visualization. *Journal of Visual Languages & Computing*, 1(1):97–123, March 1990. ISSN 1045-926X. doi: 10.1016/S1045-926X(05)80036-9. URL [https://doi.org/10.1016/S1045-926X\(05\)80036-9](https://doi.org/10.1016/S1045-926X(05)80036-9). 6.3.1
- [135] I. Nassi and B. Shneiderman. Flowchart techniques for structured programming. *ACM SIGPLAN Notices*, 8(8):12–26, August 1973. ISSN 1558-1160. doi: 10.1145/953349.953350. URL <https://doi.org/10.1145/953349.953350>. 6.3.1
- [136] Jakob Nielsen. 10 Usability Heuristics for User Interface Design. <https://www.nngroup.com/articles/ten-usability-heuristics/>, 1994. Accessed: 2021-

- [137] Jakob Nielsen. *Heuristic evaluation*, pages 25–62. John Wiley & Sons, Inc., USA, June 1994. Accessed: 2021-09-06. 4.3.4
- [138] Jamie R. Nuñez, Christopher R. Anderton, and Ryan S. Renslow. Optimizing colormaps with consideration for color vision deficiency to enable accurate interpretation of scientific data. *PLOS ONE*, 13(7):e0199239, August 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0199239. URL <https://doi.org/10.1371/journal.pone.0199239>. 4.3.1
- [139] Jason Obeid and Enamul Hoque. Chart-to-Text: Generating Natural Language Descriptions for Charts by Adapting the Transformer Model. *arXiv Preprint*, November 2020. Accessed: 2021-09-06. 4.3.1
- [140] Catherine A. Okoro, NaTasha D. Hollis, Alissa C. Cyrus, and Shannon Griffin-Blake. Prevalence of disabilities and health care access by disability status and type among adults — united states, 2016. *MMWR. Morbidity and Mortality Weekly Report*, 67(32):882–887, August 2018. ISSN 1545-861X. doi: 10.15585/mmwr.mm6732a3. URL <https://doi.org/10.15585/mmwr.mm6732a3>. 1.1, 4.2
- [141] Manuel M. Oliveira. Towards More Accessible Visualizations for Color-Vision-Deficient Individuals. *Computing in Science & Engineering*, 15(5):80–87, September 2013. doi: 10.1109/MCSE.2013.113. Conference Name: Computing in Science Engineering. 4.3.1, 4.3.4
- [142] World Health Organization. *World report on vision*. World Health Organization, 2019. ISBN 978-92-4-151657-0. Accessed: 2021-09-03. 4.2
- [143] Daniela Quiñones Otey. A methodology to develop usability / user experience heuristics. In *Proceedings of the XVIII International Conference on Human Computer Interaction, Interacción '17*, pages 1–2, New York, NY, USA, September 2017. Association for Computing Machinery, ACM. doi: 10.1145/3123818.3133832. 4.3.4
- [144] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, January 1979. ISSN 2168-2909. doi: 10.1109/tsmc.1979.4310076. URL <http://dx.doi.org/10.1109/TSMC.1979.4310076>. 6.5.4.3
- [145] Swaroop Panda. Llm-based visualization evaluation: How well do literacy-stratified personas approximate human judgments?, 2026. URL <https://doi.org/10.48550/arxiv.2606.10095>. 4.11
- [146] Amandalynne Paullada, Inioluwa Deborah Raji, Emily M. Bender, Emily Denton, and Alex Hanna. Data and its (dis)contents: A survey of dataset development and use in machine learning research. *Patterns*, 2(11):100336, November 2021. ISSN 2666-3899. doi: 10.1016/j.patter.2021.100336. URL <http://dx.doi.org/10.1016/j.patter.2021.100336>. 9.6
- [147] Jorge Poco and Jeffrey Heer. Reverse-engineering visualizations: Recovering visual encodings from chart images. *Computer Graphics Forum*, 36(3):353–363, 2017. ISSN

- 1467-8659. doi: 10.1111/cgf.13193. URL <http://dx.doi.org/10.1111/cgf.13193>. 6.5.4.3
- [148] Christopher Power, André Freire, Helen Petrie, and David Swallow. Guidelines are only half of the story: Accessibility problems encountered by blind users on the web. In *Proceedings of the SIGCHI conference on human factors in computing systems*, CHI '12, page 433–442, New York, NY, USA, May 2012. Association for Computing Machinery, ACM. doi: 10.1145/2207676.2207736. URL <https://doi.org/10.1145/2207676.2207736>. 4.2
- [149] Xin Qian, Eunye Koh, Fan Du, Sungchul Kim, Joel Chan, Ryan A. Rossi, Sana Malik, and Tak Yeon Lee. *Generating Accurate Caption Units for Figure Captioning*, page 2792–2804. WWW '21. ACM, New York, NY, USA, April 2021. doi: 10.1145/3442381.3449923. URL <https://doi.org/10.1145/3442381.3449923>. Accessed: 2021-09-06. 4.3.1
- [150] Lauren Race, Chancey Fleet, Danielle Montour, Lindsay Yazzolino, Marco Salsiccia, Claire Ferrari, Sheri Wells-Jensen, and Amy Hurst. Designing while blind: Nonvisual tools and inclusive workflows for tactile graphic creation. In *The 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '23, page 1–8. ACM, October 2023. doi: 10.1145/3597638.3614546. URL <https://doi.org/10.1145/3597638.3614546>. 6.4
- [151] J.E. Rawling, E.C. Carson, J.W. Attig, D.M. Mickelson, W.N. Mode, M.D. Johnson, and K.M. Syverson. Quaternary geology of wisconsin. Technical report, Wisconsin Geological and Natural History Survey, 2025. URL <https://doi.org/10.54915/xqpw9883>. 6.4.1
- [152] Blake Ellis Reid. The curb-cut effect and the perils of accessibility without disability. *SSRN Electronic Journal*, 2022. ISSN 1556-5068. doi: 10.2139/ssrn.4262991. URL <https://doi.org/10.2139/ssrn.4262991>. 1.1
- [153] Samuel Reinders, Matthew Butler, Ingrid Zukerman, Bongshin Lee, Lizhen Qu, and Kim Marriott. When refreshable tactile displays meet conversational agents: Investigating accessible data presentation and analysis with touch and speech. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):864–874, January 2025. ISSN 2160-9306. doi: 10.1109/tvcg.2024.3456358. URL <https://doi.org/10.1109/tvcg.2024.3456358>. 6.3.2
- [154] Rod D Roscoe, Erin K Chiou, and Abigail R Wooldridge, editors. *Advancing diversity, inclusion, and social justice through human systems engineering*. CRC Press, London, England, December 2019. 1.1
- [155] Azriel Rosenfeld and John L. Pfaltz. Sequential operations in digital picture processing. *Journal of the ACM*, 13(4):471–494, October 1966. ISSN 1557-735X. doi: 10.1145/321356.321357. URL <http://dx.doi.org/10.1145/321356.321357>. 6.5.4.3
- [156] Maurício Rossi de Oliveira and Celmar Guimarães da Silva. Adapting heuristic evaluation to information visualization - a method for defining a heuristic set by heuristic grouping. In *Proceedings of the 12th International Joint Conference on Computer Vision, Imaging and*

Computer Graphics Theory and Applications, page 225–232. INSTICC, SCITEPRESS - Science and Technology Publications, 2017. ISBN 978-989-758-228-8. doi: 10.5220/006133202250232. URL <https://doi.org/10.5220/006133202250232>. 4.3.4

- [157] Heather Ruland Staines. Digital open annotation with hypothesis: Supplying the missing capability of the web. *Journal of Scholarly Publishing*, 49(3):345–365, April 2018. ISSN 1710-1166. doi: 10.3138/jsp.49.3.04. URL <http://dx.doi.org/10.3138/jsp.49.3.04>. 10.3
- [158] Manaswi Saha, Michael Saugstad, Hanuma Teja Maddali, Aileen Zeng, Ryan Holland, Steven Bower, Aditya Dash, Sage Chen, Anthony Li, Kotaro Hara, and Jon Froehlich. Project sidewalk: A web-based crowdsourcing tool for collecting sidewalk accessibility data at scale. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI ’19, page 1–14. ACM, May 2019. doi: 10.1145/3290605.3300292. URL <http://dx.doi.org/10.1145/3290605.3300292>. 10.3
- [159] City San Francisco. Covid-19 data and reports San Francisco. <https://sf.gov/resource/2021/covid-19-data-and-reports>, 2021. Accessed: 2021-09-03. 4.3.2
- [160] Beatriz Sousa Santos, Samuel Silva, and Paulo Dias. Heuristic Evaluation in Visualization: An Empirical Study : Position paper. In *2018 IEEE Evaluation and Beyond - Methodological Approaches for Visualization (BELIV)*, page 78–85. IEEE, October 2018. doi: 10.1109/BELIV.2018.8634108. URL <https://doi.org/10.1109/beliv.2018.8634108>. 4.3.4
- [161] SAS. Sas Graphics Accelerator. <https://support.sas.com/software/products/graphics-accelerator/>, 2022. Accessed: 2021-09-08. 4.3.2
- [162] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, January 2017. ISSN 1077-2626. doi: 10.1109/tvcg.2016.2599030. URL <https://doi.org/10.1109/tvcg.2016.2599030>. 1.2, 6.2, 6.3.1, 6.5.2, 6.5.4.1
- [163] Manolis Savva, Nicholas Kong, Arti Chhajta, Li Fei-Fei, Maneesh Agrawala, and Jeffrey Heer. Revision: automated classification, analysis and redesign of chart images. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*, UIST ’11, page 393–402. ACM, October 2011. doi: 10.1145/2047196.2047247. URL <http://dx.doi.org/10.1145/2047196.2047247>. 6.5.4.3
- [164] Anastasia Schaadhardt, Alexis Hiniker, and Jacob O. Wobbrock. Understanding Blind Screen-Reader Users’ Experiences of Digital Artboards. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI ’21, New York, NY, USA, May 2021. Association for Computing Machinery. Accessed: 2021-09-06. 4.3.1
- [165] Doug Schepers. Why Accessibility Is at the Heart of Data Visualization by Doug Schepers Nightingale Medium. <https://medium.com/nightingale/accessibility-is-at-the-heart-of-data-visualization-64a38d6c505b>, 2020. Accessed: 2021-09-06. 4.4.2

- [166] Doug Schepers. On underlays. Internet Archive, 2026. URL https://web.archive.org/web/20260616140431/https://www.linkedin.com/posts/daschepers_accessibility-overlays-are-good-for-business-activity-7317652815399587840-lwVL?utm_source=share&utm_medium=member_desktop&rcm=ACoAADDawBkB0doW11I9B5DHy57VfR5jIs33Kq0. Archived version available at https://web.archive.org/web/20260616140431/https://www.linkedin.com/posts/daschepers_accessibility-overlays-are-good-for-business-activity-7317652815399587840-lwVL?utm_source=share&utm_medium=member_desktop&rcm=ACoAADDawBkB0doW11I9B5DHy57VfR5jIs33Kq0; original URL: [linkedin.com/posts/daschepers_accessibility-overlays-are-good-for-business-activity-7317652815399587840-lwVL](https://www.linkedin.com/posts/daschepers_accessibility-overlays-are-good-for-business-activity-7317652815399587840-lwVL). Accessed: 2026-06-16. 9.6
- [167] William Schiff and Emerson Foulke, editors. *Tactual Perception*. Cambridge University Press, Cambridge, England, March 1982. ISBN 0521240956. 4.3.1
- [168] Jochen Schneider. Constructing the Yellow Brick Road: Route Bricks on Virtual Tactile Maps, 2000. 4.3.1
- [169] Jean Scholtz. Developing guidelines for assessing visual analytics environments. *Information Visualization*, 10(3):212–231, July 2011. doi: 10.1177/1473871611407399. Accessed: 2022-03-06. 4.3.4
- [170] Jonathan Schwabish, Susan J Popkin, Alice Feng, Doug Schepers, Frank Elavsky, Elizabeth Hare, Leonie Watson, Sarah Fossheim, Larene Le Gassick, Dax Castro, Amy Cesal, and Melanie Mazanec. Do no harm guide: Centering accessibility in data visualization. <https://www.urban.org/research/publication/do-no-harm-guide-centering-accessibility-data-visualization>, 2022. [Accessed 01-04-2025]. 4.11
- [171] Donald Schön and John Bennett. Reflective conversation with materials, April 1996. URL <https://doi.org/10.1145/229868.230044>. 6.8.1
- [172] Muthu Selvam. Implementing ai-driven web accessibility: Wcag 2.2 features to improve usability for users with disabilities. In *2025 3rd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings)*, pages 1–6. IEEE, September 2025. doi: 10.1109/aibthings66987.2025.11296155. URL <http://dx.doi.org/10.1109/AIBThings66987.2025.11296155>. 4.11
- [173] JooYoung Seo, Yilin Xia, Bongshin Lee, Sean Mccurry, and Yu Jun Yam. Maidr: Making statistical visualizations accessible with multimodal data representation. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI ’24, page 1–22. ACM, May 2024. doi: 10.1145/3613904.3642730. URL <https://doi.org/10.1145/3613904.3642730>. 6.3.2, 10.2
- [174] Hanieh Shakeri, Ye Yuan, Benett Axtell, Denise Y. Geiskkovitch, and Carman Neustaedter. Designing smart home technology for passive co-presence over distance. In *Designing Interactive Systems Conference*, DIS ’24, pages 3389–3406. ACM, July 2024.

doi: 10.1145/3643834.3661508. URL <http://dx.doi.org/10.1145/3643834.3661508>. 10.3

- [175] Ather Sharif and Babak Forouraghi. evographs — a jquery plugin to create web accessible graphs. In *2018 15th IEEE Annual Consumer Communications & Networking Conference (CCNC)*, page 1–4. IEEE, January 2018. doi: 10.1109/ccnc.2018.8319239. URL <https://doi.org/10.1109/ccnc.2018.8319239>. 4.3.1, 6.3.3
- [176] Ather Sharif, Sanjana Shivani Chintalapati, Jacob O. Wobbrock, and Katharina Reinecke. Understanding screen-reader users’ experiences with online data visualizations. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’21, page 1–16. ACM, October 2021. doi: 10.1145/3441852.3471202. URL <https://doi.org/10.1145/3441852.3471202>. 4.3.1, 4.3.3, 6.3.2, 6.3.3, 9.1
- [177] Ather Sharif, Olivia H. Wang, Alida T. Muongchan, Katharina Reinecke, and Jacob O. Wobbrock. Voxlens: Making online data visualizations accessible with an interactive javascript plug-in. In *CHI Conference on Human Factors in Computing Systems*, CHI ’22, page 1–19. ACM, April 2022. doi: 10.1145/3491102.3517431. URL <https://doi.org/10.1145/3491102.3517431>. 6.3.3
- [178] Ather Sharif, Joo Gyeong Kim, Jessie Zijia Xu, and Jacob O. Wobbrock. Understanding and reducing the challenges faced by creators of accessible online data visualizations. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’24, page 1–20. ACM, October 2024. doi: 10.1145/3663548.3675625. URL <https://doi.org/10.1145/3663548.3675625>. 6.2, 6.3.3, 6.8.2, 9.1, 10.2
- [179] Lei Shi, Idan Zelzer, Catherine Feng, and Shiri Azenkot. Tickers and talker: An accessible labeling toolkit for 3d printed models. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, CHI’16, page 4896–4907, New York, NY, USA, May 2016. ACM. ISBN 978-1-4503-3362-7. doi: 10.1145/2858036.2858507. URL <https://doi.org/10.1145/2858036.2858507>. Accessed: 2021-09-03. 4.3.1
- [180] Shneiderman. Direct manipulation: A step beyond programming languages. *Computer*, 16(8):57–69, August 1983. ISSN 0018-9162. doi: 10.1109/mc.1983.1654471. URL <https://doi.org/10.1109/mc.1983.1654471>. 6.3.1
- [181] Ben Shneiderman, Christopher Williamson, and Christopher Ahlberg. Dynamic queries: database searching by direct manipulation. In *Proceedings of the SIGCHI conference on Human factors in computing systems - CHI ’92*, CHI ’92, page 669–670. ACM Press, 1992. doi: 10.1145/142750.143082. URL <https://doi.org/10.1145/142750.143082>. 6.2
- [182] Aleksandra Slavkovic and Karen Cross. Novice heuristic evaluations of a complex interface. In *CHI ’99 extended abstracts on Human factors in computing systems - CHI ’99*, CHI ’99, page 304, New York, NY, USA, May 1999. Association for Computing Machinery, ACM Press. doi: 10.1145/632716.632902. URL <https://doi.org/10.1145/632716.632902>. 4.3.4
- [183] David Canfield Smith. *Pygmalion: A Computer Program to Model and Stimulate Creative*

- Thought*. Birkhäuser Basel, 1977. ISBN 9783034857444. doi: 10.1007/978-3-0348-5744-4. URL <https://doi.org/10.1007/978-3-0348-5744-4>. Reissue of the author’s 1975 Stanford University doctoral dissertation. 6.3.1
- [184] Volker Sorge. Polyfilling accessible chemistry diagrams. In *Lecture Notes in Computer Science*, page 43–50. Springer International Publishing, 2016. ISBN 9783319412641. doi: 10.1007/978-3-319-41264-1_6. URL https://doi.org/10.1007/978-3-319-41264-1_6. 4.3.1, 6.3.2
 - [185] Laura South and Michelle Borkin. Generating Seizure-Inducing Sequences with Interactive Visualizations. In *Proc. Of the IEEE VIS 2020 posters*. Center for Open Science, October 2020. doi: 10.31219/osf.io/85gwy. URL <https://doi.org/10.31219/osf.io/85gwy>. 4.3.1
 - [186] Laura South, David Saffo, and Michelle Borkin. Detecting and Defending Against Seizure-Inducing GIFs in Social Media. Technical report, OSF Preprints, January 2021. URL <https://doi.org/10.31219/osf.io/4kgu6>. Accessed: 2021-08-20. 4.3.1
 - [187] Katta Spiel, Kathrin Gerling, Cynthia L. Bennett, Emeline Brulé, Rua M. Williams, Jennifer Rode, and Jennifer Mankoff. Nothing about us without us: Investigating the role of critical disability studies in hci. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI ’20, page 1–8. ACM, April 2020. doi: 10.1145/3334480.3375150. URL <https://doi.org/10.1145/3334480.3375150>. 1.1, 10.2
 - [188] Maureen Stone, Danielle Albers Szafir, and Vidya Setlur. An engineering model for color difference as a function of size. *Color and Imaging Conference*, 22(1):253–258, November 2014. ISSN 2166-9635. doi: 10.2352/cic.2014.22.1.art00045. URL <http://dx.doi.org/10.2352/CIC.2014.22.1.art00045>. 4.1.1
 - [189] Sebastian Paul Suggate. Beyond self-report: Measuring visual, auditory, and tactile mental imagery using a mental comparison task. *Behavior Research Methods*, 56(8):8658–8676, September 2024. ISSN 1554-3528. doi: 10.3758/s13428-024-02496-z. URL <https://doi.org/10.3758/s13428-024-02496-z>. 1.1
 - [190] Cella M Sum, Rahaf Alharbi, Franchesca Spektor, Cynthia L Bennett, Christina N Harrington, Katta Spiel, and Rua Mae Williams. Dreaming disability justice in hci. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, CHI ’22, page 1–5. ACM, April 2022. doi: 10.1145/3491101.3503731. URL <http://dx.doi.org/10.1145/3491101.3503731>. 10.2
 - [191] Cella M. Sum, Franchesca Spektor, Rahaf Alharbi, Leya Breanna Baltaxe-Admony, Erika Devine, Hazel Anneke Dixon, Jared Duval, Tessa Eagle, Frank Elavsky, Kim Fernandes, Leandro S. Guedes, Serena Hillman, Vaishnav Kameswaran, Lynn Kirabo, Tamanna Motahar, Kathryn E. Ringland, Anastasia Schaadhardt, Laura Scheepmaker, and Alicia Williamson. Challenging ableism: A critical turn toward disability justice in hci. *XRDS: Crossroads, The ACM Magazine for Students*, 30(4):50–55, June 2024. ISSN 1528-4980. doi: 10.1145/3665602. URL <http://dx.doi.org/10.1145/3665602>. 10.2

- [192] Ivan E. Sutherland. Sketchpad: A man-machine graphical communication system. In *Proceedings of the May 21-23, 1963, spring joint computer conference on - AFIPS '63 (Spring)*, AFIPS '63 (Spring), page 329, New York, NY, USA, 1963. ACM Press. doi: 10.1145/1461551.1461591. URL <https://doi.org/10.1145/1461551.1461591>. 6.3.1
- [193] Henny Swan, Ian Pouncey, Heydon Pickering, and Léonie Watson. Inclusive Design Principles. <https://inclusivedesignprinciples.org/>, 2019. Accessed: 2021-09-07. 4.4.2
- [194] Sarit Felicia Anais Szpiro, Shafeka Hashash, Yuhang Zhao, and Shiri Azenkot. How people with low vision access computing devices: Understanding challenges and opportunities. In *Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '16, page 171–180, New York, NY, USA, October 2016. ACM. ISBN 9781450341240. doi: 10.1145/2982142.2982168. URL <https://doi.org/10.1145/2982142.2982168>. 4.3.1
- [195] Tim Teitelbaum and Thomas Reps. The cornell program synthesizer. *Communications of the ACM*, 24(9):563–573, 1981. doi: 10.1145/358746.358755. URL <https://doi.org/10.1145/358746.358755>. 6.3.1
- [196] Yuanyang (YY) Teng and Darren Gergle. Access is not enough: Toward developmental flourishing. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI EA '26, page 1–6. ACM, April 2026. doi: 10.1145/3772363.3798552. URL <http://dx.doi.org/10.1145/3772363.3798552>. 1.1
- [197] J. Thatcher. Screen reader/2: access to os/2 and the graphical user interface. In *Proceedings of the first annual ACM conference on Assistive technologies - Assets '94*, Assets '94, page 39–46. ACM Press, 1994. doi: 10.1145/191028.191039. URL <http://dx.doi.org/10.1145/191028.191039>. 6.3.1
- [198] John R Thompson, Jesse J Martinez, Alper Sarikaya, Edward Cutrell, and Bongshin Lee. Chart reader: Accessible visualization experiences designed with screen reader users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–18. ACM, April 2023. doi: 10.1145/3544548.3581186. URL <https://doi.org/10.1145/3544548.3581186>. 6.3.2, 6.3.3, 6.4
- [199] Alexandra To, Angela D. R. Smith, Dilruba Showkat, Adinawa Adjagbodjou, and Christina Harrington. Flourishing in the everyday: Moving beyond damage-centered design in hci for bipoc communities. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, DIS '23, pages 917–933. ACM, July 2023. doi: 10.1145/3563657.3596057. URL <https://doi.org/10.1145/3563657.3596057>. 10.2
- [200] US Government, 10x. Improving Accessibility in Data Visualizations (formerly USWDS Visualization Tool) on 10x. <https://trello.com/c/OOOres7a/63-improving-accessibility-in-data-visualizations-formerly-uswds-visualization-tool>, 2020. Accessed: 2022-03-04. 4.3.4
- [201] US Government, General Services Administration. Data visualizations U.S. Web Design System (USWDS). <https://designsystem.digital.gov/components/data-visualizations/>,

2021. Accessed: 2022-03-04. 4.3.4
- [202] Visa. Visa Chart Components. <https://github.com/visa/visa-chart-components>, 2022. Accessed: 2022-12-01. 4.3.2
- [203] W3C. Web content accessibility guidelines (wcag) 2.1. <https://www.w3.org/TR/WCAG21/>, 2018. Accessed: 2021-09-03. 4.3.1, 4.3.3
- [204] WAI. Understanding success criterion 2.1.1: keyboard. WCAG standard, W3C, 2017. URL <https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html>. Accessed: 2026-03-20, <https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html>. 6.3.2
- [205] WAI. Accessibility principles. WCAG Standard, W3C, 2019. URL <https://www.w3.org/WAI/fundamentals/accessibility-principles/>. Accessed: 2021-09-06. 4.2, 4.3.3
- [206] WAI. W3C Accessibility Guidelines (WCAG) 3.0. WCAG standard, W3C, 2021. Accessed: 2022-03-04. 4.8
- [207] WebAIM. Webaim: The WebAIM Million - An annual accessibility analysis of the top 1,000,000 home pages. <https://webaim.org/projects/million/#wcag>, 2021. Accessed: 2021-09-03. 4.5.1, 10.3
- [208] Simon Wheatcroft. Making Graphs And Plots Accessible For The Blind. <https://andadapt.com/making-graphs-and-plots-accessible-for-the-blind/>, June 2020. Accessed: 2021-09-06. 4.3.2
- [209] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, January 2010. ISSN 1537-2715. doi: 10.1198/jcgs.2009.07098. URL <https://doi.org/10.1198/jcgs.2009.07098>. 6.2
- [210] David Gray Widder and Richmond Wong. Thinking upstream: Ethics and policy opportunities in ai supply chains, 2023. URL <https://doi.org/10.48550/arxiv.2303.07529>. 1.2
- [211] David Gray Widder, Meredith Whittaker, and Sarah Myers West. Why ‘open’ ai systems are actually closed, and why this matters. *Nature*, 635(8040):827–833, November 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-08141-1. URL <https://doi.org/10.1038/s41586-024-08141-1>. 1.2
- [212] Leland Wilkinson. The grammar of graphics. In *Handbook of Computational Statistics*, page 375–414. Springer Berlin Heidelberg, December 2011. ISBN 9783642215513. doi: 10.1007/978-3-642-21551-3_13. URL https://doi.org/10.1007/978-3-642-21551-3_13. 6.2
- [213] Rua M. Williams and LouAnne E. Boyd. Prefigurative politics and passionate witnessing. In *Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’19, page 262–266. ACM, October 2019. doi: 10.1145/3308561.3355617. URL <http://dx.doi.org/10.1145/3308561.3355617>. 10.2
- [214] Rua M. Williams, Kathryn Ringland, Amelia Gibson, Mahender Mandala, Arne Maibaum, and Tiago Guerreiro. Articulations toward a crip hci. *Interactions*, 28(3):28–37, April

2021. ISSN 1558-3449. doi: 10.1145/3458453. URL <http://dx.doi.org/10.1145/3458453>. 10.2
- [215] Rua Mae Williams, Louanne Boyd, and Juan E. Gilbert. Counterventions: a reparative reflection on interventionist HCI. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–11. ACM, April 2023. doi: 10.1145/3544548.3581480. URL <http://dx.doi.org/10.1145/3544548.3581480>. 10.2
- [216] Brianna L. Wimer, Ritesh Kanchi, Kaija Frierson, Venkatesh Potluri, Ronald Metoyer, Jennifer Mankoff, Miya Natsuhara, and Matt X. Wang. Nonvisual support for understanding and reasoning about data structures, 2026. URL <https://doi.org/10.48550/arxiv.2601.19168>. 6.3.3
- [217] Langdon Winner. Do artifacts have politics? In *Computer Ethics*, page 177–192. Routledge, May 2017. ISBN 9781315259697. doi: 10.4324/9781315259697-21. URL <https://doi.org/10.4324/9781315259697-21>. 1.2
- [218] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. Ability-based design: Concept, principles and examples. *ACM Transactions on Accessible Computing*, 3(3):1–27, April 2011. ISSN 1936-7236. doi: 10.1145/1952383.1952384. URL <https://doi.org/10.1145/1952383.1952384>. 1.2, 4.4.3
- [219] Mintesnot Woldeamanuel and Andrew Kent. Measuring walk access to transit in terms of sidewalk availability, quality, and connectivity. *Journal of Urban Planning and Development*, 142(2), June 2016. ISSN 1943-5444. doi: 10.1061/(asce)up.1943-5444.0000296. URL [http://dx.doi.org/10.1061/\(ASCE\)UP.1943-5444.0000296](http://dx.doi.org/10.1061/(ASCE)UP.1943-5444.0000296). 10.3
- [220] Keke Wu, Emma Petersen, Tahmina Ahmad, David Burlinson, Shea Tanis, and Danielle Albers Szafr. Understanding Data Accessibility for People with Intellectual and Developmental Disabilities. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, page 1–16, New York, NY, USA, May 2021. Association for Computing Machinery, ACM. doi: 10.1145/3411764.3445743. URL <https://doi.org/10.1145/3411764.3445743>. Accessed: 2021-09-03. 4.3.1
- [221] Cindy Xiong, Lianne Van Weelden, and Steven Franconeri. The Curse of Knowledge in Visual Data Communication. *IEEE Transactions on Visualization and Computer Graphics*, 26(10):3051–3062, October 2020. ISSN 2160-9306. doi: 10.1109/TVCG.2019.2917689. URL <https://doi.org/10.1109/tvcg.2019.2917689>. Conference Name: IEEE Transactions on Visualization and Computer Graphics. 4.5.4
- [222] Rebecca Yeo and Karen Moore. Including disabled people in poverty reduction work: “nothing about us, without us”. *World Development*, 31(3):571–590, March 2003. ISSN 0305-750X. doi: 10.1016/s0305-750x(02)00218-8. URL [https://doi.org/10.1016/s0305-750x\(02\)00218-8](https://doi.org/10.1016/s0305-750x(02)00218-8). 1.1
- [223] Haixia Zhao, Catherine Plaisant, Ben Shneiderman, and Jonathan Lazar. Data sonification for users with visual impairment: A case study with georeferenced data. *ACM Transactions on Computer-Human Interaction*, 15(1):1–28, May 2008. ISSN 1557-7325. doi: 10.1145/1352782.1352786. URL <https://doi.org/10.1145/1352782.1352786>.

- [224] Jonathan Zong. Using real names of disabled participant-contributors to practice citational justice in accessibility. In *2025 IEEE Workshop on Accessible Data Visualization (AccessViz)*, page 30–33. IEEE, November 2025. doi: 10.1109/accessviz68666.2025.00011. URL <https://doi.org/10.1109/accessviz68666.2025.00011>. 6.4
- [225] Jonathan Zong, Crystal Lee, Alan Lundgard, JiWoong Jang, Daniel Hajas, and Arvind Satyanarayan. Rich screen reader experiences for accessible data visualization. *Computer Graphics Forum*, 41(3):15–27, June 2022. doi: 10.1111/cgf.14519. URL <https://doi.org/10.1111/cgf.14519>. 6.3.2, 6.3.3, 6.4
- [226] Jonathan Zong, Josh Pollock, Dylan Wootton, and Arvind Satyanarayan. Animated vegalite: Unifying animation with a grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):149–159, January 2023. ISSN 2160-9306. doi: 10.1109/tvcg.2022.3209369. URL <https://doi.org/10.1109/tvcg.2022.3209369>. 6.2
- [227] Jonathan Zong, Isabella Pedraza Pineros, Mengzhu (Katie) Chen, Daniel Hajas, and Arvind Satyanarayan. Umwelt: Accessible structured editing of multi-modal data representations. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI ’24, page 1–20. ACM, May 2024. doi: 10.1145/3613904.3641996. URL <https://doi.org/10.1145/3613904.3641996>. 6.3.3, 6.8.4